



Carreras de Ingeniería Electrónica e Ingeniería en Computación

Informe del Trabajo de Graduación

Desarrollo de un dispositivo de seguimiento ocular como herramienta de apoyo académico para personas con movilidad reducida

Autores:
Natalia Carolina Borbón
Facundo Ordóñez Nadra

Directores:
Federico Marcelo Vargas
Ezequiel Alberto Musa

Jurados:
Esteban Daniel Volentini
Martín Gustavo Juárez
Luis Roque Di Pinto

Este trabajo fue realizado en la Ciudad de San Miguel de Tucumán, entre noviembre de 2025 y agosto de 2026.

Resumen

Este trabajo desarrolló L.E.V.I., un dispositivo de seguimiento ocular de bajo costo, pensado para que estudiantes con limitaciones motrices puedan utilizarlo tanto para rendir evaluaciones de opción múltiple como para comunicarse con su entorno educativo. El dispositivo está compuesto por dos módulos basados en microcontroladores económicos y opera de manera autónoma, ya que genera su propia red inalámbrica sin necesidad de conexión a internet. A través del mismo, los docentes administran los cuestionarios, los tutores realizan el seguimiento académico del alumno y los compañeros se comunican con él.

Su valor principal es ofrecer una alternativa considerablemente más económica que las soluciones comerciales existentes y, a diferencia de estas, está diseñada específicamente para el ámbito académico. Esta propuesta amplía la autonomía y participación de estos estudiantes en las aulas.

En la presente memoria se detallan el proceso de selección del método de detección de la mirada, el diseño y la configuración de la red inalámbrica del sistema, el desarrollo del firmware, de la base de datos y de la interfaz web, el diseño y fabricación de las placas de circuito propias, las mediciones de consumo y autonomía realizadas, las conclusiones y trabajos futuros propuestos. Se aplicaron contenidos de sistemas embebidos, redes, diseño de circuitos impresos, bases de datos y desarrollo web correspondientes a las carreras de Ingeniería electrónica e Ingeniería en computación.

Índice general

Resumen **I**

1. Introducción General **1**
1.1. Introducción . . . 1
1.2. Estado del arte . . . 1
1.3. Motivación . . . 2
1.4. Alcance y objetivos . . . 3

2. Introducción Específica **4**
2.1. Seguimiento Ocular (Eye-tracking) . . . 4
2.2. Procesamiento de Imágenes y Visión por Computadora . . . 4
2.2.1. Conceptos fundamentales y biblioteca OpenCV . . . 4
2.2.2. Coincidencia de patrones (Template Matching) . . . 5
2.3. Redes Neuronales Artificiales . . . 6
2.3.1. Definición y funcionamiento . . . 6
2.3.2. Tipos y arquitecturas de redes neuronales . . . 8
2.4. Sistemas Embebidos y Control Gráfico . . . 8
2.4.1. Microcontroladores y familia ESP32 . . . 8
2.4.2. Interfaces gráficas embebidas y biblioteca LVGL . . . 9
2.5. Redes y Comunicaciones . . . 10
2.5.1. Redes locales Wi-Fi . . . 10
2.5.2. Protocolos de comunicación y transmisión de video . . . 11
2.6. Arquitectura de una aplicación web . . . 12
2.6.1. Modelo Cliente/Servidor . . . 12
2.6.2. Patrón de diseño Modelo-Vista-Controlador (MVC) . . . 12
2.6.3. Frontend . . . 13
2.6.4. Backend . . . 14
2.6.5. Base de datos . . . 15

3. Diseño e implementación **18**
3.1. Requisitos y especificaciones del sistema . . . 18
3.1.1. Requisitos funcionales . . . 18
3.1.2. Requisitos técnicos . . . 19
3.1.3. Restricciones de diseño . . . 20
3.2. Detector ocular . . . 20
3.2.1. Procesamiento local como planteamiento inicial . . . 21
3.2.2. Procesamiento remoto por coordenadas . . . 29
3.2.3. Template Matching como solución adoptada . . . 32
3.3. Arquitectura del sistema . . . 34
3.3.1. Funciones de cada módulo . . . 34
3.3.2. Interacción entre módulos . . . 35
3.3.3. Mapa de memoria . . . 36

3.4. Microcontroladores y periféricos . . . 37
3.4.1. Elección de la familia ESP32 . . . 37
3.4.2. Módulo de captura . . . 38
3.4.3. Módulo central . . . 39
3.5. Red y comunicación . . . 40
3.5.1. Red Wi-Fi local . . . 40
3.5.2. Acceso a la aplicación . . . 41
3.5.3. Transmisión de video . . . 41
3.5.4. Comunicación con el módulo central . . . 41
3.6. Aplicación web . . . 42
3.6.1. Base de datos . . . 42
3.6.2. Backend . . . 45
3.6.3. Frontend . . . 50
3.7. Interfaz de usuario táctil . . . 54
3.7.1. Bibliotecas y controladores . . . 54
3.7.2. Funcionamiento . . . 54
3.7.3. Vistas . . . 55
3.8. Alimentación . . . 56
3.8.1. Requisitos previos . . . 56
3.8.2. Soluciones propuestas . . . 56
3.8.3. Descripción detallada del sistema elegido . . . 59
3.9. Diseño físico . . . 69
3.9.1. Montaje . . . 69
3.10. Costo total de desarrollo . . . 71

4. Conclusiones **72**
4.1. Resultados obtenidos . . . 72
4.2. Comparación con productos similares . . . 72
4.3. Trabajo futuro . . . 73

Índice de figuras

1.1. Tobii Glasses 2

2.1. Representación gráfica del Eye-Tracking 4
2.2. Imagen ilustrativa del proceso de *Template Matching* 5
2.3. Imagen ilustrativa de una matriz de resultados de *Template Matching* 6
2.4. El proceso del *Machine Learning* 7
2.5. Diagrama simplificado de una red neuronal 7
2.6. Algunas placas de desarrollo de la familia *ESP32* 8
2.7. Placa de desarrollo *ESP32-CYD* (vista frontal y trasera) 10
2.8. Red Wi-Fi con un microcontrolador ESP32 como punto de acceso 11
2.9. El sistema *Cliente/Servidor* 12
2.10. Patrón de diseño Modelo-Vista-Controlador (MVC) 13
2.11. Arquitectura del frontend: HTML (su estructura), CSS (cómo se ve) y JavaScript (cómo funciona) 14

3.1. Diferencias entre procesamiento local y remoto 20
3.2. Distintas partes del ojo y su nivel de oscuridad 22
3.3. El *glint* en el centro de la pupila 23
3.4. El *glint* no sigue a la pupila 23
3.5. Pantalla de creación de modelos de *Edge Impulse* 24
3.6. Reconocimiento de vegetales con *EdgeImpulse* 24
3.7. Entorno de *Google Colab* 26
3.8. Una muestra de las imágenes utilizadas para el dataset 27
3.9. Planilla con etiquetado de imágenes 27
3.10. Resultados del modelo desarrollado en *Google Colab*. Con un punto verde se indica el valor etiquetado, mientras que la cruz roja muestra la posición que la red predice 28
3.11. Detección de rostro con *MediaPipe* 30
3.12. Facciones del rostro en *MediaPipe* 30
3.13. Facciones del rostro en *WebGazer* 30
3.14. Placa *ESP32-CAM* con su cámara. 38
3.15. Cámaras OV2640 con distintos ángulos de visión y longitud de conector 39
3.16. Diagrama de pines del módulo *TP4056* 61
3.17. Corriente vs voltaje de carga 62
3.18. Circuito típico de aplicación del *TP4056* 62
3.19. Circuito elevador Boost 63
3.20. Circuito interno del chip *MT3608* 64
3.21. Gráfico explicativo del comportamiento del amplificador de error 65
3.22. Modulación por ancho de pulso (PWM) 65
3.23. Diagrama de pines del módulo *MT3608* 66
3.24. Regulación de carga del *MT3608* 66
3.25. Frecuencia vs Tensión de entrada en el chip *MT3608* 67
3.26. Eficiencia vs Tensión de entrada en el chip *MT3608* 67

3.27. Circuito comercial de los módulos *MT3608* 68
3.28. Esquema de conexiones para la PCB del circuito capturador 69
3.29. Esquema simplificado de la PCB del circuito capturador 69
3.30. Esquema simplificado de la PCB del circuito servidor 70
3.31. Layout de la PCB del circuito capturador 70
3.32. Layout de la PCB del circuito servidor 71

Índice de Tablas

3.1. Costo de placas compatibles con cámaras, a octubre de 2026 38
3.2. Tamaño del código compilado con *React* y con *Preact* 50
3.3. Análisis de costos de todos los métodos de alimentación propuestos (octubre 2026) 59

Capítulo 1

Introducción General

En este capítulo se presenta el dispositivo de seguimiento ocular desarrollado. Se detallan la motivación que impulsó el proyecto, el estado actual de esta tecnología, los objetivos planteados y el alcance definido para el sistema.

1.1. Introducción

En el ámbito educativo, los estudiantes con movilidad reducida enfrentan dificultades tanto para ser evaluados académicamente como para comunicarse con docentes, tutores y compañeros. Esta situación limita su autonomía y participación en las clases, principalmente porque las instituciones no cuentan con herramientas adaptadas a sus posibilidades físicas.

Frente a esto, la tecnología de seguimiento ocular (*eye-tracking*) se presenta como una alterativa práctica para atacar este problema. Esta herramienta permite a un usuario interactuar con un sistema mediante la mirada. De este modo, se elimina la necesidad de recurrir al movimiento de los brazos o a la asistencia de terceros.

Sin embargo, los sistemas de seguimiento ocular comerciales presentan dos grandes inconvenientes. Tienen costos muy elevados y no están enfocados en la dinámica educativa. En la práctica, no existen soluciones accesibles que permitan a los profesores evaluar a alumnos con estas condiciones y que, al mismo tiempo, faciliten su comunicación en el ámbito de estudio.

Para dar respuesta a esta necesidad, este trabajo presenta el desarrollo de un dispositivo de seguimiento ocular de bajo costo, diseñado para que un estudiante con movilidad reducida pueda resolver cuestionarios de opción múltiple y comunicarse con su entorno académico.

1.2. Estado del arte

La tecnología del *eye-tracking* no es una novedad. Desde hace décadas, múltiples empresas han implementado sistemas de detección ocular para diversas aplicaciones. El caso más destacado es el de la compañía *Tobii AB*, pionera en el desarrollo de *hardware* especializado. Entre sus productos se encuentran los *Tobii Pro Glasses*, anteojos diseñados para analizar el comportamiento visual del usuario en interacción libre con su entorno físico (figura 1.1). A su vez, alternativas como el *Tobii Pro Fusion* y el *Tobii Pro Spectrum* consisten en sistemas fijos y más robustos, orientados a la investigación científica de alta precisión.



Figura 1.1: Tobii Glasses

Por otro lado, la firma *EyeComTec* desarrolla productos exclusivamente de software como *ECT-Keyboard*, *ECTMouse* y *ECTTracker*, que permiten controlar computadoras con tan solo el movimiento de los ojos.

La compañía *LC Technologies* ofrece el sistema *Eyegaze Edge*, un equipo que integra hardware y software dedicados. Está diseñado como una tecnología de asistencia médica de alta precisión y comunicación alternativa para personas con discapacidades motrices severas.

También se puede mencionar a *EyeHarp*, un instrumento musical digital que permite tocar música mediante el movimiento de los ojos. Funciona principalmente con rastreadores oculares *Tobii*, aunque también puede utilizarse con otros dispositivos que controlan el puntero del mouse. Está diseñado especialmente para personas con discapacidades físicas y ofrece una forma inclusiva de expresión musical.

Todos estos desarrollos demuestran que el seguimiento ocular es una tecnología sumamente avanzada y madura en la actualidad. Sin embargo, el presente proyecto se enfoca específicamente en el ámbito educativo, ofreciendo además ciertas ventajas prácticas frente a estos dispositivos comerciales que se detallarán más adelante.

1.3. Motivación

Todos los ejemplos de sistemas que utilizan *eye-tracking* expuestos anteriormente, ya sea por hardware, software o una combinación de ambos, son herramientas de calidad que pueden aplicarse en una gran variedad de campos. Han aportado una amplia gama de dispositivos versátiles que ayudan tanto a investigaciones científicas y de marketing, como también a facilitar acciones cotidianas.

Sin embargo, el principal problema de estos equipos es su elevado precio. El uso de tecnologías de tan alto nivel implica, a su vez, altos costos de desarrollo. En consecuencia, muchos de ellos no resultan accesibles para usuarios en una situación económica desfavorable.

Otro detalle observado durante la investigación es que ninguna de estas soluciones está centrada exclusivamente en el ámbito educativo, a pesar de su versatilidad. Por lo tanto, se necesita una alternativa más optimizada, intuitiva y sencilla para resolver este problema.

Por todo lo expuesto se propone este proyecto. Se trata de una solución que mantiene la ergonomía y la portabilidad de los dispositivos mencionados y que, al mismo tiempo, reduce los costos de producción. Esto último se logra mediante el uso de hardware y software de bajos recursos. Además, su orientación es exclusivamente educativa.

Este equipo tiene la intención de ser utilizado por instituciones académicas y familias de escasos recursos. De este modo, se garantiza una accesibilidad mucho mayor que la de cualquier otro producto de alta tecnología.

1.4. Alcance y objetivos

El objetivo general de este proyecto es desarrollar un dispositivo de bajo costo basado en microcontroladores económicos. Debe permitir que personas con movilidad reducida sean evaluadas mediante exámenes de opción múltiple y se puedan comunicar a través del seguimiento ocular (*eye-tracking*). Además, tiene que funcionar de forma autónoma y sin conexión a internet (*offline*).

Para alcanzar esta meta se plantean los siguientes objetivos específicos:

- Diseñar un dispositivo económico, ergonómico y portable, que funcione sin necesidad de acceso a internet.
- Brindar una interfaz intuitiva y de fácil uso, accesible desde cualquier equipo con navegador web.
- Permitir que los profesores gestionen cuestionarios de opción múltiple personalizados.
- Posibilitar que los tutores realicen un seguimiento académico integral del alumno.
- Facilitar un canal de comunicación directo para que docentes, tutores y compañeros puedan interactuar con el estudiante mediante preguntas informales.
- Implementar un sistema de registro e inicio de sesión, con una base de datos local para el almacenamiento de usuarios, cuestionarios y respuestas.
- Realizar el procesamiento de la imagen del ojo del alumno mediante un microcontrolador embebido en el dispositivo.

En cuanto al alcance, el proyecto comprende el desarrollo integral de un dispositivo autónomo capaz de capturar la imagen ocular del alumno, procesar la dirección de la mirada y traducir esa información en acciones concretas sobre una interfaz web. El sistema está delimitado para operar en un entorno local, sin depender de conexión a internet ni de infraestructura de red externa. El acceso a la plataforma se realiza mediante cualquier navegador web compatible.

Respecto a los usuarios del sistema, se delimitan los siguientes roles y permisos:

- Alumno: interactúa con el sistema mediante la mirada para resolver las evaluaciones de opción múltiple y responder a preguntas informales del entorno.
- Profesor: encargado de crear, editar y administrar sus cuestionarios académicos.
- Tutor: capacitado para consultar todos los cuestionarios, junto a las respuestas y los resultados obtenidos por el alumno.
- Compañero: habilitado para realizar preguntas esporádicas de opción múltiple con el fin de comunicarse con el alumno. Estas preguntas no quedan registradas en el sistema.

Capítulo 2

Introducción Específica

En este capítulo se explorarán en términos muy generales los distintos conceptos, tecnologías, arquitecturas, paradigmas y técnicas que resultaron útiles para el desarrollo del trabajo.

2.1. Seguimiento Ocular (Eye-tracking)

El *eye-tracking*, que se traduce literalmente como *seguimiento ocular*, es el proceso tecnológico que permite medir y registrar el punto de la mirada de una persona o el movimiento de sus ojos en relación con la cabeza. Para ello, comúnmente se utiliza como punto de referencia el centro de la pupila. A diferencia del seguimiento facial (que analiza músculos, cejas o expresiones), el *eye-tracking* se concentra estrictamente en la forma y movimientos del globo ocular (Fig. 2.1).



Figura 2.1: Representación gráfica del Eye-Tracking

Si bien esta tecnología cuenta con décadas de investigación y ha sido ampliamente utilizada en laboratorios de psicología, neuromarketing y análisis de usabilidad, el desarrollo de hardware más accesible ha permitido expandir sus aplicaciones. Uno de estos usos directos es la *interacción basada en la mirada* (*gaze interaction*), donde los datos del movimiento ocular se utilizan para manipular un entorno digital.

En el caso del presente trabajo, orientado a la inclusión en el ámbito educativo, el seguimiento ocular se emplea para detectar hacia cuál de las opciones de respuesta a una pregunta dirige la mirada el alumno. Estas opciones se disponen de manera clara en una pantalla. Luego esa información se procesa para registrar la selección correspondiente. De este modo, el estudiante puede responder cuestionarios de opción múltiple sin necesidad de manipular un cursor ni de contar con destreza física.

2.2. Procesamiento de Imágenes y Visión por Computadora

2.2.1. Conceptos fundamentales y biblioteca OpenCV

El procesamiento de imágenes puede definirse como una técnica que modifica una imagen de entrada, mediante algoritmos matemáticos, para obtener otra distinta a la salida. Por ejemplo,

permite cambiar su resolución, recortarla, modificar su paleta de colores y varias otras operaciones. En este contexto, dichas modificaciones se realizan con el fin de acondicionar la imagen para posteriores análisis computacionales.

Por otro lado, la visión computacional es el análisis que realiza una máquina sobre una imagen (usualmente procesada con anterioridad), para extraer características, interpretarlas y aprender de ellas. De este modo se simula la percepción visual humana. El *Eye-Tracking*, descrito en secciones anteriores, es un claro ejemplo de un proceso de visión computacional.

Lograr que una máquina interprete de manera autónoma el comportamiento de las imágenes es una tarea compleja, motivo por el cual se emplean herramientas especializadas que facilitan el proceso. Una de las más destacadas es *OpenCV (Open Source Computer Vision Library)*, la biblioteca de código abierto más popular e importante en el campo del análisis visual. Fue creada por la empresa *Intel* y cuenta con más de 2500 algoritmos matemáticos optimizados que sirven tanto para el procesamiento de imágenes como para la visión computacional. Entre sus principales capacidades destacan la manipulación estructural, la detección de características geométricas (como bordes, esquinas o líneas), la coincidencia de patrones, la detección de objetos y el análisis de movimiento.

2.2.2. Coincidencia de patrones (Template Matching)

La coincidencia de patrones, conocida en inglés como *Template Matching*, es una técnica de visión computacional diseñada para encontrar la ubicación de una imagen pequeña, denominada molde o plantilla (*template*), dentro de una imagen de mayor tamaño, denominada imagen de entrada (*input*). El proceso consiste en deslizar la plantilla píxel por píxel sobre la imagen de entrada, de izquierda a derecha y de arriba hacia abajo.

El recorrido comienza con las coordenadas X=0 e Y=0 de ambas imágenes alineadas, es decir, con sus esquinas superiores izquierdas superpuestas. En esa posición se realiza un cálculo matemático. Luego, la plantilla se desplaza un píxel hacia la derecha, hasta las coordenadas X=1 e Y=0 y el cálculo se vuelve a realizar. Este desplazamiento continúa hasta completar la primera fila. A continuación, la plantilla se alinea con la imagen de entrada una fila más abajo (X=0, Y=1) y el recorrido vuelve a comenzar. Esta secuencia se repite en cada fila hasta cubrir por completo la imagen de entrada (Fig. 2.2).

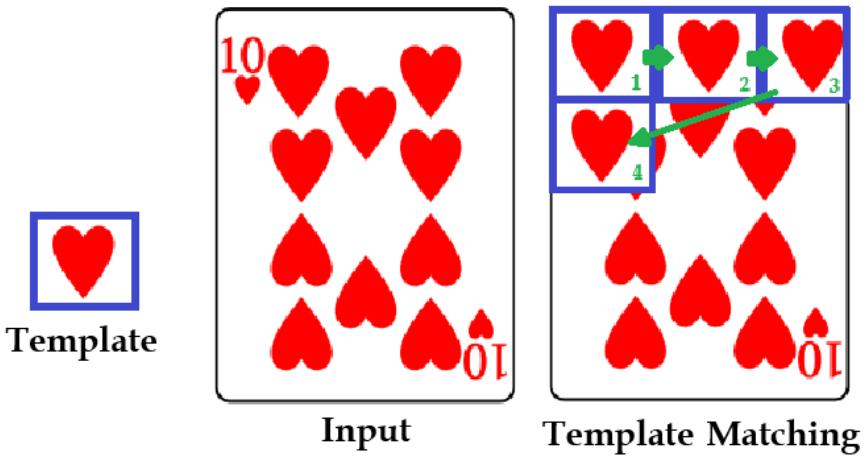


Figura 2.2: Imagen ilustrativa del proceso de *Template Matching*

En cada posición de la plantilla, el algoritmo aplica una fórmula estadística denominada *coeficiente de correlación normalizado con media cero*, presentada a continuación:

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') - \bar{T}) \cdot (I(x+x',y+y') - \bar{I})}{\sqrt{\sum_{x',y'}(T(x',y') - \bar{T})^2 \cdot \sum_{x',y'}(I(x+x',y+y') - \bar{I})^2}} \tag{2.1}$$

En esta fórmula, T corresponde a la imagen de la plantilla e I a la imagen de entrada. Ambas son matrices cuyos elementos representan sus píxeles, y su valor depende del formato de las imágenes. Los valores (x, y) son las coordenadas de referencia, en las que se alinean la plantilla y la imagen de entrada. Los valores (x', y'), en cambio, son las coordenadas de los píxeles dentro de la plantilla.

Esta expresión se basa en el *coeficiente de correlación de Pearson*. Evalúa qué tan similar es la plantilla respecto de la sección de la imagen de entrada sobre la que está superpuesta y, en función de eso, otorga un valor flotante entre $-1{,}0$ y $+1{,}0$. Un valor de $+1{,}0$ indica que la plantilla es idéntica a la porción de imagen analizada. Un valor de $-1{,}0$ significa que la plantilla tiene los colores opuestos a los de la imagen, mientras que un valor de 0 refleja que no existe correlación alguna.

El resultado tras todo el recorrido de la plantilla es una matriz bidimensional de valores. El punto con el valor más alto indica la coordenada en la que el algoritmo encontró la mayor coincidencia (Fig. 2.3).

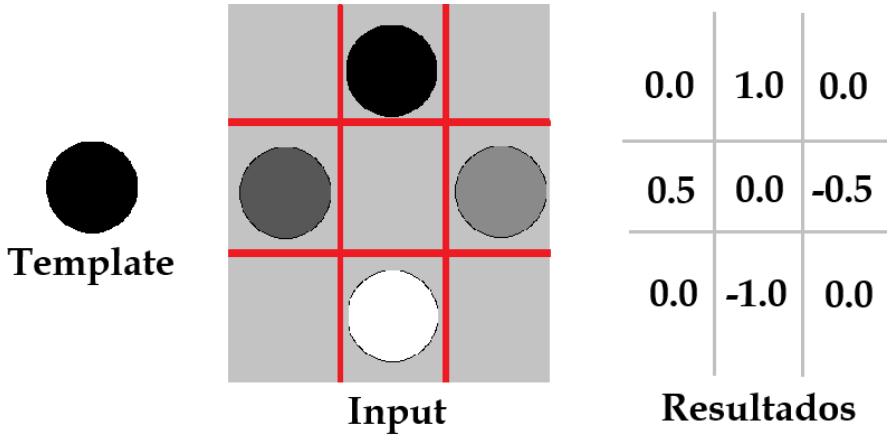


Figura 2.3: Imagen ilustrativa de una matriz de resultados de *Template Matching*

Esta matriz permite obtener no solo la ubicación de la coincidencia con la plantilla, sino también su nivel de semejanza.

2.3. Redes Neuronales Artificiales

2.3.1. Definición y funcionamiento

Para comprender este concepto es necesario definir previamente el *Machine Learning (ML)* o *Aprendizaje automático*. Esta rama de las ciencias de la computación permite a los sistemas aprender y resolver problemas sin ser programados de forma tradicional. El proceso consiste en suministrar datos a una computadora para que construya un modelo predictivo basado en esa experiencia (Fig. 2.4).

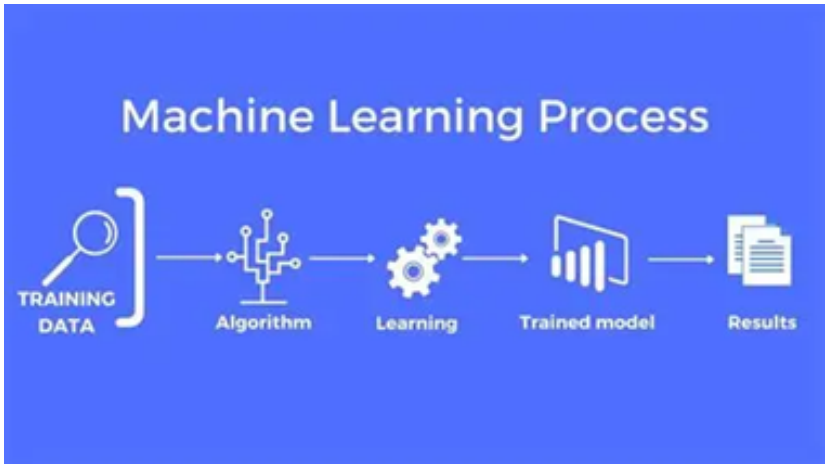


Figura 2.4: El proceso del *Machine Learning*

Dentro de los modelos de ML, algunos de los más eficaces son las Redes Neuronales Artificiales. Reciben este nombre debido a que su estructura (basada en celdas o nodos interconectados para el procesamiento de información) se inspira directamente en el comportamiento biológico del sistema nervioso.

Estas redes organizan sus neuronas en tres niveles jerárquicos principales. La capa de entrada recibe los datos en crudo, como los píxeles de una imagen. Las capas ocultas constituyen el núcleo de procesamiento y detectan desde características básicas hasta patrones complejos. La capa de salida entrega el resultado final, como las coordenadas de una pupila (Fig. 2.5).

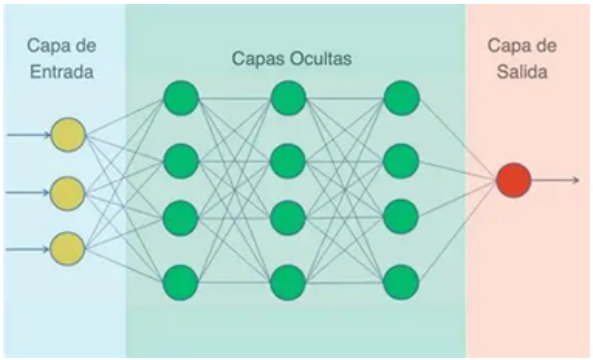


Figura 2.5: Diagrama simplificado de una red neuronal

El aprendizaje del modelo se logra mediante el entrenamiento, un proceso iterativo donde la red evalúa datos conocidos y ajusta su matemática interna para minimizar el margen de error. Durante estos ciclos, cada neurona ejecuta una ecuación fundamental:

$$z = \sum w_i x_i + b \tag{2.2}$$

Donde x_i representa el dato de entrada individual, w_i es el peso (la importancia relativa asignada a esa entrada específica) y b es el sesgo o bias (una constante de adaptación algorítmica). El resultado z es evaluado finalmente por una función de activación, la cual determina con qué intensidad la neurona transmite la señal hacia la siguiente capa.

2.3.2. Tipos y arquitecturas de redes neuronales

La arquitectura del modelo varía estrictamente según la naturaleza del problema a resolver. Se distinguen tres categorías fundamentales:

- Prealimentadas (*Feedforward*): La información viaja de forma unidireccional (de entrada a salida). Son ideales para problemas sencillos con datos livianos que no requieren memoria temporal.
- Convolucionales (*CNN*): Utilizan filtros matemáticos para analizar datos espaciales por secciones. Son la arquitectura por excelencia para procesar imágenes y video en tiempo real.
- Recurrentes (*RNN*): Poseen bucles internos que les otorgan memoria secuencial, perfectas para contextos de análisis temporal (como procesamiento de lenguaje natural o secuencias largas de video).

En función de los requerimientos de este proyecto, se planteó como hipótesis inicial el entrenamiento e implementación de un modelo convolucional liviano, cuyo desempeño y viabilidad sobre hardware de bajos recursos se analizan en el Capítulo 3.

2.4. Sistemas Embebidos y Control Gráfico

2.4.1. Microcontroladores y familia ESP32

Un microcontrolador (MCU) es un sistema informático completo encapsulado en un único circuito integrado. Su estructura reúne un microprocesador central junto con bloques de memoria interna (tanto volátil para datos como no volátil para el programa) y múltiples periféricos. Estos módulos adicionales pueden incluir temporizadores, protocolos de comunicación serial y puertos de entrada/salida de propósito general (GPIO), lo que le permite interactuar directamente con el entorno físico y dispositivos externos de manera autónoma.

Una familia de placas de desarrollo que ha ganado enorme popularidad en el ámbito de los sistemas embebidos y el Internet de las Cosas (IoT) son los microcontrolador ESP32 (Fig. 2.6). Creados por la compañía Espressif Systems, estos dispositivos surgieron como una solución de bajo costo y alta eficiencia energética diseñada para suplir la creciente necesidad de conectividad inalámbrica en proyectos electrónicos. A diferencia de las placas más tradicionales, el ESP32 destaca por integrar módulos de Wi-Fi y Bluetooth directamente dentro del mismo chip.

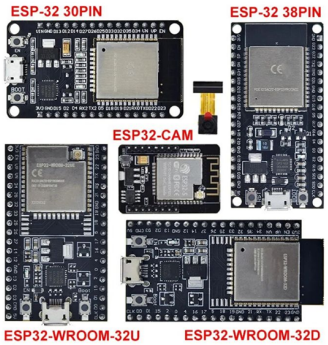


Figura 2.6: Algunas placas de desarrollo de la familia *ESP32*

Al estar generalmente equipado con un procesador de doble núcleo y una memoria considerablemente mayor que la de sus predecesores, permite manejar tareas más complejas, alojar

servidores web locales y procesar datos a mayor velocidad. Con todo esto logra un equilibrio entre rendimiento, conectividad y tamaño reducido.

2.4.2. Interfaces gráficas embebidas y biblioteca LVGL

El desarrollo de interfaces gráficas de usuario (GUI, por sus siglas en inglés) en sistemas embebidos presenta desafíos asociados a las restricciones de memoria RAM, capacidad de cómputo y consumo de energía de los microcontroladores. A diferencia de los entornos convencionales, donde el sistema operativo gestiona los recursos visuales mediante unidades de procesamiento gráfico dedicadas (GPU), en un microcontrolador el renderizado debe ejecutarse de forma eficiente. Así se garantiza fluidez visual y respuesta táctil en tiempo real sin sobrecargar la CPU.

Para resolver esta problemática, se recurre a bibliotecas gráficas optimizadas para hardware limitado, entre las que destaca *LVGL* (*Light and Versatile Graphics Library*). LVGL es una herramienta de código abierto escrita en lenguaje C, diseñada para operar con requerimientos mínimos de memoria. Implementa un modelo orientado a objetos basado en elementos visuales predefinidos (*widgets*) como botones, etiquetas, listas, deslizadores y contenedores, los cuales pueden personalizarse mediante estilos y animaciones.

La arquitectura de LVGL se fundamenta en el desacoplamiento entre la lógica de la interfaz y el hardware subyacente. Esto se logra a través de dos componentes principales:

- Controlador de pantalla (*Display Driver*): transfiere los píxeles renderizados desde el búfer de dibujo interno de la biblioteca hacia el circuito de control del panel (por ejemplo, a través de interfaces SPI o paralelas). Además, admite técnicas de doble búfer (*double buffering*) para evitar parpadeos en pantalla.
- Controlador de dispositivo de entrada (*Input Device Driver*): gestiona la lectura y calibración de periféricos interactivos, como pantallas táctiles resistivas o capacitivas, teclados y codificadores rotativos.

En el ámbito del hardware comercial de bajo costo, esta combinación de pantalla, interfaz táctil y microcontrolador suele integrarse en módulos compactos denominados comúnmente *Cheap Yellow Display* (CYD). Estas placas incorporan un SoC ESP32, un panel TFT y un controlador táctil resistivo (Fig. 2.7). El uso de LVGL sobre estas plataformas permite ejecutar interfaces interactivas fluidas directamente en el dispositivo embebido de manera autónoma.



Figura 2.7: Placa de desarrollo *ESP32-CYD* (vista frontal y trasera)

2.5. Redes y Comunicaciones

2.5.1. Redes locales Wi-Fi

El estándar IEEE 802.11, conocido comúnmente como Wi-Fi, define los protocolos para la implementación de redes de área local inalámbricas (WLAN). La topología de estas redes se estructura a través de dos modos de operación:

- Modo Estación (*Station* - STA): en esta configuración, el nodo inalámbrico actúa como cliente. Su función consiste en explorar el espectro de radiofrecuencia y detectar una red existente a través de su identificador (SSID). Luego se autentica mediante los protocolos de seguridad correspondientes (como WPA2/WPA3) y se asocia a un punto de acceso central para integrarse al tráfico de dicha red.
- Modo Punto de Acceso (*Access Point* - AP o SoftAP): en este modo, el dispositivo asume el rol de concentrador e iniciador de la red. Emite periódicamente tramas de sincronización (*beacon frames*) para anunciar la presencia de la red, autentica a los nodos que solicitan conexión y gestiona la tabla de clientes asociados. Así, crea una infraestructura de red local independiente.

En el marco de este trabajo, uno de los módulos del sistema se configura en modo Access Point para generar una red local cerrada e independiente, mientras que los dispositivos de los usuarios se conectan a dicha red en modo Station (Fig. 2.8). Esto permite la comunicación directa entre todos los componentes sin depender de un router externo ni de acceso a internet.

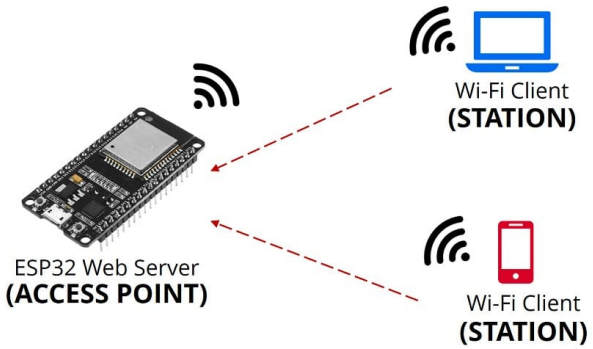


Figura 2.8: Red Wi-Fi con un microcontrolador ESP32 como punto de acceso

2.5.2. Protocolos de comunicación y transmisión de video

La comunicación en una red de datos se rige por protocolos que definen cómo se estructura, transmite y recibe la información. Dentro de estas reglas, el direccionamiento lógico permite identificar a cada dispositivo de manera unívoca para asegurar que los datos alcancen su destino.

- Protocolo IP (*Internet Protocol*): estructura los datos en paquetes que contienen las direcciones de origen y destino para su entrega en la red. En redes locales la dirección de cada dispositivo se establece mediante dos modalidades:
  - Asignación estática: la dirección se define de manera manual en el equipo y permanece fija a lo largo del tiempo, lo que permite localizar siempre al dispositivo en esa misma ubicación.
  - Asignación dinámica: el protocolo de configuración dinámica de host (DHCP, *Dynamic Host Configuration Protocol*) asigna una dirección disponible de forma automática al asociarse el equipo a la red, lo que agiliza la incorporación de nuevos dispositivos sin requerir configuraciones individuales.
- Resolución de nombres mediante mDNS (*Multicast Domain Name System*): permite que los dispositivos de una red local se identifiquen mediante nombres de dominio legibles por humanos, en lugar de direcciones IP numéricas.
- Protocolo HTTP (*Hypertext Transfer Protocol*): opera bajo el modelo cliente-servidor mediante un esquema de peticiones y respuestas. Para asegurar que la información no sufra pérdidas ni alteraciones en su secuencia, HTTP utiliza TCP (*Transmission Control Protocol*), el cual establece una conexión confiable previa al intercambio de datos. A través de este mecanismo se transfieren páginas web, estilos y datos estructurados. Estos últimos suelen enviarse en formato JSON (*JavaScript Object Notation*). Este formato organiza la información como texto, en pares que asocian un nombre con un valor. Además, admite listas y permite agrupar elementos dentro de otros. Al tratarse de texto plano, programas escritos en lenguajes distintos pueden generarlo e interpretarlo sin dificultad.
- Transmisión continua de video (*Streaming* MJPEG): en lugar de realizar una petición HTTP por cada imagen, la técnica MJPEG (*Motion* JPEG) mantiene una única conexión abierta para enviar una secuencia ininterrumpida de fotogramas JPEG independientes delimitados por un separador. El cliente actualiza la imagen en pantalla a medida que recibe cada cuadro. De este modo, se logra una visualización de video fluida en tiempo real.

Imagen de protocolos con circulitos para señalar (Natty)

2.6. Arquitectura de una aplicación web

En el desarrollo de software, las aplicaciones web suelen combinar el modelo cliente/servidor con el patrón MVC. Con este enfoque, la aplicación se divide en tres partes, que son el *frontend*, el *backend* y la base de datos. En esta sección se explican primero el modelo cliente/servidor y el patrón MVC, y luego cada una de estas tres partes.

2.6.1. Modelo Cliente/Servidor

La arquitectura cliente/servidor (Fig. 2.9) es un modelo de diseño de software y redes en el cual las tareas a realizar se distribuyen entre un proveedor de recursos denominado *Servidor* y un demandante de dichos recursos denominado *Cliente*. Se basa en una dinámica de petición/respuesta entre ambas entidades, las cuales se detallan a continuación:

- El cliente: es el programa que interactúa con el usuario final. Su función principal es elaborar y enviar peticiones de datos o ejecución de tareas a través de una red. Luego, recibe, formatea y presenta los resultados obtenidos en la interfaz de usuario. Es el componente que siempre inicia la comunicación.
- El servidor: es el programa en ejecución continua diseñado para escuchar, recibir y procesar las peticiones de uno o múltiples clientes. Su función es la de administrar los recursos y servicios para devolver al cliente la respuesta solicitada.

La separación entre cliente y servidor es lógica. Ambos pueden ejecutarse en equipos distintos o en un mismo equipo.

Esta modalidad es muy utilizada en los sistemas modernos para centralizar la carga de trabajo en el servidor. De este modo, los clientes no necesitan su propia base de datos ni sus propios recursos. De requerirlos, se incrementarían significativamente los costos y la complejidad de hardware en cada terminal. Además, los procesos se volverían ineficientes.

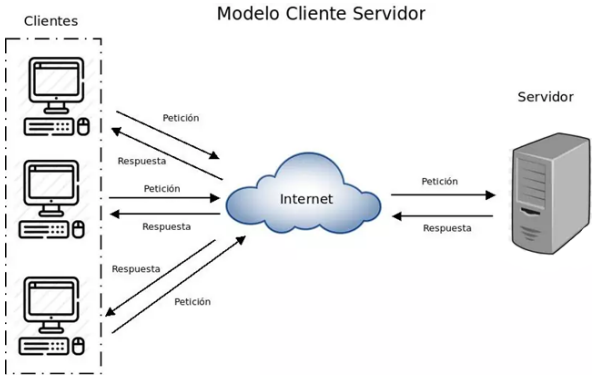


Figura 2.9: El sistema *Cliente/Servidor*

2.6.2. Patrón de diseño Modelo-Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador (*Model-View-Controller* - MVC) es una estrategia de diseño de software orientada a desacoplar la lógica de negocio, la gestión de datos y la interfaz de usuario en tres componentes con responsabilidades bien definidas (Fig. 2.10):

- Modelo (*Model*): representa la estructura de los datos del sistema y gestiona las reglas de negocio. Se encarga de interactuar con los mecanismos de almacenamiento y persistencia

para consultar, insertar, modificar o eliminar registros. Así, asegura la integridad de la información sin vincularse con la interfaz gráfica.

- Vista (*View*): constituye la capa de presentación con la que interactúa el usuario. Su función es recibir los datos procesados y mostrarlos en un formato comprensible. Además, captura los eventos y acciones del usuario para transferirlos al Controlador.
- Controlador (*Controller*): actúa como intermediario entre el Modelo y la Vista. Procesa las solicitudes y eventos originados en la Vista, ejecuta la lógica correspondiente, solicita las operaciones necesarias al Modelo y actualiza la Vista con el resultado.

Esta separación de responsabilidades favorece la modularidad, simplifica el mantenimiento del sistema y permite modificar la interfaz visual o el almacenamiento de datos de forma independiente sin alterar la lógica central de la aplicación. Por estas características, es un patrón muy utilizado en el desarrollo de aplicaciones web.

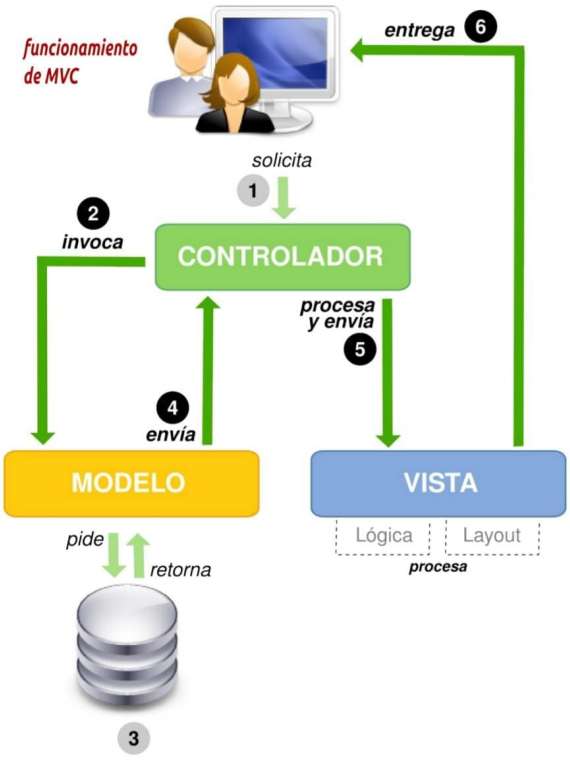


Figura 2.10: Patrón de diseño Modelo-Vista-Controlador (MVC)

2.6.3. Frontend

El *frontend* opera como la Vista del patrón MVC. Presenta la interfaz gráfica, captura las acciones del usuario y emite las peticiones.

Al solicitar la aplicación, el navegador descarga un conjunto de archivos que procesa localmente para generar la interfaz. Esta tarea se apoya en tres tecnologías complementarias (Fig, 2.11):

- HTML (*HyperText Markup Language*, lenguaje de marcado de hipertexto): define la estructura y el contenido del documento. Mediante etiquetas, especifica qué función cumple cada elemento en la interfaz (títulos, bloques de texto, botones o casillas para ingresar datos) y los organiza en un orden lógico que el navegador interpreta para armar la pantalla.

- CSS (*Cascading Style Sheets*, hojas de estilo en cascada): controla el aspecto visual y la ubicación de los elementos definidos por HTML. Aplica reglas para establecer colores, tipografías, tamaños y espacios. También organiza cómo se distribuyen los bloques en la pantalla. De este modo, la interfaz se adapta al tamaño del dispositivo.
- JavaScript: es el lenguaje de programación que se ejecuta directamente en el navegador. Mientras que el HTML define qué elementos existen y el CSS define cómo se ven, JavaScript hace que la página responda a las acciones del usuario. Detecta eventos como clics o el envío de un formulario y altera el contenido en pantalla en tiempo real según esa interacción. Además, se comunica con el servidor para intercambiar información mientras la aplicación está en uso.

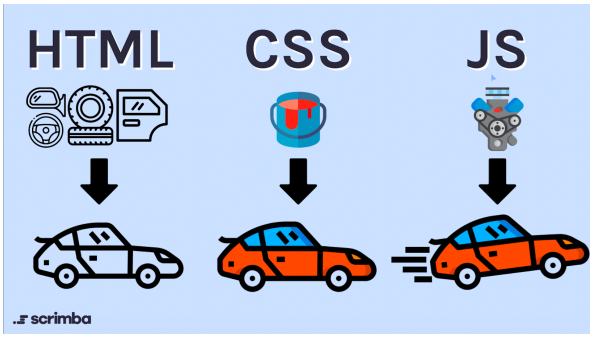


Figura 2.11: Arquitectura del frontend: HTML (su estructura), CSS (cómo se ve) y JavaScript (cómo funciona)

A medida que una aplicación suma pantallas y funciones, mantener el código fuente en archivos globales vuelve el sistema denso y difícil de ordenar. Para resolver esto, el desarrollo adopta una arquitectura basada en componentes (CBA, *Component-Based Architecture*). En este enfoque, la interfaz se estructura a partir de componentes independientes, donde cada elemento funciona como una entidad autónoma que reúne su estructura, presentación y lógica asociada (sus propios códigos HTML, CSS y JavaScript independientes del resto).

Bajo este modelo, el código fuente se divide en múltiples archivos modulares que se importan y articulan en un archivo principal. La única relación de dependencia entre componentes se da cuando uno contiene a otro. En esa situación, únicamente el componente padre puede alterar el estado del hijo. Esta estructura ofrece una fácil reusabilidad de elementos. Además, garantiza la modularidad y el aislamiento de fallos. Es decir, si un componente específico presenta un error, no compromete el funcionamiento del programa en su totalidad.

Existen diversas herramientas disponibles para implementar este esquema, entre las cuales destaca *React*, una biblioteca de JavaScript enfocada en la construcción de interfaces de usuario mediante componentes reutilizables. Por otra parte, *Preact* se presenta como una alternativa más ligera y eficiente para entornos con recursos limitados. Esta herramienta ofrece las mismas capacidades de desarrollo que *React*, pero omite características accesorias, lo que optimiza drásticamente el peso del empaquetado final (*bundle*). En consecuencia, se reduce en gran medida el tamaño de los archivos estáticos que el microcontrolador debe alojar en su memoria y transferir posteriormente al navegador al momento de utilizar la interfaz.

2.6.4. Backend

El *backend* reúne las funciones del Controlador y del Modelo del patrón MVC. Atiende las solicitudes entrantes, procesa la lógica de la aplicación y administra la persistencia de los datos.

En este desarrollo, el backend adopta una arquitectura en capas basada en el patrón Controlador-Servicio-Repositorio, que deriva directamente de MVC. La Vista se resuelve de forma independiente en el cliente web. En consecuencia, los Controladores conservan su rol original y el Modelo tradicional se divide en dos capas separadas. Esta división diferencia la lógica de la persistencia de datos.

- Controladores (*controllers*): reciben las peticiones que llegan desde el cliente web, extraen los datos de la solicitud y llaman al Servicio correspondiente para devolver luego la respuesta. No aplican reglas de funcionamiento ni interactúan con los datos guardados.
- Servicios (*services*): representan la parte del Modelo encargada de la lógica del sistema. Reciben los datos procesados por el Controlador, aplican las reglas operativas, realizan validaciones y coordinan cuándo consultar o modificar información mediante los Repositorios.
- Repositorios (*repositories*): representan la parte del Modelo dedicada exclusivamente a los datos y su almacenamiento. Son los únicos que leen y escriben en la base de datos, aislando los detalles de la persistencia del resto del sistema.

Imagen de Controller/Service/Repository (Natty)

Las peticiones que atienden los Controladores llegan a través de una interfaz de programación de aplicaciones (API, *Application Programming Interface*). Esta interfaz define un conjunto de rutas, cada una asociada a un tipo de información, como por ejemplo los usuarios de una aplicación. Sobre cada ruta, el método HTTP de la petición indica la operación solicitada. Los más utilizados son:

- GET: consulta información sin modificarla.
- POST: crea un registro nuevo.
- PUT: modifica un registro completo.
- PATCH: modifica solo una parte de un registro.
- DELETE: elimina un registro.

Por su parte, cada respuesta incluye un código de estado numérico que informa el resultado. Los códigos de la serie 200 indican que la operación se completó con éxito. Los de la serie 400 señalan un error en la petición, como datos inválidos, falta de permisos o un recurso inexistente. Por último, los de la serie 500 indican una falla del servidor.

El protocolo HTTP no conserva información entre una petición y la siguiente, ya que trata a cada una de forma independiente. Por este motivo, el backend necesita un mecanismo para reconocer a un usuario que ya inició sesión. Una solución habitual consiste en entregarle al cliente, una vez verificados su usuario y contraseña, una cadena única denominada *token*. A partir de ese momento, el cliente la adjunta en cada petición, y el servidor la utiliza para identificar al usuario y comprobar sus permisos. Cuando la sesión finaliza o expira, esa cadena deja de ser válida.

2.6.5. Base de datos

Una base de datos es una colección organizada e integrada de datos almacenados de forma persistente para su posterior consulta y manipulación. En el patrón MVC, el Modelo es el único componente que accede a ella.

Dentro de los distintos paradigmas existentes, el modelo relacional organiza la información en estructuras bidimensionales denominadas tablas (o relaciones), compuestas por filas (registros o tuplas) y columnas (campos o atributos).

Los fundamentos de este modelo se basan en los siguientes conceptos:

- Estructura y esquemas: cada tabla posee un esquema definido que determina el tipo de dato que puede almacenar cada columna (numérico, texto, fecha, etc.), lo que garantiza la consistencia del formato.
- Claves e integridad relacional: para identificar de forma unívoca cada registro se utiliza una clave primaria (*Primary Key*). La vinculación entre distintas tablas se establece mediante claves foráneas (*Foreign Keys*), lo que permite relacionar entidades sin duplicar información (integridad referencial).
- Lenguaje de consulta estructurado (SQL): es el lenguaje formal estándar (*Structured Query Language*) empleado para definir la estructura de los datos, realizar búsquedas mediante filtros, e insertar, modificar o eliminar información de manera declarativa.
- Transacciones y propiedades ACID: una transacción es un conjunto de operaciones que se ejecutan como una única unidad lógica. Para garantizar la confiabilidad, los sistemas relacionales implementan las propiedades ACID. Estas son:
  - Atomicidad: la operación se completa en su totalidad o no se aplica.
  - Consistencia: se respetan todas las reglas e integridad del esquema.
  - Aislamiento: las operaciones simultáneas no interfieren entre sí.
  - Durabilidad: los cambios confirmados persisten ante fallos o cortes de energía.

Agregar imagen para base de datos (Natty)

En el desarrollo de software existen dos formas principales de gestionar una base de datos: mediante sistemas basados en un servidor independiente (que atienden peticiones a través de una red) o mediante motores embebidos.

Un motor de base de datos embebido es una biblioteca de software que se integra directamente dentro del mismo código de la aplicación. En este esquema, el motor no se ejecuta como un proceso separado, sino que forma parte del propio ejecutable del sistema.

SQLite es un motor de base de datos relacional embebido de código abierto ampliamente utilizado que implementa el estándar SQL. Sus características teóricas esenciales son:

- Arquitectura sin servidor (*Serverless*): los procesos leen y escriben directamente en el soporte de almacenamiento, lo que elimina la necesidad de configurar, administrar o mantener un servicio en segundo plano.
- Almacenamiento en archivo único: la base de datos completa (esquema, tablas, índices y datos) se guarda dentro de un único archivo en el sistema de archivos del dispositivo, lo que simplifica su copia, transporte y respaldo.
- Autonomía y portabilidad (*Self-contained*): no depende de bibliotecas externas complejas ni de configuraciones previas en el sistema operativo, lo que permite su funcionamiento en entornos con recursos de memoria y procesamiento acotados.
- Cumplimiento transaccional: a pesar de su tamaño compacto, implementa transacciones atómicas que aseguran que los datos no se corrompan ante reinicios o pérdidas de alimentación eléctrica imprevistas.

Entre la información que guarda una base de datos suelen encontrarse las contraseñas de los usuarios, que por seguridad no se almacenan en texto plano. En su lugar, se guarda el resultado de aplicarles una función de *hash*. Esta función convierte un dato de cualquier longitud en una cadena de tamaño fijo. El proceso es irreversible, por lo que a partir del resultado no es posible conocer el dato original. Además, una misma entrada produce siempre el mismo resultado, y un cambio mínimo genera uno completamente distinto. Uno de los algoritmos más utilizados es SHA-256 (*Secure Hash Algorithm*), que produce resultados de 256 bits.

Capítulo 3

Diseño e implementación

En este capítulo se presenta el diseño y la implementación de la solución propuesta. Se definen los requisitos del sistema y se expone la evaluación de las técnicas de seguimiento ocular hasta la selección de la adoptada, proceso que determinó la arquitectura global. A partir de esta estructura, se detallan los módulos de hardware utilizado, la infraestructura de red y los protocolos de comunicación Asimismo, se describe la implementación del software, que comprende la lógica del servidor, la aplicación web y la interfaz táctil. Luego se detallan el sistema de alimentación, el diseño físico y, para finalizar, los costos totales del desarrollo.

3.1. Requisitos y especificaciones del sistema

En esta sección se detallan los requisitos planteados al inicio del proyecto. Derivan de los objetivos y del alcance presentados en el capítulo 1. Se clasifican en requisitos funcionales, que describen qué debe hacer el sistema; requisitos técnicos, que establecen las condiciones que debe cumplir el hardware y la comunicación; y restricciones de diseño, que delimitan cómo debe construirse.

3.1.1. Requisitos funcionales

Los usuarios del sistema (alumno, profesor, tutor y compañero) se definieron en el capítulo 1. A continuación se listan los requisitos que debe cumplir el dispositivo para atender a cada uno de ellos, agrupados según su función.

Interacción mediante la mirada:

- Acciones oculares: se detectarán seis acciones. Cuatro correspondientes a la mirada hacia cada una de las esquinas de la pantalla (arriba a la izquierda, arriba a la derecha, abajo a la izquierda y abajo a la derecha). Las otras dos serán la mirada hacia el centro y el parpadeo.
- Selección de respuestas: cada pregunta tendrá entre dos y cuatro opciones, asociadas a las esquinas de la pantalla. El parpadeo confirmará la opción elegida.
- Posición neutra: la mirada hacia el centro no ejecutará ninguna acción. Así, el alumno podrá corroborar su respuesta o descansar la vista.

Evaluaciones:

- Gestión de cuestionarios: creación, edición y administración de cuestionarios de opción múltiple personalizados.
- Resolución: el alumno debe ser capaz de resolver los cuestionarios únicamente con la mirada. El sistema registrará sus respuestas y los resultados obtenidos.

- Seguimiento académico: consulta de todos los cuestionarios, junto con las respuestas y los resultados del alumno, para realizar un seguimiento integral.

Comunicación con el alumno:

- Preguntas informales: envío de preguntas esporádicas de opción múltiple por parte de personas de su entorno, sin necesidad de registrarse.
- Sin persistencia: estas preguntas no se almacenarán en el sistema.

Usuarios y datos:

- Autenticación: registro e inicio de sesión de profesores y tutores.
- Permisos por rol: las funciones disponibles dependerán del rol de cada usuario.
- Base de datos local: almacenamiento de usuarios, cuestionarios y respuestas.

Acceso a la interfaz:

- Facilidad de uso: interfaz intuitiva y sencilla.
- Acceso universal: se debe poder utilizar desde cualquier dispositivo con navegador web, sin instalar software adicional.
- Comunicación local: los distintos bloques del sistema se comunicarán de forma inalámbrica, sin acceso a internet.

3.1.2. Requisitos técnicos

Los requisitos técnicos traducen las funciones anteriores en condiciones concretas para el hardware y la comunicación del sistema.

- Tiempo real: la detección debe responder con la rapidez suficiente para una interacción fluida. Por este motivo, la cámara tiene que ofrecer una tasa de fotogramas por segundo (FPS) adecuada.
- Robustez ante la iluminación: la detección no debe depender de las condiciones de luz del ambiente. Esta variable es incontrolable y cambia de manera constante.
- Captura de imagen: no se exige una cámara de alta resolución, ya que un exceso de datos puede saturar la memoria del microcontrolador. El campo de visión tampoco debe ser muy amplio, para evitar que estímulos ajenos a la pupila entorpezcan el análisis. Mediante el estudio empírico de imágenes con distintos ángulos, se determinó que la tarea requiere un ángulo de visión de entre 60 y 120 grados.
- Recursos limitados: el sistema debe funcionar con microcontroladores de memoria y capacidad de procesamiento acotadas. Esta condición es coherente con la restricción de costo.
- Conectividad inalámbrica: el microcontrolador que aloja la aplicación debe contar con conexión Wi-Fi. De este modo, se comunica con los dispositivos de los usuarios sin necesidad de acceso a internet.
- Independencia del sistema operativo: La interfaz debe ser una página web y no una aplicación instalable. Este tipo de programa depende del sistema operativo del dispositivo en el que se ejecuta. Una página web, en cambio, funciona en cualquier equipo con navegador, sea una laptop, un celular, una tablet o una computadora de escritorio.

3.1.3. Restricciones de diseño

Las restricciones de diseño limitan la forma en que se construye el dispositivo, más allá de las funciones que debe cumplir.

- Costo: el dispositivo debe ser económico. Por este motivo, se emplean microcontroladores de bajo precio y fáciles de adquirir.
- Portabilidad: el sistema debe poder trasladarse sin depender de una instalación fija.
- Ergonomía: el módulo de captura debe montarse en la visera de una gorra común. Esta ubicación ofrece a la cámara un buen ángulo del ojo y un uso cómodo y seguro. Además, mantiene el sensor en una posición fija respecto al ojo.
- Autonomía energética: el dispositivo debe ser inalámbrico y autosuficiente al momento de utilizarlo. Para ello, cada módulo cuenta con su propia alimentación, independiente de cualquier fuente externa durante su funcionamiento.

3.2. Detector ocular

El detector ocular es el componente central del sistema. Su función es determinar la dirección de la mirada del alumno a partir de la imagen de su ojo. Los métodos evaluados deben cumplir con dos condiciones fundamentales, que son la rapidez de respuesta y la precisión de la detección.

En esta sección se describe la evolución del detector. Primero se presentan los métodos evaluados de procesamiento local, que se ejecutan en el propio microcontrolador. Luego se exponen los de procesamiento remoto, que corren en el dispositivo cliente. Por último, se detalla la solución adoptada.

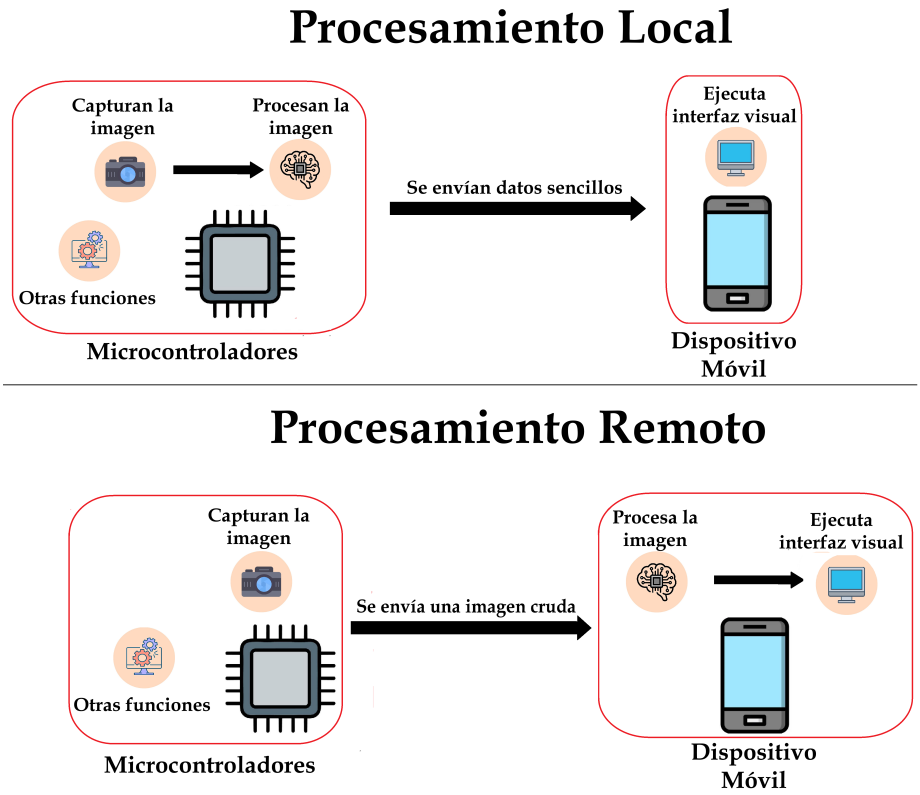


Figura 3.1: Diferencias entre procesamiento local y remoto

3.2.1. Procesamiento local como planteamiento inicial

En un principio se estableció que el procesamiento de las imágenes se realizaría en un microcontrolador, mediante una red neuronal artificial de tipo convolucional. Cuando la placa gestiona todas las tareas de detección, se habla de procesamiento local.

Antes de implementar una red neuronal se optó por intentar un método más sencillo. Se consideró que este tipo de modelo podía ser una complicación innecesaria para este problema. Este primer método se desarrolló en el *ESP32-CAM*, una variante de las placas de desarrollo *ESP32* que incluye un módulo para cámaras. En principio, este microcontrolador capturaría y procesaría las imágenes. Sin embargo, al no alcanzar la precisión requerida, se recurrió a las redes neuronales. Para poder ejecutarlas, se lo sustituyó por el *ESP32-S3*.

Esta última placa se eligió porque su procesador incorpora instrucciones vectoriales diseñadas para acelerar el cómputo de redes neuronales. Dichas instrucciones permiten procesar varios datos en una sola operación. *Espressif* las pone a disposición mediante la biblioteca *ESP-NN*, cuyas funciones optimizadas están orientadas a modelos cuantizados a números enteros de 8 bits (*int8*). La cuantización reduce el tamaño del modelo y el costo de cada operación. Por estos motivos, el *ESP32-S3* resulta una opción adecuada para ejecutar redes neuronales con recursos limitados.

Aun con esta aceleración, lograr que un microcontrolador ejecute una red neuronal de este calibre no es sencillo. La mayoría de estos modelos exige una capacidad de memoria y una velocidad que suelen exceder lo que ofrece el *ESP32-S3*. Por este motivo, se exploraron formas de diseñar y entrenar redes lo más livianas y sencillas posibles. Este paradigma busca implementar inteligencia artificial con recursos de software y hardware limitados. Se lo conoce como *Tiny Machine Learning* (o *TinyML*) y es la filosofía que se aplicó en el desarrollo de este dispositivo.

A continuación se desarrollan los tres métodos evaluados.

Umbral de oscuridad y brillantez

Este método de visión computacional clásica consiste en capturar una imagen en escala de grises de forma continua para analizar sus patrones de oscuridad (intensidad lumínica) en tiempo real.

Cuando se toma una imagen del ojo, el punto más oscuro suele ser la pupila (Figura 3.2). El resto del cuadro presenta porciones irregulares con tonos grises más claros. El algoritmo aprovecha este contraste extremo para localizar las coordenadas pupilares y, a partir de ellas, inferir la dirección de la mirada.

Para entender este método, se debe aclarar antes cómo el microcontrolador procesa una imagen. Al capturar un fotograma, el *ESP32* lo almacena en un *framebuffer*, es decir, un bloque de memoria reservado y estructurado como una matriz bidimensional. En formato de escala de grises, cada elemento de esta matriz contiene un número entero entre 0 y 255. Este valor representa la intensidad lumínica del píxel en esa posición espacial (un 0 equivale a oscuridad absoluta y 255 a saturación blanca).

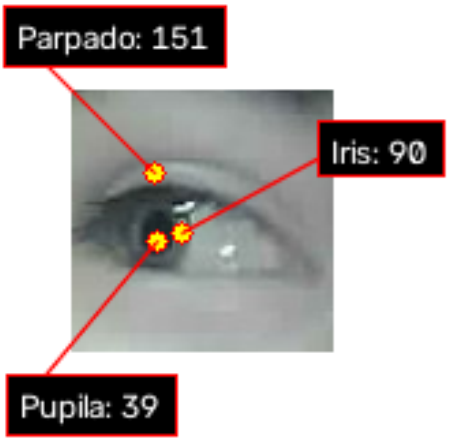


Figura 3.2: Distintas partes del ojo y su nivel de oscuridad

El algoritmo realiza un barrido inicial, en el que revisa cada píxel para identificar el valor de intensidad mínimo (el punto más oscuro del fotograma). Como depender de un único píxel comprometería la estabilidad del sistema, se establece un umbral de oscuridad dinámico. De esta manera, el programa no aísla un solo punto, sino toda una región cuyos valores caen por debajo de dicho límite.

Una vez delimitada la región de interés (*ROI*) que en teoría abarca la pupila, el programa ejecuta un segundo barrido. Éste calcula las coordenadas centrales de la región mediante un centroide ponderado. En un promedio simple, se sumarían todas las coordenadas para luego dividirlas por la cantidad de píxeles detectados. En cambio, el centroide ponderado asigna a cada coordenada un peso en función de su nivel de oscuridad. Mientras más oscuro es un píxel, mayor es su peso en la ecuación final. Esta ponderación es fundamental para la estabilidad del sistema. Otorga una influencia dominante al núcleo de la pupila (altamente oscuro) y reduce las desviaciones causadas por sombras periféricas o pestañas que ingresaron dentro del umbral de detección (grises más claros).

Este algoritmo es liviano y se ejecuta con rapidez en el *ESP32*. Sin embargo, al evaluar este método se descubrió que la precisión era insuficiente. Las sombras proyectadas sobre el rostro por la iluminación ambiental, sumadas a la presencia de cejas, pestañas o armazones de anteojos, confundían fácilmente al detector. Si bien la pupila seguía siendo el punto de menor intensidad lumínica, la gran similitud de tonos con estos elementos externos volvía inestable al seguimiento.

Ante este obstáculo, se decidió cambiar de enfoque. En lugar de buscar objetos por su nivel de oscuridad, se buscó aprovechar un fenómeno físico conocido como *efecto de brilo corneal*. Al apuntar una fuente de luz hacia el globo ocular, la superficie húmeda y curva de la córnea actúa como un espejo esférico. Esto hace que la luz rebote y forme un punto blanco concentrado e intenso en la imagen, denominado *glint* (Figura 3.3).

Como la ubicación de este destello suele coincidir con la de la pupila, parecía una alternativa viable para reemplazar el algoritmo anterior por un detector de brillantez. Sin embargo, iluminar de forma constante el ojo del usuario con luz visible resultaría molesto y perjudicial para su visión. Para solucionarlo, se planteó utilizar luz infrarroja (IR).

Este tipo de iluminación tiene una longitud mayor a la del espectro visible. Por lo tanto, el ojo no las percibe, pero los sensores fotográficos sí logran captarlas. Dado que la cámara del

ESP32-CAM venía de fábrica con un filtro bloqueador de luz infrarroja, fue necesario removerlo de forma manual. Luego se colocó un diodo LED IR junto a la cámara.



Figura 3.3: El *glint* en el centro de la pupila

Esta solución tampoco resultó viable. Durante las pruebas de seguimiento, se comprobó que el glint no se desplazaba de la misma forma que la pupila. Su posición relativa dependía estrictamente de la geometría del ojo frente a la fuente de luz (Fig. 3.4).



Figura 3.4: El *glint* no sigue a la pupila

Tras estos resultados, se descartó la visión computacional clásica y se dio paso a las redes neuronales.

Edge Impulse

Un primer acercamiento al *TinyML* es la plataforma *Edge Impulse*. Se trata de un entorno de desarrollo basado en la nube, orientada específicamente al aprendizaje automático con bajos recursos computacionales (Figura 3.5). Fue fundada en 2019 con el objetivo de facilitar esta tarea. Su función principal es trasladar el procesamiento desde servidores en la nube directamente al hardware local. Además, es una herramienta versátil ya que ofrece compatibilidad con el framework de Arduino.

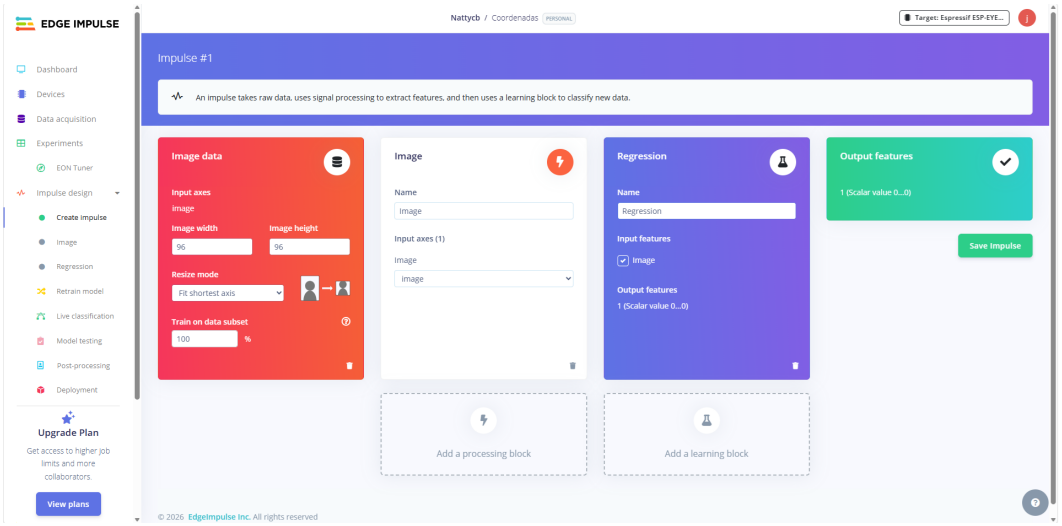


Figura 3.5: Pantalla de creación de modelos de *Edge Impulse*

Su funcionamiento se basa en una estructura de trabajo lineal (conocida como *pipeline* o *Impulse*). Este flujo unifica todas las etapas del desarrollo de un modelo predictivo. Comienza con la adquisición y el etiquetado de datos estructurados. Continúa con bloques de procesamiento digital de señales, que extraen las características más relevantes de las imágenes. Sigue con el entrenamiento de la red neuronal propiamente dicha y finaliza con el empaquetado del modelo. Todo esto se realiza mediante una interfaz de usuario intuitiva.

Gracias a sus técnicas de cuantización y a sus métodos de aceleración de hardware, resulta una opción prometedora que optimiza todo el proceso de aprendizaje. Funciona muy bien en tareas sencillas, como el reconocimiento de objetos o rostros (Figura 3.6).

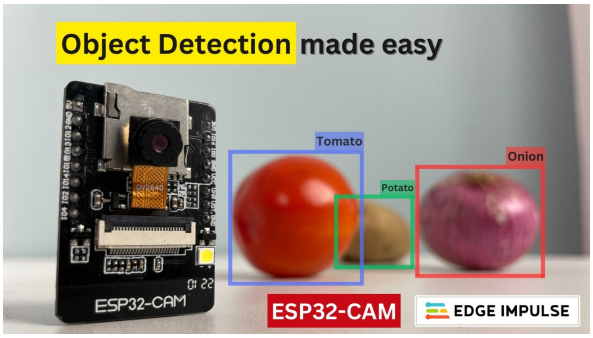


Figura 3.6: Reconocimiento de vegetales con *EdgeImpulse*

La plataforma permite elegir entre distintas arquitecturas de red preentrenadas para la detección de objetos. En este proyecto se seleccionó *FOMO* (*Faster Objects, More Objects*), un algoritmo diseñado específicamente para dispositivos con recursos extremadamente limitados.

A diferencia de las redes tradicionales que calculan coordenadas geométricas complejas, esta arquitectura utiliza *MobileNetV2* como motor de extracción de características y luego aplica un enfoque más simple y eficiente: divide la imagen en una cuadrícula y evalúa cada celda de forma individual. En lugar de predecir el ancho y alto del objeto, el sistema actúa como un clasificador espacial que determina estadísticamente si cada celda corresponde al fondo de la imagen o si contiene el centro (centroide) de un objeto de interés. Esta simplificación matemática es lo que

le permite ocupar menos de 100 KB de memoria y admitir entradas en escala de grises o RGB a cualquier resolución.

Bajo este esquema, se eligieron las variantes *MobileNetV2 0.1* y *MobileNetV2 0.35* como redes base para comparar su rendimiento. Si bien la confección del conjunto de datos y el etiquetado inicial se trabajaron mediante cuadros delimitadores (*bounding boxes*), la red extrae de ellos la posición central para su entrenamiento. Finalmente, los modelos resultantes fueron cuantizados al formato *int8*. Esta conversión fue fundamental para reducir el consumo de memoria y aprovechar al máximo la aceleración de hardware del *ESP32-S3*.

Antes de cargar cada modelo en la placa, *Edge Impulse* estima su tiempo de inferencia. Para un *ESP32* común, la estimación era de aproximadamente 1 segundo por inferencia. Al cargar el código en el *ESP32-S3*, el monitor serial indicó alrededor de 200 milisegundos. Esta diferencia es coherente con la aceleración por hardware del *ESP32-S3*.

Aun con esta mejora, una latencia de 200 milisegundos es demasiado alta para un seguimiento en tiempo real. Esta lentitud fue la razón principal por la que se descartó esta solución.

Imágenes del monitor serial con los tiempos de inferencia en cada placa? (Natty)

TensorFlow Lite

Una herramienta muy utilizada en *Machine Learning* es el framework de código abierto *TensorFlow* (*TF*). Está diseñado para entrenar y ejecutar modelos de redes neuronales propios o ya existentes. Sin embargo, no está optimizado para correr redes en sistemas con hardware limitado como los *ESP32*. Por este motivo, *Google* desarrolló una versión más liviana, llamada *TensorFlow Lite* (*TFLite*). Su variante específica para arquitecturas sin sistema operativo es *TensorFlow Lite for Microcontrollers* (*TFLM*).

Como siguiente paso, se optó por entrenar un modelo propio utilizando imágenes capturadas directamente por el *ESP32-CAM*. A diferencia de *Edge Impulse*, que automatiza el proceso pero restringe las decisiones de diseño, este nuevo enfoque otorga una libertad de personalización absoluta. Permite tener control total sobre cada etapa del flujo de trabajo, como elegir individualmente las capas matemáticas, ajustar los parámetros de entrenamiento o definir la estrategia exacta de compresión y exportación. Esta independencia resulta vital para adaptar el modelo a las limitaciones del microcontrolador y además se elimina la sobrecarga de código genérico que suele imponer una plataforma cerrada.

Para facilitar la creación del modelo, *Google* ofrece también *Keras*, una API de alto nivel que agiliza y simplifica enormemente la programación de la arquitectura neuronal. El entorno de desarrollo seleccionado fue *Google Colab* (Figura 3.7). Se trata de una plataforma basada en la nube que permite ejecutar código Python en *notebooks* interactivos.

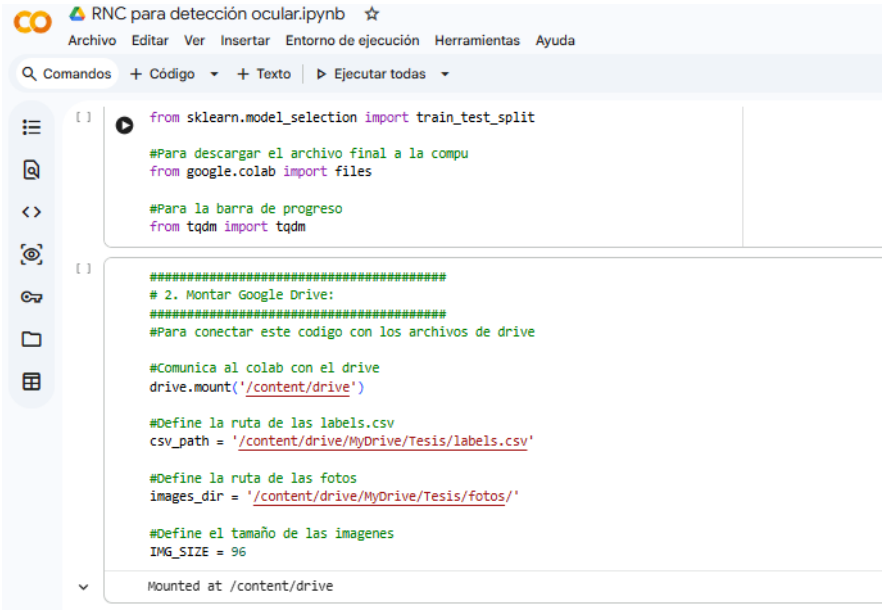


Figura 3.7: Entorno de *Google Colab*

Antes de empezar a codificar es necesario un *dataset* (conjunto de datos de entrada). Para una red de este tipo, normalmente se utilizan cientos de imágenes asociadas a lo que se denomina *etiquetas*. Una etiqueta representa el valor objetivo que el modelo debe predecir y aprender. En este proyecto, las etiquetas son números enteros fijos que representan las coordenadas espaciales (X e Y) del centro de la pupila.

Las imágenes de entrenamiento se capturaron con el módulo *ESP32-CAM*, en escala de grises (*Grayscale*) y con una resolución de 96x96 píxeles. Esta configuración se seleccionó para minimizar el consumo de recursos computacionales sin comprometer los detalles necesarios de la imagen. Para garantizar cantidad y variabilidad, se tomó un conjunto total de 1315 capturas de cinco personas distintas (Figura 3.8). Se contemplaron distintas condiciones de iluminación y variaciones físicas, como la presencia y la ausencia de anteojos.

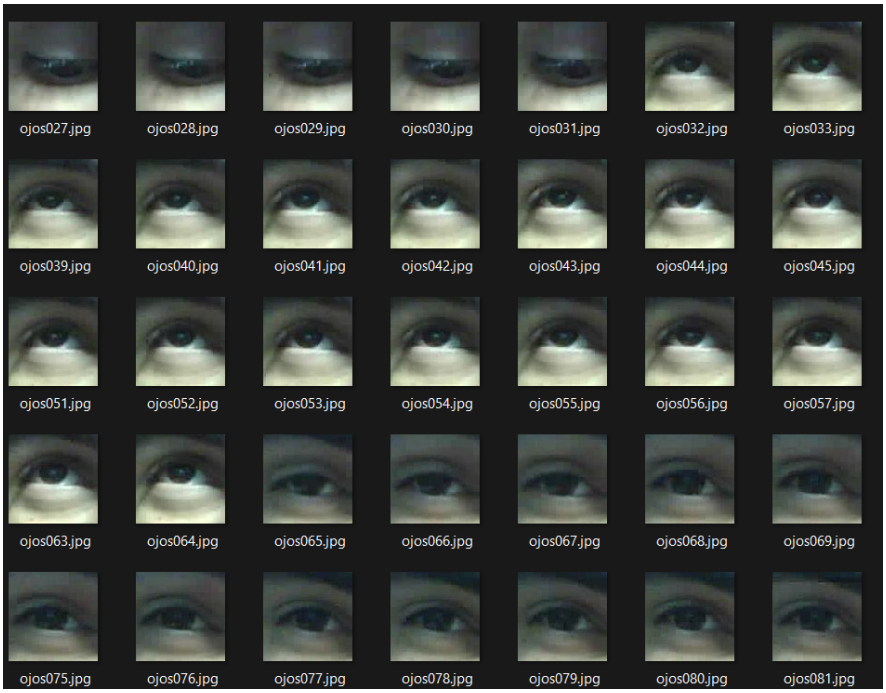


Figura 3.8: Una muestra de las imágenes utilizadas para el dataset

En paralelo, las coordenadas de cada imagen (expresadas en píxeles dentro de una imagen de 96x96) se anotaron en una hoja de cálculo. De este modo, los datos quedaron estructurados para su posterior utilización en el entrenamiento (Figura 3.9).

labels

Archivo Editar Ver Insertar Formato Datos Herramientas G

Menús 100% € % .0 .00 123

K7

	A	B	C	D	E
1	filename	x	y		
2	ojos0001	41	47		
3	ojos0002	41	47		
4	ojos0003	41	47		
5	ojos0004	41	47		
6	ojos0005	41	47		
7	ojos0006	41	47		
8	ojos0007	41	47		
9	ojos0008	43	46		
10	ojos0009	24	48		
11	ojos0010	18	48		
12	ojos0011	18	48		
13	ojos0012	18	48		
14	ojos0013	18	48		
15	ojos0014	18	48		
16	ojos0015	50	28		
17	ojos0016	50	28		
18	ojos0017	50	28		
19	ojos0018	50	28		
20	ojos0019	50	28		
21	ojos0020	50	28		

Figura 3.9: Planilla con etiquetado de imágenes

A continuación se listan y explican brevemente las etapas de la creación de un modelo de Red Neuronal Convolucional (CNN) en *Google Colab* con *Python/Keras*:

1. Preparación: Se importan en el código las herramientas necesarias para construir la CNN, entre ellas *TensorFlow*, *Pandas*, *NumPy* y *OpenCV*. Se monta *Google Drive* para que el entorno acceda a las imágenes y etiquetas almacenadas allí.
2. Formateo del dataset: Se extraen de *Google Drive* las 1315 imágenes, se recortan a 96x96 de ser necesario y se leen y almacenan las etiquetas en arreglos.
3. Creación de la red: Se define el formato de las entradas, las capas internas y la función de activación. Al final se compila el modelo y se establece cómo trabajará con los errores.
4. Separación de datos: Aquí se aplica una técnica para entrenar y evaluar la red al mismo tiempo. Se dividen las imágenes del dataset en dos grupos: el de entrenamiento (80 % de las imágenes) y el de evaluación (20 % restante). Esta separación es aleatoria y sirve para comprobar el resultado de la construcción del modelo.
5. Entrenamiento: Se ejecuta el aprendizaje de la red. Se elige el número de *épocas* (veces que el modelo analiza el grupo de imágenes) y se aplican *callbacks* (funciones que detienen el entrenamiento cuando se alcanza la precisión deseada). Con esto último se evita que la red memorice fotos en lugar de aprender de ellas.
6. Evaluación: Se pone a prueba la red con la porción de datos que nunca vio durante el entrenamiento. Se muestran los resultados y el análisis de distintos parámetros estadísticos, como el porcentaje de aciertos, los errores y la muestra con peor desviación.
7. Exportación: Se guarda el modelo en un archivo con formato *.tflite*. Este se utiliza luego en *Visual Studio Code* (con la extensión *PlatformIO*), el entorno en el que se programa el microcontrolador.

Se iteró sobre la arquitectura de la red y se ajustaron sus hiperparámetros (la cantidad de capas y los filtros de procesamiento, entre otros) para maximizar la precisión (Figura 3.10). Luego, el modelo se exportó y se compiló en el entorno de C++ para evaluar su rendimiento en vivo.

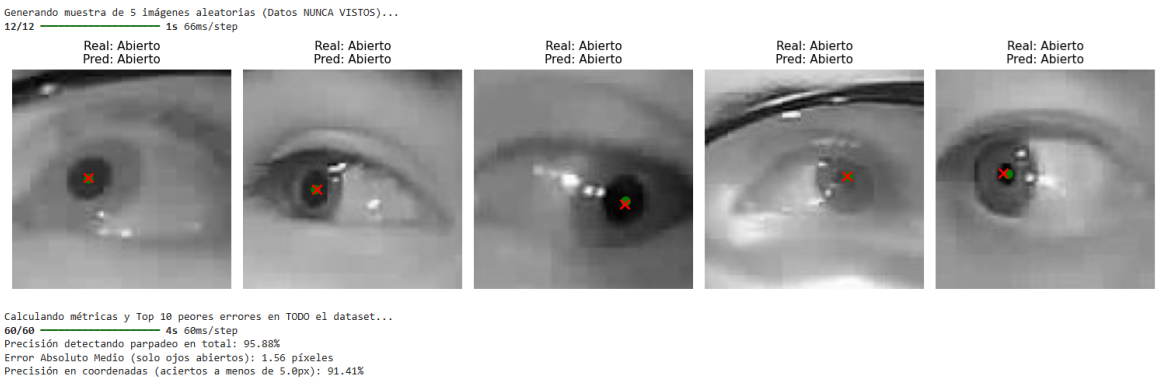


Figura 3.10: Resultados del modelo desarrollado en *Google Colab*. Con un punto verde se indica el valor etiquetado, mientras que la cruz roja muestra la posición que la red predice

Este modelo resultó significativamente más rápido, eficiente y controlable que el generado con *Edge Impulse*. Sin embargo, las pruebas empíricas en tiempo real mostraron que no cumplía los requisitos de precisión para el dispositivo final. Durante la operación, las coordenadas predichas eran muy inestables. Oscilaban de forma pequeña y constante (*jittering*) y no lograban mantenerse en un valor firme.

Imagen/es de TFLite fallando? (Top 10 errores)

Para reducir esa inestabilidad, se intentó aumentar la complejidad y la profundidad de la red neuronal. Sin embargo, esto elevó la latencia de procesamiento a niveles inaceptables. Por el contrario, simplificar la arquitectura para recuperar velocidad degradaba la exactitud de las coordenadas. Se encontró así un límite de hardware insalvable.

Conclusión del procesamiento local

Ninguno de los tres métodos cumplió las dos condiciones establecidas inicialmente. El umbral de oscuridad y brillantez no alcanzó la precisión requerida. *Edge Impulse* resultó demasiado lento para el tiempo real. El modelo propio con *TensorFlow Lite* no logró estabilidad en sus predicciones y, al aumentar su complejidad, perdió velocidad. Por lo tanto, el procesamiento de imágenes de forma puramente local, dentro de las limitaciones de memoria del microcontrolador, no resulta viable para las exigencias de este proyecto. Esta restricción motivó el descarte de las redes neuronales en hardware de bajos recursos.

Ante esta situación, se decidió derivar la detección a un dispositivo externo. Este enfoque se describe en la siguiente subsección.

3.2.2. Procesamiento remoto por coordenadas

Tras descartar el procesamiento local, se decidió trasladar la detección a un dispositivo externo. Este equipo es el mismo desde el cual el usuario utiliza el sistema, como una PC, un celular o una *tablet*. Cuando la detección se ejecuta en este dispositivo cliente, y no en el microcontrolador, se habla de procesamiento remoto.

Este enfoque cambia la distribución de tareas. El microcontrolador ya no procesa las imágenes. Solo las captura y las transmite al dispositivo cliente por Wi-Fi. Este último recibe los fotogramas y ejecuta sobre ellos el método de detección. La transmisión de imágenes se detalla más adelante en este informe.

La principal ventaja de este esquema es la capacidad de procesamiento disponible. Un dispositivo cliente cuenta con mucha más memoria y velocidad que un microcontrolador. Por lo tanto, ya no es necesario reducir los métodos de detección a modelos livianos como los de *TinyML*. Esto permite evaluar herramientas más complejas, que resultaban inviables en el *ESP32-S3*. Como contrapartida, el sistema pasa a depender de la transmisión continua de imágenes entre ambos dispositivos.

Al mover la detección al dispositivo cliente, la aceleración de redes neuronales que ofrece el *ESP32-S3* dejó de ser necesaria. Por lo tanto, se volvió a utilizar el *ESP32-CAM* para la captura de imágenes.

A continuación se desarrollan los dos métodos evaluados, que son las redes neuronales preentrenadas *MediaPipe* y *WebGazer*, y *OpenCV* con la Transformada de Hough.

MediaPipe y WebGazer

La mayor capacidad de procesamiento del dispositivo cliente permite utilizar una red neuronal preentrenada para la detección de la mirada. Dos alternativas consideradas fueron *MediaPipe* (un conjunto de bibliotecas provisto por *Google*) y *WebGazer* (un modelo desarrollado en el ámbito académico). Ambas son herramientas basadas en tecnologías web que solucionarían todos los problemas descritos con anterioridad. Sin embargo, fueron descartadas debido a dos inconvenientes.

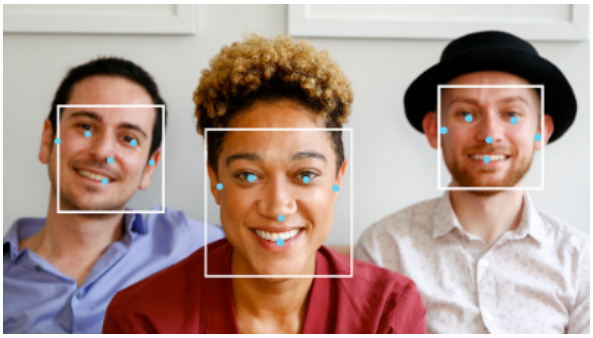


Figura 3.11: Detección de rostro con *MediaPipe*

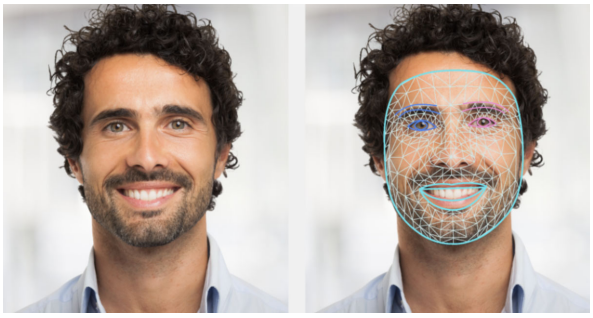


Figura 3.12: Facciones del rostro en *MediaPipe*

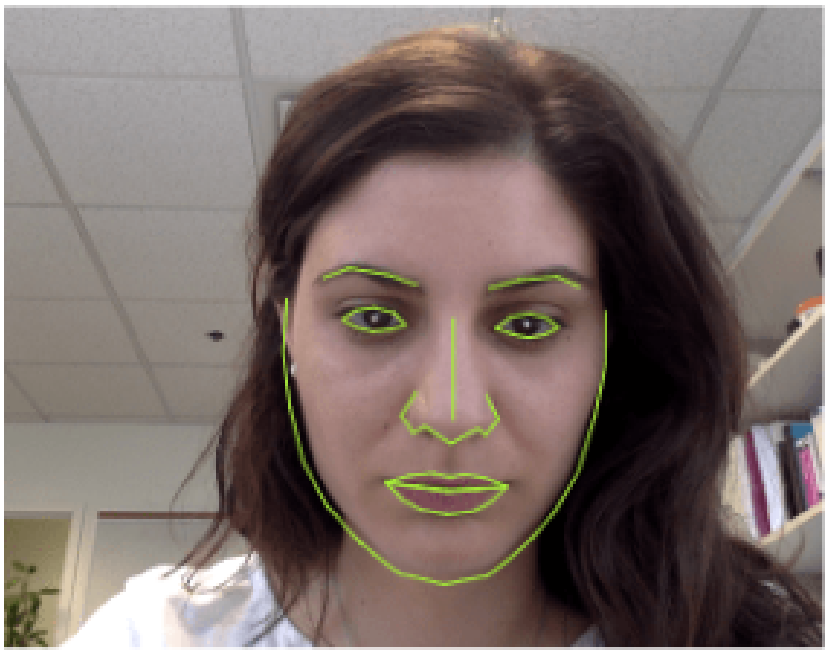


Figura 3.13: Facciones del rostro en *WebGazer*

Por un lado, las dos requieren que la imagen abarque el rostro completo. *MediaPipe* identifica puntos de referencia en toda la cara. *WebGazer*, si bien está diseñado específicamente para estimar la mirada, ubica los ojos mediante el detector facial de *MediaPipe*. En consecuencia, en ambos casos los ojos ocupan una cantidad reducida de píxeles. Para medir con precisión sus variaciones, se requiere una mayor resolución de imagen.

Esta mayor resolución puede obtenerse de dos formas. La primera es aumentar la resolución de

captura del *ESP32-CAM*, lo que implica un mayor tráfico de imágenes entre el microcontrolador y el dispositivo cliente. La segunda es derivar la captura a la cámara integrada del dispositivo cliente. Sin embargo, esto comprometería la versatilidad, un requisito vital del dispositivo final. Cada equipo cliente posee una cámara con propiedades distintas, cuya resolución, ubicación, distancia focal e inclinación varían drásticamente. Todas estas son variables incontrolables que afectarían la precisión del seguimiento espacial. Un dispositivo universal, que mantiene siempre su propia cámara en una posición fija respecto al ojo, evita por completo este problema.

Por otro lado, durante las pruebas, el cursor que indicaba en pantalla la posición estimada de la mirada vibraba de forma constante. Estas vibraciones se debían a los micromovimientos naturales del ojo. Como resultado, el seguimiento no resultaba estable.

Por estos motivos empíricos y de diseño, se decidió prescindir definitivamente de las redes neuronales como método de detección espacial.

OpenCV y Transformada de Hough

Tras descartar las redes neuronales, se evaluó *OpenCV*, la biblioteca de visión computacional descrita en el capítulo 2. A diferencia de *MediaPipe* y *WebGazer*, sus funciones pueden aplicarse directamente sobre una imagen del ojo sin necesidad de detectar el rostro completo. Por este motivo, se perfiló como una alternativa sumamente viable para este trabajo.

Las pruebas de este método se realizaron con un script de Python ejecutado en una computadora, que procesaba las imágenes capturadas por el *ESP32-CAM*. Este script funcionaba como prototipo. En caso de obtener buenos resultados, el detector se iba a trasladar a *OpenCV.js*, la versión de la biblioteca para navegadores web, que se ejecutaría en el dispositivo del usuario.

Para aprovechar al máximo las capacidades de *OpenCV*, se decidió complementarlo con la Transformada de Hough, un método clásico de extracción de características en el análisis de imágenes digitales. Su variante para círculos, conocida como *HoughCircles* en *OpenCV*, analiza los bordes detectados en un fotograma mediante un sistema matemático de votación. El algoritmo busca conjuntos de píxeles que compartan una curvatura y se encuentren a una distancia constante (el radio) de un posible punto central. Si suficientes píxeles votan por el mismo centro, el algoritmo confirma la existencia de un círculo en esa región. En este contexto, el círculo corresponde al iris del ojo, y las coordenadas de la pupila se obtienen al calcular la ubicación del centro de dicho círculo.

Una vez recibido el fotograma, *OpenCV* realiza un filtrado espacial (preprocesamiento) para limpiar la imagen de ruido digital y prepararla para el análisis. A continuación se aplica la función de círculos de Hough. Cuando se detecta el contorno del iris, los resultados se grafican sobre la imagen a modo de depuración visual. Un trazo verde delimita la circunferencia detectada y un punto rojo en su centro representa las coordenadas de la pupila.

Imagen de Hough funcionando (Natty)

Aquí surge un problema importante. La función de círculos de Hough está diseñada para detectar circunferencias perfectas, y el iris humano no siempre se presenta como tal frente a la cámara. Con frecuencia, los párpados ocluyen una parte superior o inferior y alteran su geometría circular. Esto implicaría que el usuario debería forzar permanentemente sus músculos oculares para mantener los ojos abiertos de forma antinatural y así garantizar el correcto funcionamiento del sistema. Esta situación resulta poco práctica y sumamente indeseable.

Imagen de Hough fallando por cerrar ojos (Natty)

Una posible solución consiste en disminuir la exigencia de *OpenCV* para que tolere ciertas deformaciones, como arcos incompletos o formas elípticas, y no detecte únicamente círculos perfectos. Sin embargo, esto genera un problema secundario grave. El algoritmo se vuelve demasiado permisivo y registra falsos positivos al detectar curvas en párpados, cejas, arrugas y otras facciones del rostro.

Imagen de falsos positivos (Natty)

Ambos problemas presentan un compromiso técnico recíproco, ya que si se intenta reducir uno, el otro empeora. Se realizaron sucesivas pruebas con herramientas complementarias, como umbrales de radio y detectores de brillantez y oscuridad apoyados por el LED infrarrojo. Aun así, la precisión alcanzada no resultó adecuada para un seguidor ocular en tiempo real, por lo que esta solución queda definitivamente descartada.

Conclusión de la detección por coordenadas

Ninguno de los dos métodos alcanzó un resultado satisfactorio. *MediaPipe* y *WebGazer* requerían una mayor resolución de imagen, que no podía obtenerse sin comprometer el diseño del dispositivo, y su estimación de la mirada resultaba inestable. Con *OpenCV* y la Transformada de Hough no se logró detectar el iris con precisión, ya que la oclusión de los párpados y los falsos positivos presentaban un compromiso que no pudo resolverse.

Las dos alternativas compartían un mismo enfoque, que consistía en detectar coordenadas espaciales específicas del ojo. Por este motivo, se tomó una decisión radical en el desarrollo de este trabajo. Se dejó atrás la detección de coordenadas y se pasó a identificar patrones de orientación de la mirada. Este cambio busca simplificar el detector y eliminar las causas de error encontradas en los métodos anteriores. El objetivo es lograr un algoritmo independiente de la luz ambiental, de la resolución de la cámara y de las variaciones en la forma del ojo. Esta estrategia se describe en la siguiente subsección.

3.2.3. Template Matching como solución adoptada

Para identificar la orientación de la mirada se utilizó *Template Matching*, técnica descrita en el capítulo 2. En este caso, las plantillas son imágenes del ojo del alumno, una por cada una de las seis acciones posibles. Como cada dispositivo está destinado a un único estudiante, las plantillas corresponden siempre a su propio ojo.

El algoritmo se ejecuta en el navegador del dispositivo cliente mediante *OpenCV.js*. De esta forma, el detector no requiere instalar ninguna aplicación adicional en el equipo del usuario.

A continuación se describen las tres etapas del método, que son la calibración, el preprocesamiento de fotogramas y la comparación de plantillas.

Calibración

El método requiere, como paso inicial, un proceso de calibración que proporcione las plantillas. Durante esta etapa se captura una imagen del ojo del alumno para cada una de las acciones posibles dentro de la interfaz. Para garantizar la estabilidad del algoritmo y reducir las confusiones entre plantillas, estas acciones se limitaron a seis. A continuación se detalla la plantilla que se obtiene para cada una.

- Esquina superior izquierda: imagen del ojo con la mirada hacia arriba a la izquierda, asociada a la opción 1.

- Esquina superior derecha: imagen del ojo con la mirada hacia arriba a la derecha, asociada a la opción 2.
- Esquina inferior izquierda: imagen del ojo con la mirada hacia abajo a la izquierda, asociada a la opción 3.
- Esquina inferior derecha: imagen del ojo con la mirada hacia abajo a la derecha, asociada a la opción 4.
- Centro: imagen del ojo con la mirada hacia el centro de la pantalla. Corresponde a la zona neutra, en la que el alumno puede descansar la vista sin realizar ninguna acción. Si antes resaltó una opción, esta se conserva hasta que la confirma o mira hacia otra.
- Parpadeo: imagen del ojo cerrado. Permite confirmar la opción elegida después de mirar hacia una de las esquinas.

Imagen de las seis plantillas (Natty)

Cada plantilla se guarda con una resolución de 96x96 píxeles en escala de grises. Luego se almacenan en el navegador, donde se conservan mientras no se cierre la pestaña de la aplicación. De esta forma, quedan disponibles como referencia para comparar cada fotograma durante la detección.

Preprocesamiento de fotogramas

Con el sistema en funcionamiento, el *ESP32-CAM* captura el ojo en tiempo real con una resolución de 320x240 píxeles, en color. Los fotogramas se envían al navegador del dispositivo cliente a través de la red local. Allí se realiza todo el procesamiento de las imágenes.

Imagen del stream? (Natty)

Antes de la comparación, cada fotograma se acondiciona. Primero se recorta a un tamaño de 128x128 píxeles (región de interés), y luego se cambia su formato a escala de grises. Ambos pasos reducen la carga del procesamiento posterior. El recorte es más grande que las plantillas por un motivo que se explica en la siguiente etapa. Con el mismo fin, la tasa de análisis se limita a 15 fotogramas por segundo (FPS). Experimentalmente, esta frecuencia resultó suficiente para lograr una interacción fluida en tiempo real sin sobrecargar el dispositivo cliente.

Durante las pruebas también se observó un problema relacionado con la iluminación. El método de comparación tolera los cambios uniformes de luz ambiental, como se detalla más adelante. Sin embargo, las sombras que proyecta una fuente de luz externa inclinada dificultan el procesamiento. Por este motivo, se volvió a incorporar el LED infrarrojo al diseño. Este LED proporciona una iluminación frontal uniforme sobre el ojo y evita la aparición de esas sombras.

Comparación de plantillas

Cada fotograma acondicionado se compara con las seis plantillas mediante el *coeficiente de correlación normalizado con media cero*, descrito en el capítulo 2.

Como se observa en el numerador de la fórmula, a cada píxel se le resta el valor promedio de su matriz. Por este motivo, el coeficiente recibe el nombre de media cero. Así, la fórmula no evalúa la intensidad lumínica absoluta de cada píxel, sino cuánto se diferencia del promedio de la imagen. Por ejemplo, si el rostro del alumno se ilumina de forma repentina por una fuente externa, todos los píxeles aumentan su valor, pero la diferencia relativa entre ellos se mantiene. De esta forma, el método resulta inmune a las variaciones uniformes de la iluminación ambiental.

La plantilla mide 96x96 píxeles y el fotograma 128x128, aunque en ambos casos el ojo tiene el mismo tamaño. Esta diferencia permite que la plantilla recorra el fotograma mediante el barrido explicado en el capítulo 2, en lugar de compararse en una única posición.

El barrido compensa los errores de posicionamiento entre el ojo y la cámara. Si la comparación se realizara en un solo punto, un desplazamiento mínimo del dispositivo, incluso de un milímetro, desalinearía las matrices y haría fallar el cálculo. Al recorrer todo el fotograma, el método obtiene un margen para compensar estas variaciones involuntarias. Así, el sistema tolera los pequeños movimientos de la cabeza del alumno.

De la matriz de resultados de cada plantilla se toma el coeficiente más alto, que corresponde a la posición de mayor coincidencia. Este proceso se repite con las seis plantillas. La plantilla con el mayor de los seis valores indica la acción que realiza el alumno.

En las pruebas también se detectaron falsos positivos. En ocasiones, el algoritmo resultaba demasiado permisivo y confundía las orientaciones del ojo. Para reducir este efecto se incorporó un umbral de similitud, por debajo del cual toda coincidencia se descarta. A partir de los ensayos, el umbral se fijó por defecto en un 80 %, y el usuario puede ajustarlo desde la interfaz web. Con esta medida, el problema quedó resuelto.

Imagen del slider y el log indicando el mayor parecido (El texto verde que aparece abajo, Natty)

Tras numerosas evaluaciones, este método basado en la similitud de patrones resultó el más efectivo, estable y eficiente de los evaluados. Por lo tanto, es la solución implementada en el dispositivo final.

3.3. Arquitectura del sistema

El sistema definitivo se compone de tres tipos de elementos que se comunican a través de una red Wi-Fi local. El módulo de captura, montado sobre una gorra, obtiene las imágenes del ojo del alumno y las envía a su dispositivo. El módulo central genera la red, aloja la aplicación web y almacena la información del sistema. Además, cuenta con una pantalla táctil que permite configurar el dispositivo y consultar su estado sin depender de otro equipo. A través de los dispositivos cliente, los distintos usuarios acceden a la aplicación web desde su navegador. Solo uno de ellos corresponde al alumno, y en él se realiza la detección de la mirada.

(Diagrama de bloques del sistema definitivo)

Esta distribución permite que cada elemento cumpla una función específica. A continuación se describen las funciones de cada uno y la forma en que interactúan entre sí. Por último, se presenta el mapa de memoria del módulo central, que muestra dónde se almacena cada tipo de información.

3.3.1. Funciones de cada módulo

Módulo de captura

La función del módulo de captura es obtener imágenes del ojo del alumno en tiempo real y enviarlas de forma continua a su dispositivo. No realiza ningún procesamiento sobre ellas, ya que la detección de la mirada ocurre en el navegador. Además, ilumina el ojo con luz infrarroja. Esta iluminación evita las sombras que generan las fuentes de luz externas y mantiene constantes las condiciones de captura.

Módulo central

El módulo central coordina el funcionamiento de todo el sistema. En primer lugar, genera la red Wi-Fi local a la que se conectan los demás elementos. Así, el conjunto funciona sin acceso a internet. En segundo lugar, actúa como servidor. Entrega la aplicación web y la biblioteca *OpenCV.js* a cada dispositivo que las solicita y responde a sus pedidos de información.

Además, funciona como el núcleo de datos del sistema. Se encarga de autenticar a profesores y tutores, controlar las funciones disponibles para cada rol y administrar los cuestionarios. Durante las evaluaciones, recibe y registra las respuestas del alumno en una base de datos local. De esta forma, toda la información queda unificada en un único lugar, al cual los distintos usuarios acceden desde sus dispositivos.

Por último, el módulo central cuenta con una interfaz propia en su pantalla táctil. Desde ella se realiza la configuración inicial del dispositivo, se muestra un código QR para acceder a la aplicación web y se consulta el estado del sistema y el progreso de los exámenes.

Dispositivos cliente

Los dispositivos cliente son la vía de acceso de los usuarios al sistema. Cualquier equipo con navegador web puede cumplir esta función, sin instalar software adicional. Todos utilizan la misma aplicación web, pero la función de cada dispositivo depende de quien lo opere.

El dispositivo del alumno cumple un papel particular, ya que también realiza la detección de la mirada. Recibe el video del módulo de captura, lo compara con las plantillas de la calibración y traduce cada acción del ojo en una selección dentro de la interfaz. Así, el alumno puede resolver cuestionarios y responder preguntas solo con la mirada.

Desde estos dispositivos, los profesores y tutores gestionan los cuestionarios y realizan el seguimiento de los resultados del alumno, según los permisos de cada rol. Además, cada uno puede editar sus datos personales. Los profesores, a su vez, acceden a los datos de contacto de los tutores, mientras que los tutores consultan los de los profesores.

Por su parte, los compañeros acceden como invitados, sin registrarse, y pueden enviar preguntas informales al alumno. Estas preguntas no se guardan en la base de datos.

3.3.2. Interacción entre módulos

Toda la comunicación ocurre dentro de la red Wi-Fi que genera el módulo central. A través de ella, los módulos intercambian dos tipos de información. Por un lado, el video del ojo del alumno, que se transmite de forma continua. Por otro lado, los datos del sistema, como los cuestionarios, las respuestas y el estado de cada elemento.

El video viaja desde el módulo de captura hasta el dispositivo del alumno, que lo solicita solo durante la calibración y el examen. Como punto de acceso, el módulo central hace circular esta transmisión por su red, pero no la procesa ni la almacena.

Los datos del sistema, en cambio, son procesados por el módulo central. Al conectarse, cada dispositivo cliente descarga desde él la aplicación web. El dispositivo del alumno descarga además la biblioteca *OpenCV.js*, que conserva en el navegador para los accesos siguientes. Luego, cada vez que un usuario realiza una acción, su dispositivo envía la información al módulo central, que la gestiona y la pone a disposición de los demás.

Durante un examen, este intercambio ocurre de forma continua. El alumno confirma cada respuesta con un parpadeo, y su dispositivo la envía en ese momento al módulo central. Este la registra en la base de datos y actualiza el progreso del examen.

Las preguntas de los compañeros también pasan por el módulo central, pero no se guardan en la base de datos. El dispositivo del invitado la envía a ese módulo, que la conserva temporalmente en su memoria de trabajo y la entrega al dispositivo del alumno.

3.3.3. Mapa de memoria

El módulo central, basado en un microcontrolador *ESP32*, dispone de tres memorias con características distintas. La memoria flash retiene su contenido sin alimentación, pero su acceso es lento y su capacidad es reducida. La SRAM interna es rápida, aunque también tiene un tamaño limitado y se borra al apagar el equipo. La tarjeta microSD ofrece un espacio mucho mayor y tampoco pierde los datos. La ubicación de cada tipo de información responde a tres criterios. El primero es si debe conservarse al apagar el equipo. El segundo es con qué frecuencia cambia. El tercero es cuánto espacio ocupa.

Mapa de memoria del módulo central (Natty)

La memoria flash guarda información persistente, que rara vez cambia y que ocupa poco espacio. Se divide en tres partes, cada una destinada a un tipo de dato distinto.

Imagen del mapa de memoria (natty)

- NVS (*Non-Volatile Storage*): almacena las claves maestras de profesores y tutores y el nombre del alumno. Estos datos se definen una única vez, durante la configuración inicial, y se consultan al encender el equipo. La NVS está diseñada para guardar pocos valores de pequeño tamaño, por lo que resulta adecuada para esta configuración.
- *Firmware*: contiene el código ejecutable del sistema, que solo cambia al actualizar el programa.
- *Filesystem*: aloja las páginas de la aplicación web. Son archivos que no cambian durante el uso y que el módulo central entrega tal como están. Como son livianos, pueden ubicarse en la flash.

La SRAM interna funciona como memoria de trabajo. Guarda información que cambia de forma constante, que debe leerse con rapidez y que solo tiene sentido mientras el sistema está encendido.

- IRAM: contiene el código de las interrupciones y de las funciones críticas, que deben ejecutarse con rapidez.
- DRAM: almacena las variables de control, las sesiones activas con sus *tokens* y las preguntas de los invitados. También aloja la memoria dinámica que el sistema solicita durante su funcionamiento.

Las sesiones y sus *tokens* cambian cada vez que un usuario inicia o cierra sesión, y el módulo central los consulta en cada pedido que recibe. Guardarlos en una memoria permanente no tendría sentido, ya que dejan de ser necesarios al apagar el equipo. Al reiniciarse el sistema, los usuarios vuelven a iniciar sesión. Del mismo modo, las preguntas de los invitados no deben persistir, por lo que la DRAM resulta la ubicación adecuada.

La tarjeta microSD guarda la información que debe conservarse a largo plazo y que ocupa demasiado espacio para la flash.

- Base de datos: almacena los usuarios, los cuestionarios y las respuestas del alumno. Esta información debe mantenerse tras el apagado y crece con el uso, a medida que se crean

cuestionarios y se registran respuestas. Por eso requiere una memoria permanente y de gran capacidad.

- *OpenCV.js*: contiene la biblioteca de procesamiento de imágenes que se entrega al dispositivo del alumno. Su tamaño excede la capacidad del *Filesystem* de la flash, a diferencia de las páginas de la aplicación web, que son más livianas.

3.4. Microcontroladores y periféricos

Tanto el módulo de captura como el módulo central se implementan con una placa de desarrollo basada en un microcontrolador. La elección de estas placas responde a las restricciones del proyecto. Debían ser económicas y fáciles de conseguir. Además, ambas necesitaban conectividad Wi-Fi, ya que toda la comunicación del sistema ocurre de forma inalámbrica. El módulo de captura requería, a su vez, una interfaz física para conectar y manipular una cámara.

En esta sección se justifica la elección de la familia *ESP32* frente a otras alternativas del mercado. Luego se describen las placas utilizadas en cada módulo. Para el módulo de captura se detalla también la elección de la cámara. Para el módulo central se consideró, además, que debía contar con una ranura para tarjeta microSD y con una pantalla táctil integrada.

3.4.1. Elección de la familia ESP32

Para implementar los dos módulos se compararon distintas placas de desarrollo. Los criterios de evaluación fueron el costo, la conectividad Wi-Fi integrada y la facilidad de adquisición. También se consideró que cada módulo necesita periféricos específicos. El módulo de captura requiere una cámara, y el módulo central, un medio para almacenar la base de datos.

Se optó por la familia *ESP32*, de la empresa *Espressif Systems*, cuyas características se describieron en el capítulo 2. Sus placas integran la conectividad Wi-Fi en el mismo chip y tienen un costo de entre 7 y 15 USD a octubre de 2026. Además, se consiguen con facilidad, tanto en el mercado local como a través de tiendas internacionales. La familia también ofrece variantes que reúnen en un solo dispositivo los periféricos que necesita cada módulo, como una cámara, una pantalla o una ranura para tarjeta microSD. Esto permitió implementar ambos módulos con microcontroladores de la misma familia y, por lo tanto, con las mismas herramientas de desarrollo.

A continuación se detallan las alternativas consideradas y los motivos de su descarte. Los precios corresponden a las tiendas oficiales de cada fabricante a octubre de 2026.

- *Arduino Uno*: es una de las placas más difundidas, pero no cuenta con conectividad Wi-Fi. Además, sus 2 KB de memoria RAM resultan insuficientes para alojar una aplicación web.
- *Arduino UNO R4 WiFi*, *Arduino Nano 33 IoT* y *Arduino Nano ESP32*: incorporan Wi-Fi y cuentan con el respaldo de la marca *Arduino*. Sin embargo, su costo, de entre 18 y 28 USD, es considerablemente mayor que el de las placas de *Espressif*. Además, ninguna integra una interfaz para cámara ni una ranura para tarjeta microSD.
- *Raspberry Pi Pico 2 W*: incorpora Wi-Fi y tiene un costo de 7 USD. Sin embargo, tampoco dispone de interfaz para cámara ni de lector de tarjetas microSD. Por lo tanto, ambos módulos requerirían componentes externos adicionales.
- *Raspberry Pi Zero 2 W*: incluye Wi-Fi y un conector para cámara, con un costo de 15 USD. Sin embargo, no es un microcontrolador, sino una computadora que requiere un

sistema operativo, y el proyecto se plantea sobre microcontroladores de bajos recursos. Además, la cámara se adquiere por separado a un precio de entre 25 y 35 USD.

Por último, las placas de la familia *ESP32* son compatibles con las bibliotecas de *Arduino*. Esto agilizó el desarrollo y las pruebas durante las etapas iniciales del proyecto.

3.4.2. Módulo de captura

El módulo de captura se implementa con la placa *ESP32-CAM* (Fig. 3.14). Esta combina un microcontrolador *ESP32* con una interfaz para cámara y se vende junto con el módulo *OV2640*. También incorpora una memoria PSRAM de 4 MB, que permite almacenar los fotogramas antes de transmitirlos.



Figura 3.14: Placa *ESP32-CAM* con su cámara.

Dentro de la familia *ESP32* existen otras placas que también incorporan un conector para cámara, como las basadas en el *ESP32-S3*. Estos modelos ofrecen mayor capacidad de procesamiento y memoria, pero tienen un costo mayor. Una placa genérica con el módulo *ESP32-S3-WROOM N16R8* y cámara cuesta aproximadamente 20 USD a octubre de 2026. El *ESP32-CAM*, en cambio, se basa en el *ESP32* clásico y es la más básica y económica de las opciones con cámara de la familia. Como el módulo de captura solo obtiene y transmite imágenes, no requiere la capacidad adicional del *ESP32-S3*, por lo que esta alternativa resulta suficiente.

Frente a placas de otras marcas, la principal ventaja del *ESP32-CAM* es el bajo costo de inclusión de una cámara. En esos modelos, incorporarla requiere módulos adicionales o adquirirla por separado, lo que eleva el precio final. La siguiente tabla compara el costo del *ESP32-CAM* con el de estas alternativas, a octubre de 2026.

Tabla 3.1: Costo de placas compatibles con cámaras, a octubre de 2026

Opción	**Placa**	**Cámara**	**Costo total**
ESP32-CAM	7 a 10 USD	Incluida (*OV2640*)	7 a 10 USD
Raspberry Pi Pico 2 W	7 USD	*Arducam Mini 2MP*, 26 USD	33 USD
Arduino UNO R4 WiFi	27,50 USD	*Arducam Mini 2MP*, 26 USD	53,50 USD
Raspberry Pi Zero 2 W	15 USD	*Camera Module 3*, 25 a 35 USD	40 a 50 USD

Como se mencionó con anterioridad, se utilizará cámara *OV2640*, que tiene una resolución máxima de 2 megapíxeles (Fig: 3.15) . El sistema no requiere una resolución alta, ya que las imágenes se procesan a 320x240 píxeles. Lo que sí resulta importante es su campo de visión. Al estar montada en la visera de una gorra, la cámara debe captar el ojo sin incluir estímulos ajenos. La lente de la *OV2640* tiene un ángulo de visión de 68°, dentro del rango de entre 60 y

120 grados establecido para esta tarea. Además, se le retiró el filtro infrarrojo para que registre la luz del LED.

También se evaluó la cámara *OV3660*, de 3 megapíxeles, que ofrece resultados similares. Sin embargo, la *OV2640* incluida con la compra de la placa resultó adecuada. Utilizar otro sensor habría requerido adquirirla por separado, lo que aumentaría el costo total sin aportar una mejora significativa.

Se consideró, además, la posibilidad de utilizar una cámara *OV2640* con un conector flex de meyor longitud. Esto hubiera permitido una mayor flexibilidad a la hora del montaje final. Sin embargo, se optó por utilizar la cámara de flex corto ue llegó junto a la compra del *ESP32-CAM* para abaratar costos.



Figura 3.15: Cámaras OV2640 con distintos ángulos de visión y longitud de conector

3.4.3. Módulo central

El módulo central se implementa con la placa *ESP32-2432S028R*, conocida como *ESP32-CYD* (*Cheap Yellow Display*). Esta placa se basa en el módulo *ESP32-WROOM-32*, con el *ESP32* clásico, e integra una pantalla táctil de 2,8 pulgadas y una ranura para tarjeta microSD. Su costo es de entre 13 y 18 USD a octubre de 2026. Este chip resulta suficiente para las tareas del módulo central, ya que el procesamiento de las imágenes ocurre en el dispositivo del alumno. Por este motivo, no fue necesario recurrir a un microcontrolador más potente, como el *ESP32-S3*.

El principal criterio de elección fue la ranura para tarjeta microSD. El módulo central almacena una base de datos que crece con el uso y aloja la biblioteca *OpenCV.js*, cuyo tamaño excede la capacidad de la memoria flash. La microSD ofrece el espacio necesario para ambos archivos sin agregar componentes externos. Otro aspecto favorable es la versatilidad de almacenamiento de la placa, descrita en el mapa de memoria.

La pantalla táctil, en cambio, no era imprescindible. Algunas de sus funciones, como la consulta del estado del sistema, podrían haberse implementado en la aplicación web. Sin embargo, la placa la incluye y la diferencia de precio con otros modelos *ESP32* no es significativa. Por este motivo, se decidió aprovecharla como una funcionalidad adicional. Permite configurar datos de inicio y supervisar el estado del sistema directamente desde el módulo central.

Imagen del ESP32-CYD? (Ya la mostramos, pero podriamos poner otra. Natty)

3.5. Red y comunicación

En esta sección se describe la infraestructura de comunicación del sistema. Primero se presenta la red Wi-Fi local que genera el módulo central y la forma en que los usuarios acceden a la aplicación. Luego se detalla la transmisión del video desde el módulo de captura. Por último, se explica cómo se comunican los dispositivos cliente con el módulo central.

3.5.1. Red Wi-Fi local

El módulo central genera una red Wi-Fi propia mediante el modo punto de acceso (SoftAP) del *ESP32*. La red se identifica con el nombre LEVI y está protegida con contraseña. Opera en la banda de 2,4 GHz, la única que admite el *ESP32*, sobre el canal 1. Además, acepta hasta cuatro clientes conectados en simultáneo. Si bien el *ESP32* permite más conexiones, cada dispositivo conectado ocupa parte de su memoria, que es acotada. Por este motivo, se fijó este límite. Durante el uso del sistema intervienen tres de ellos, que son el módulo de captura, el dispositivo del alumno y el de un usuario externo. El cuarto lugar funciona como margen, en caso de que se conecte un equipo adicional a la red.

Dentro de esta red, el módulo central ocupa la dirección 192.168.4.1, que funciona además como puerta de enlace. El módulo de captura, en cambio, utiliza una dirección IP estática, la 192.168.4.50. Así, el dispositivo del alumno siempre encuentra el video en la misma ubicación. Los dispositivos cliente, por su parte, reciben una dirección de forma dinámica al conectarse. El módulo central asigna estas direcciones en orden creciente, a partir de la 192.168.4.2. En consecuencia, para la cámara se eligió un valor alejado del inicio. Como la red tolera como máximo cuatro conexiones, resulta prácticamente imposible que la asignación dinámica alcance esa dirección y que dos equipos compartan la misma.

(Imagen de la red Wi-Fi local) (Natty)

Ambas placas utilizan un archivo de configuración con los mismos parámetros de red. Este archivo define el nombre y la contraseña de la red, junto con las direcciones IP de cada módulo. De esta forma, el módulo de captura dispone de las credenciales necesarias para conectarse, y cada placa conoce de antemano la dirección de la otra. Al encenderse, el módulo de captura se une a la red en modo estación. Si la conexión falla, vuelve a intentarlo hasta lograrla.

Una vez establecida la red, el módulo central supervisa su estado. Por un lado, registra la cantidad de dispositivos conectados a partir de la tabla de clientes que mantiene el punto de acceso. Cuando un celular o una computadora abandona la red, envía un aviso de desconexión y el punto de acceso lo quita de la tabla de inmediato. En cambio, si un equipo deja de responder sin avisar, permanece en la tabla hasta que transcurren cinco minutos sin actividad. Esta situación se da cuando un dispositivo sale del área de cobertura o se apaga de forma repentina.

Por otro lado, el módulo de captura se encuentra en este último caso cada vez que se apaga, ya que no se desconecta de la red antes de hacerlo. Si solo se consultara la tabla, la cámara seguiría figurando como conectada durante varios minutos. Por este motivo, su estado se verifica de forma independiente. Con un intervalo de dos segundos, el módulo central envía un mensaje de eco (*ping*) a la dirección de la cámara. Si no obtiene respuesta en 500 milisegundos, la considera desconectada. Así, el sistema conoce su estado prácticamente en tiempo real. Para los demás dispositivos esta verificación no resulta necesaria, ya que normalmente se desconectan de forma explícita. La pantalla táctil utiliza ambos datos para informar el estado del sistema.

3.5.2. Acceso a la aplicación

Para utilizar el sistema, el usuario conecta su dispositivo a la red LEVI y luego ingresa a la aplicación web desde su navegador.

El módulo central publica el nombre levi.local mediante mDNS, descrito en el capítulo 2. Así, se accede a la aplicación al escribir ese nombre en la barra de direcciones o al escanear un código QR que lo contiene. Si un usuario ingresa con la dirección IP, el servidor lo redirige automáticamente a levi.local. De esta forma se mantiene una única dirección de acceso.

En el primer acceso, el navegador obtiene los archivos de la aplicación desde el *Filesystem* de la memoria flash del módulo central. La aplicación funciona como una página única (*Single Page Application*). Por lo tanto, se transfiere una sola vez y, a partir de ese momento, cambia de pantalla sin pedir nuevas páginas al servidor. Si se solicita una ruta que no corresponde a ningún archivo, el servidor responde con esa misma página, que luego muestra la pantalla adecuada.

El dispositivo del alumno descarga además la biblioteca *OpenCV.js*, que el módulo central entrega desde la tarjeta microSD. Debido a su tamaño, es el archivo más pesado que transmite el sistema. Por este motivo, el navegador lo guarda en su almacenamiento, de modo que en los accesos siguientes no vuelve a solicitarlo.

(Imagen del acceso a la aplicación) (Natty)

3.5.3. Transmisión de video

El módulo de captura transmite el video mediante la técnica MJPEG, detallada en el capítulo 2. Se optó por esta técnica porque el navegador la reproduce de forma nativa, sin bibliotecas ni código adicional. Para implementarla, el módulo ejecuta un servidor HTTP propio, independiente del que funciona en el módulo central. Este servidor atiende las solicitudes en el puerto 80 de la dirección fija del módulo de captura, en la ruta /stream, y entrega un único flujo de video a la vez, que corresponde al dispositivo del alumno.

La cámara toma cada fotograma con una resolución de 320x240 píxeles y lo comprime en formato JPEG antes de enviarlo. Esta compresión la realiza el codificador por hardware que incorpora el sensor *OV2640*, por lo que no consume recursos del microcontrolador. Además, se configuró en un nivel intermedio, que reduce el tamaño de cada imagen sin perder los detalles del ojo necesarios para la detección. Junto con cada fotograma, el módulo indica su tamaño, lo que permite al navegador reconocer dónde termina uno y comienza el siguiente.

La aplicación web solicita el video únicamente durante la calibración y el examen. Mientras el dispositivo del alumno mantiene la conexión, el servidor envía los fotogramas de manera continua. Fuera de esas etapas, la aplicación cierra la conexión y la cámara deja de transmitir, por lo que no se ocupa la red de forma innecesaria. Si la conexión se interrumpe por otro motivo, la transmisión también se detiene hasta recibir una nueva solicitud.

(Imagen de la transmisión de video) (Natty)

3.5.4. Comunicación con el módulo central

Los dispositivos cliente se comunican con el módulo central mediante peticiones HTTP, a través de la red local. El servidor web del módulo central las atiende en el puerto 80, el estándar de este protocolo. Las peticiones de datos se dirigen a rutas que comienzan con /api/, mientras que el resto corresponde a los archivos de la aplicación. La información viaja en formato JSON, descrito en el capítulo 2. El servidor procesa las peticiones de forma asíncrona, por lo que puede atender a varios dispositivos sin bloquearse.

La comunicación siempre se inicia desde los dispositivos cliente. Esto se debe a que HTTP opera bajo el modelo cliente-servidor presentado en el capítulo 2, en el que el cliente es quien inicia cada intercambio. El módulo central solo puede enviar información como respuesta a una solicitud, y no por iniciativa propia. Por este motivo, los dispositivos consultan al servidor de manera periódica para mantenerse actualizados.

El dispositivo del alumno realiza una consulta cada dos segundos. En ella informa que sigue activo y obtiene la situación actual, como la pregunta del examen en curso o la pregunta enviada por un invitado. Los dispositivos de los demás usuarios, por su parte, envían cada cinco segundos un aviso similar, denominado *heartbeat*, que indica que su sesión continúa activa. El uso de estos avisos en la gestión de las sesiones se describe en la sección del *backend*.

(Imagen de la comunicación con el módulo central) (Natty)

3.6. Aplicación web

La aplicación web reúne todo el software que permite a los usuarios interactuar con el sistema. Se organiza en tres capas, que son la base de datos, el *backend* y el *frontend*. Las dos primeras se ejecutan en el módulo central. El *backend* forma parte del *firmware*, escrito en C++, y la base de datos es un archivo *SQLite* ubicado en la tarjeta microSD, que el propio *firmware* lee y escribe. El *frontend*, en cambio, funciona en el navegador de cada dispositivo cliente.

(Diagrama de la arquitectura de la aplicación web) (Natty)

A continuación se describe cada capa, en el orden en que se apoyan unas sobre otras. Primero se presenta la base de datos, luego el *backend* y, por último, el *frontend*.

3.6.1. Base de datos

La base de datos almacena de forma permanente la información del sistema, que incluye los usuarios registrados, los cuestionarios (con sus preguntas y opciones) y los resultados del alumno. Así, estos registros se conservan aunque el módulo central se apague. En esta subsección se explica primero cómo se creó y configuró la base, y luego la estructura de sus tablas.

Creación y configuración

La base de datos del sistema utiliza *SQLite*, el motor embebido presentado en el capítulo 2. Se lo eligió porque no requiere un servidor independiente y funciona con los recursos acotados del módulo central. Toda la información se guarda en un único archivo, *levi.db*, ubicado en la tarjeta microSD.

La estructura de la base se diseñó en una computadora mediante la herramienta *DB Browser for SQLite*. Allí se definieron las tablas, sus campos, las relaciones y las restricciones. Luego, el archivo resultante se exportó y se copió a la tarjeta microSD. De esta forma, el *firmware* no necesita crear las tablas al iniciar, sino que abre directamente el archivo existente.

Al abrir la base, el *firmware* ajusta algunas opciones de funcionamiento de *SQLite* (*PRAGMA*). Estas opciones responden a la memoria acotada del microcontrolador y a las características de la tarjeta microSD.

- Claves foráneas (*foreign_keys*): se activa su verificación, que *SQLite* deshabilita por defecto. Así se garantiza que las relaciones entre tablas se respeten.
- Tamaño de la caché (*cache_size*): se limita a 16 KB para reducir el consumo de RAM.

- Almacenamiento temporal (*temp_store*): los datos intermedios que *SQLite* genera durante las consultas se guardan en la tarjeta y no en la RAM.
- Registro de transacciones (*journal_mode*): *SQLite* conserva el archivo de registro de cambios entre una transacción y otra, en lugar de borrarlo y crearlo cada vez. Esto reduce las escrituras en la microSD.
- Sincronización (*synchronous*): se emplea un nivel intermedio, que equilibra la protección ante cortes de energía con la velocidad de escritura.
- Tiempo de espera (*busy_timeout*): si la base está ocupada, la operación espera hasta tres segundos antes de fallar.

(Captura de DB Browser for SQLite con la estructura de la base) (Natty)

Estructura de las tablas

La base de datos se compone de cuatro tablas, que son: usuarios, cuestionarios, preguntas y opciones. En todas ellas, cada registro se identifica con un número entero (su clave primaria), que *SQLite* asigna de forma automática al insertarlo. A continuación se describe el contenido de cada tabla.

Usuarios

La tabla de usuarios guarda los datos de los profesores y los tutores. El alumno no forma parte de ella, ya que cada dispositivo pertenece a un único alumno y no es necesaria una tabla para identificarlo. Por este motivo, su nombre se define en la configuración inicial y se guarda en la NVS del módulo central. Los invitados tampoco se almacenan, porque acceden sin registrarse. Cada usuario cuenta con los siguientes campos.

- Usuario: nombre que se utiliza para iniciar sesión. Debe ser único en todo el sistema y tener entre 4 y 30 caracteres, sin espacios.
- Nombre y apellido: datos personales del usuario.
- Rol: indica si se trata de un profesor o de un tutor. No admite otros valores.
- Referencia: su significado depende del rol. Para un profesor indica la materia que dicta, mientras que para un tutor indica su relación con el alumno, como madre o padre.
- Contacto: medio para comunicarse con el usuario, como un teléfono o un correo electrónico.
- *Hash* de la contraseña: resultado de aplicar la función de *hash* a la contraseña.
- *Salt*: cadena aleatoria que se combina con la contraseña antes de calcular el *hash*.

Ninguno de los campos de texto puede quedar vacío.

Las contraseñas no se almacenan en texto plano. En su lugar, se guarda el resultado de aplicarles la función de *hash* SHA-256, descrita en el capítulo 2. Esta función es irreversible, por lo que a partir del valor guardado no es posible recuperar la contraseña original. De esta forma, aun si alguien accediera al archivo de la base de datos, no podría conocer las contraseñas de los usuarios. Esto resulta relevante porque los usuarios suelen repetir sus contraseñas en otros servicios.

Para su implementación se utilizó la biblioteca *mbedTLS*, que ya forma parte del entorno de desarrollo del *ESP32*. Además, este microcontrolador cuenta con un acelerador por hardware

para el algoritmo SHA. Así, el cálculo no requiere bibliotecas adicionales y se realiza con rapidez, sin comprometer los recursos del módulo central.

El *salt* se genera de forma aleatoria al crear cada usuario. Sin este valor, dos usuarios con la misma contraseña tendrían el mismo *hash*, y quien accediera a la base podría advertirlo. Con el *salt*, en cambio, cada resultado es distinto. Al iniciar sesión, el sistema combina la contraseña ingresada con el *salt* almacenado, repite el cálculo y compara el resultado con el valor guardado.

Cuestionarios

La tabla de cuestionarios guarda tanto la configuración de cada evaluación como su resultado. Considerando que cada dispositivo pertenece a un único alumno, no hace falta una tabla aparte para los resultados, sino que se pueden almacenar directamente en el cuestionario. Cada uno cuenta con los siguientes campos.

- Profesor: referencia al profesor que creó el cuestionario.
- Título: nombre del cuestionario. No puede quedar vacío ni repetirse entre los cuestionarios de un mismo profesor.
- Puntaje para aprobar: puntaje mínimo que debe alcanzar el alumno. No puede ser negativo.
- Estado: etapa en la que se encuentra el cuestionario.
- Puntaje obtenido: resultado del alumno, que se completa al finalizar.
- Fecha de inicio: momento en que el cuestionario se inició por primera vez.
- Tiempo: duración de la resolución, expresada en segundos. No contabiliza los períodos en que el cuestionario estuvo pausado.

El estado de un cuestionario solo puede tomar uno de los siguientes valores.

- Pendiente: el cuestionario fue creado, pero todavía no se inició.
- En progreso: el alumno lo está resolviendo. Solo un cuestionario puede encontrarse en este estado a la vez.
- Pausado: la resolución se interrumpió de forma temporal y puede retomarse.
- Finalizado: la resolución terminó y el resultado quedó registrado.

Al crearse, todo cuestionario queda en estado *pendiente*. Cuando el profesor lo inicia, pasa a estar *en progreso* y se registra la fecha de inicio. Desde ese estado puede pasar a *pausado* en tres casos. El primero es cuando el profesor lo pausa voluntariamente. El segundo, cuando el alumno pierde la conexión con el sistema. El tercero, cuando se inicia o reanuda otro cuestionario, ya que solo uno puede estar en progreso a la vez. Un cuestionario pausado vuelve a estar en progreso cuando el profesor lo reanuda.

Por último, el cuestionario pasa a *finalizado* cuando el alumno responde la última pregunta o cuando el profesor lo da por terminado (independiente de que esté en progreso o pausado). En ese momento se calculan y guardan el puntaje obtenido y el tiempo de resolución. Un cuestionario finalizado no vuelve a cambiar de estado.

(Diagrama de transición de estados del cuestionario) (Natty)

Preguntas

La tabla de preguntas guarda el contenido de cada pregunta y la respuesta del alumno. Cada una cuenta con los siguientes campos.

- Cuestionario: referencia al cuestionario al que pertenece la pregunta.
- Enunciado: texto de la pregunta. No puede quedar vacío.
- Puntaje si es correcta: puntos que se suman al resultado cuando el alumno elige la opción correcta.
- Puntaje si es incorrecta: puntos que se restan al resultado cuando el alumno elige otra opción. Permite penalizar las respuestas erróneas.
- Opción correcta: referencia a la opción que responde correctamente la pregunta.
- Opción elegida: referencia a la opción que seleccionó el alumno. Permanece vacía hasta que responde.

La opción correcta se registra en la pregunta y no como una marca dentro de cada opción. De esta forma, cada pregunta tiene siempre una única opción correcta. Por su parte, la opción elegida también se guarda en la pregunta, ya que el sistema contempla un único alumno.

Opciones

La tabla de opciones guarda las respuestas posibles de cada pregunta. Cada pregunta tiene entre dos y cuatro opciones, cuyos campos son los siguientes.

- Pregunta: referencia a la pregunta a la que pertenece la opción.
- Texto: contenido de la opción. No puede quedar vacío.

Relaciones

Las tablas se vinculan mediante claves foráneas. Un profesor puede crear varios cuestionarios, un cuestionario contiene varias preguntas y cada pregunta reúne varias opciones. Además, cada pregunta se relaciona con dos de sus opciones, la correcta y la elegida por el alumno.

(Diagrama entidad-relación de la base de datos) (Natty)

Estas relaciones definen el orden en que se crean los registros. Un cuestionario requiere que su profesor ya exista, y una pregunta requiere que exista su cuestionario. En el caso de las preguntas, primero se crea la pregunta, luego sus opciones y, por último, se indica cuál de ellas es la correcta.

Las relaciones también determinan qué ocurre al eliminar un registro. Si se elimina un profesor, se borran en cascada sus cuestionarios, junto con sus preguntas y opciones. De esta forma no quedan registros huérfanos en la base. En cambio, si se elimina una opción, la pregunta no se borra, sino que la referencia a esa opción queda vacía.

3.6.2. Backend

Las herramientas habituales para desarrollar un *backend*, como *Node.js* o *Django*, requieren un sistema operativo y una cantidad de memoria muy superior a la del módulo central. Por este motivo, el *backend* se escribió en C++, como parte del mismo *firmware* que controla la red, la pantalla y la tarjeta microSD.

Su código se organiza según el patrón Controlador-Servicio-Repositorio, presentado en el capítulo 2. Cada petición que llega al módulo central atraviesa estas capas en orden. Primero la recibe un controlador, luego un servicio aplica las reglas del sistema y, por último, un repositorio

accede a la base de datos. Las capas intercambian información mediante modelos, que son estructuras de datos comunes a todas ellas. La gestión de las sesiones, en cambio, funciona como un componente aparte que utiliza tanto los controladores como los servicios.

Modelos

Los modelos son las estructuras con las que el *backend* agrupa la información para transmitirla de una capa a otra. No realizan operaciones, sino que solo contienen datos. Según su función se distinguen tres tipos.

- Entidades: reflejan los registros de la base de datos según se almacenan. Existe una por cada tabla, con su misma estructura.
- Objetos de transferencia de datos (*Data Transfer Object*, DTO): transportan la información que se intercambia con el *frontend*, organizada según lo que requiere cada caso y no según cómo está guardada. De esta forma, lo que se expone hacia afuera queda separado del modelo interno. Así, un DTO puede agregar valores calculados, como el resultado de un cuestionario, que reúne el puntaje obtenido, el máximo posible y el necesario para aprobar. También puede omitir campos reservados, como la opción correcta en la pregunta que recibe el alumno.
- Resultados de operación: indican si una acción se completó correctamente o, en caso contrario, cuál fue el error. Los repositorios y los servicios los devuelven a la capa que les hizo el pedido.

Por la memoria limitada del módulo central, los modelos que agrupan varios elementos deberán tener una capacidad máxima fija. De esta forma, un cuestionario admite hasta 20 preguntas, cada pregunta hasta cuatro opciones y cada listado hasta 50 cuestionarios.

Controladores

Los controladores constituyen el punto de entrada del *backend*. Cada uno agrupa las rutas de un recurso del sistema y existen cuatro: controlador de autenticación, de usuarios, de cuestionarios y del alumno. Todas las rutas comienzan con el prefijo /api/, lo que permite al servidor distinguirlas de los archivos de la aplicación web.

Ante cada petición, el controlador verifica primero la sesión y el rol del usuario. Luego interpreta los datos recibidos en formato JSON, invoca al servicio correspondiente y, por último, construye la respuesta. En resumen, los controladores no aplican reglas del sistema, sino que solo traducen las peticiones HTTP en llamadas a los servicios.

La acción que se solicita sobre cada recurso se indica mediante el método HTTP.

- GET: consulta información, como el listado de cuestionarios o el estado del examen.
- POST: crea registros, como un usuario o un cuestionario; o ejecuta acciones, como iniciar sesión o registrar una respuesta.
- PUT: reemplaza todos los datos de un registro por los que se envían. Se utiliza, por ejemplo, para editar un cuestionario pendiente. En ese caso, el profesor envía nuevamente el cuestionario completo, que sustituye al anterior.
- PATCH: modifica solo una parte de un registro. Se utiliza para cambiar el estado de un cuestionario, es decir, para iniciarlo, pausarlo, reanudarlo o finalizarlo, sin alterar su contenido.

- DELETE: elimina un registro.

Todas las respuestas comparten el mismo formato. Incluyen un campo que indica si la operación tuvo éxito y, ante un error, un mensaje que lo describe. Además, el código de estado HTTP señala el tipo de problema.

- 400: los datos enviados son inválidos o están incompletos.
- 401: la sesión no existe o expiró.
- 403: el rol del usuario no tiene permiso para la acción.
- 404: el recurso solicitado no existe.
- 409: la acción entra en conflicto con la situación actual del sistema, como cuando un invitado intenta ingresar durante un examen.

Gracias a este formato común, el *frontend* interpreta todas las respuestas de la misma manera.

Al crear o editar un cuestionario, el cuerpo de la petición puede superar el tamaño que el servidor entrega de una sola vez. En ese caso llega dividido en fragmentos y el controlador los reúne antes de interpretar el JSON.

Servicios

Los servicios concentran las reglas del sistema. Existen cuatro: servicios de autenticación, de usuarios, de cuestionarios y de respuestas del alumno. Cada uno valida los datos que recibe, decide si la acción solicitada está permitida y coordina a los repositorios necesarios para llevarla a cabo.

Autenticación

El servicio de autenticación verifica las credenciales de profesores y tutores mediante el procedimiento de *hash* explicado en la sección de base de datos. Si el usuario no existe o la contraseña no coincide, responde con el mismo mensaje en ambos casos. De esta forma no revela qué nombres de usuario están registrados. Cuando la verificación es correcta, solicita al gestor de sesiones que abra una nueva sesión.

Usuarios

Para registrar un usuario o restablecer una contraseña olvidada, el servicio exige la clave maestra correspondiente al rol, definida en la configuración inicial del sistema. Por otra parte, cada vez que se cambia una contraseña, genera un nuevo *salt*.

Al eliminar un usuario, el servicio no recurre al borrado en cascada de la base de datos. En su lugar, borra de manera secuencial las opciones, las preguntas y los cuestionarios del profesor, y por último el usuario. Así, cada operación afecta a pocos registros, lo que optimiza el uso de la memoria.

Cuestionarios

El servicio de cuestionarios aplica las transiciones de estado descritas en la sección de base de datos. Antes de cada cambio, comprueba que el cuestionario pertenezca al profesor que lo solicita y que su estado actual permita la acción. Además, como solo un cuestionario puede estar en progreso, al iniciar o reanudar uno pausa automáticamente al que estuviera en curso. La edición se admite solo en cuestionarios pendientes y la eliminación en cualquier estado, salvo en progreso o pausado. Esta última sigue el mismo criterio de borrado secuencial que la de usuarios.

Al crear o editar un cuestionario, el servicio verifica que tenga título y al menos una pregunta, y que cada pregunta tenga entre dos y cuatro opciones, con una sola correcta. Si alguna operación falla durante el guardado, elimina lo que se había creado. En la edición, primero guarda las preguntas nuevas y luego borra las anteriores. De esta forma, ante un error, el cuestionario conserva su contenido original.

Este servicio también lleva el cronómetro del examen. El tiempo se mide con el reloj interno del microcontrolador, a partir de las consultas periódicas que realiza el dispositivo del alumno. Entre una consulta y la siguiente se suma el tiempo real transcurrido, pero si ese intervalo supera un valor máximo solo se contabiliza dicho máximo. Así, una demora breve de la red no altera la medición, y un corte de conexión no se suma como tiempo de examen. Al pausar, el tiempo acumulado se guarda en la base de datos, y al reanudar se recupera.

Por último, al finalizar un cuestionario el servicio calcula el puntaje: ssuma los puntos de cada respuesta correcta, resta los de cada incorrecta y no considera las preguntas sin responder. El resultado se guarda junto con el tiempo de resolución.

Respuestas

El servicio de respuestas determina qué se muestra en el dispositivo del alumno. En cada consulta, si hay un examen en progreso, entrega la primera pregunta sin responder. Si no lo hay y un invitado cargó una pregunta, entrega esa pregunta. En cualquier otro caso indica que no hay nada pendiente.

Al recibir una respuesta, comprueba que la pregunta pertenezca al examen en curso y que la opción corresponda a esa pregunta. Luego la guarda y, si era la última, finaliza el cuestionario y devuelve el resultado. En cambio, la respuesta a una pregunta de invitado no se guarda y la pregunta se descarta.

Repositorios

La capa de respositorios es la única que accede a la base de datos. Existe uno por cada tabla, encargado de leer, insertar, modificar y eliminar sus registros. De esta forma, las instrucciones SQL quedan reunidas en un solo lugar y el resto del *backend* trabaja con modelos sin conocer cómo se almacenan los datos.

Todos los repositorios comparten una única conexión con la base de datos. El módulo central la abre al encender, configura las opciones de *SQLite* descritas en la sección de base de datos y la mantiene abierta mientras funciona. Así se evita abrir y cerrar el archivo en cada petición, lo que demandaría tiempo y memoria adicionales.

Las consultas se ejecutan como sentencias preparadas. Primero se define la instrucción con marcadores en lugar de los valores y luego se asocian a ellos los datos recibidos. Como estos nunca forman parte del texto de la instrucción, se impide la inyección de SQL, es decir, que un dato ingresado por el usuario se interprete como una orden para la base.

Los listados de cuestionarios, por su parte, se obtienen mediante sentencias simples. En lugar de combinar varias tablas en una sola operación, primero se leen los cuestionarios y luego se obtiene por separado, para cada uno, la cantidad de preguntas y la materia del profesor (esto último solo para el caso de los tutores). Este enfoque implica más accesos a la base, pero cada uno procesa menos datos a la vez, lo que reduce el consumo de RAM.

Gestión de sesiones

La gestión de sesiones controla quién está conectado al sistema y con qué rol. Funciona como un componente independiente de las capas anteriores, ya que lo utilizan tanto los controladores, para verificar cada petición, como los servicios para abrir y cerrar sesiones o consultar su estado.

Tokens de sesión

Al iniciar sesión, el módulo central genera un *token*, que es una cadena de 32 caracteres hexadecimales obtenida con el generador de números aleatorios por hardware del *ESP32*. El *token* se envía al navegador y éste lo incluye en el encabezado de cada petición posterior. El servidor, por su parte, guarda en la RAM ese *token* junto con el rol del usuario y el momento de su última actividad. Para validar una petición basta con comparar el valor recibido con el almacenado. Como estos datos residen en la RAM, las sesiones se pierden al reiniciar el módulo central y los usuarios deben volver a ingresar.

En aplicaciones web con muchos usuarios suele emplearse el *JSON Web Token* (JWT), un *token* firmado que el servidor valida sin almacenar las sesiones. En este caso no resulta conveniente, ya que el sistema necesita registrar las sesiones activas para aplicar las prioridades y la expiración que se describen a continuación.

Lugares de sesión

El sistema admite dos sesiones simultáneas. Una corresponde al dispositivo del alumno, y la otra al panel, que puede ocupar un profesor, un tutor o un invitado. Si el lugar del alumno ya está ocupado, se rechaza cualquier nuevo ingreso.

En el panel, los roles tienen distinta prioridad. El tutor tiene la mayor, seguido por el profesor y, por último, el invitado. Si alguien intenta ingresar mientras el panel está ocupado, el sistema compara ambos roles. Cuando el nuevo usuario tiene mayor prioridad, la sesión anterior se cierra y el lugar pasa a él. En cambio, si su prioridad es igual o menor, el ingreso se rechaza.

Expiración

Cada sesión registra el momento de su última actividad, que se actualiza con cada petición válida y con los avisos periódicos descritos en la sección de red. El módulo central revisa estos registros de forma continua. Cuando una sesión supera su tiempo límite sin actividad, la cierra y libera su lugar. Cada tipo de sesión cuenta con su propio tiempo límite. Así, ningún lugar queda ocupado de forma indefinida cuando un usuario cierra el navegador sin salir del sistema.

Si la sesión que expira es la del alumno y hay un examen en progreso, el sistema lo pausa automáticamente. De esta forma, el cronómetro se detiene hasta que el profesor lo reanuda.

Invitado

El invitado ingresa sin registrarse y ocupa el panel con la menor prioridad. Mientras hay un examen en progreso no puede ingresar ni enviar preguntas. Su pregunta se guarda solo en la RAM, junto con su sesión, y se borra al cerrarse. Así, el modo invitado no interfiere con las evaluaciones ni deja registros en la base de datos.

Verificación de peticiones

Antes de procesar una petición protegida, los controladores recurren a un componente intermedio (*middleware*). Este extrae el *token* del encabezado y lo compara con el de la sesión correspondiente. Si no coincide o la sesión expiró, responde con el código 401. Si el *token* es válido, pero el rol no tiene permiso para la acción, responde con el código 403. Cuando la verificación es correcta, también actualiza el momento de la última actividad.

3.6.3. Frontend

El *frontend* es la parte visual del código. Su función es actuar como intermediario entre el usuario y el *backend*, que gestiona los datos y los permisos. Se ejecuta en el navegador de cada dispositivo cliente y se organiza según la arquitectura basada en componentes presentada en el capítulo 2.

Entorno y herramientas

Para su desarrollo se optó por *Preact*, una biblioteca de *JavaScript* de código abierto, diseñada para construir interfaces de usuario dinámicas e interactivas a partir de componentes. Ofrece las mismas capacidades de desarrollo que *React*, la biblioteca más difundida para este fin, creada y mantenida por *Meta*, pero con una sobrecarga mucho menor. Al prescindir de funciones accesorias, el código resultante es más liviano, lo que reduce el tamaño de los archivos que el módulo central debe almacenar y transmitir al navegador.

El código fuente se escribió en computador, utilizando el software *Visual Studio Code*. Sin embargo, en ese formato no puede ser interpretado directamente por los navegadores ni almacenarse de forma eficiente en un microcontrolador. Por este motivo, antes de cargarlo en el módulo central, se utilizó la herramienta de construcción *Vite*, que compiló, minimizó y empaquetó todo el código en archivos estáticos. El resultado fue un documento HTML, una hoja de estilos CSS y un archivo *JavaScript*.

Estos archivos se cargan por USB en el *Filesystem* de la memoria flash del módulo central. Esta partición es independiente de la que contiene el *firmware*, por lo que la interfaz puede actualizarse sin volver a cargar el programa del microcontroladors y viceversa.

Una vez finalizado el desarrollo del *frontend*, se verificó la elección de la biblioteca. Para ello, la misma aplicación se compiló con *React* y se comparó el tamaño del archivo *JavaScript* resultante con el obtenido mediante *Preact* (Tabla 3.2). Con *Preact*, el archivo ocupó un 60 % menos, mientras que la hoja de estilos no varió.

Biblioteca	Tamaño del archivo *JavaScript*
React	286 KB
Preact	113 KB

Tabla 3.2: Tamaño del código compilado con *React* y con *Preact*

Estructura interna

El *frontend* se estructuró en pantallas, donde cada una cumple un conjunto acotado de funciones. Cada pantalla es un componente con su propio archivo de código y su hoja de estilos. Las pantallas destinadas a profesores y tutores, en cambio, se agruparon en un conjunto llamado *panel*. Comparten una misma hoja de estilos, de modo que mantienen una apariencia uniforme y una misma ruta base en el navegador. Además, las hojas de estilos ajustan la disposición de los elementos al tamaño de la pantalla, de modo que la interfaz puede utilizarse tanto en una computadora como en un celular (En el caso del alumno, en posición horizontal).

Imagen de un diagrama con la estructura interna

Los componentes se organizan de forma jerárquica: en la raíz se encuentra el componente principal que contiene a todas las pantallas. A su vez, algunas de ellas incluyen sus propios componentes. Por ejemplo, la pantalla de cuestionarios contiene el listado, el editor y la vista

de revisión. Cuando el profesor elimina un cuestionario desde el listado, este avisa a la pantalla que lo contiene y es ella la que realiza la petición al servidor y actualiza la lista.

El componente principal actúa además como gestor de rutas. Según la dirección del navegador decide qué pantalla mostrar y, si no corresponde a ninguna, redirige al inicio. Utilizando la ruta común, aplica a todas las pantallas del panel un mismo control de inactividad. Si el usuario no interactúa durante un tiempo o si el servidor informa que la sesión expiró, lo devuelve a la pantalla de inicio de sesión.

El profesor y el tutor comparten las mismas pantallas del panel. Cada una consulta el rol del usuario (que el navegador guarda al iniciar sesión), y muestra u oculta sus elementos según corresponda. Por ejemplo, el campo que el profesor ve como materia, el tutor lo ve como vínculo. Así se evitó duplicar pantallas casi idénticas, lo que reduce el código y mantiene ambas versiones coherentes. Este dato solo determina qué se muestra. Los permisos los verifica el *backend* en cada petición, por lo que ocultar un botón no constituye una medida de seguridad.

Las comunicaciones con el *backend* se aislaron en un módulo independiente, que centraliza todas las peticiones dirigidas al módulo central. De esta forma, las pantallas solicitan datos, como el estado de un examen o la lista de profesores, mediante funciones simples. El módulo, por su parte, arma los encabezados, adjunta el *token* de sesión e interpreta las respuestas en formato JSON. Si el servidor responde con un error, lo convierte en un mensaje que la pantalla muestra al usuario.

Recepción de datos y ejecución del detector

En la sección 3.5 se describió cómo el módulo central y el módulo de captura envían la información. A continuación se explica qué hace el navegador con ella. Primero se detalla cómo recibe la aplicación y la biblioteca *OpenCV.js*, y luego cómo recibe el video y ejecuta el detector a partir de esos elementos.

Alguna imagen que ilustre todo esto

Carga de la aplicación

Cuando un usuario ingresa a *levi.local*, el módulo central envía al navegador los tres archivos del *frontend*. El documento HTML llega prácticamente vacío y solo indica qué otros dos archivos descargar. Una vez recibidos, el navegador ejecuta el código *JavaScript*, que arma la interfaz y muestra la pantalla solicitada. A partir de ese momento, todo el funcionamiento de la interfaz ocurre en el dispositivo del usuario.

Obtención de OpenCV.js

La biblioteca *OpenCV.js* no se envía junto con la aplicación, sino solo cuando se abre la pantalla del alumno, que es la única que la necesita. Como es el archivo más pesado del sistema, el navegador guarda una copia después de la primera descarga, de modo que en los accesos siguientes no vuelve a pedirla al módulo central. Cuando debe descargarla, verifica que el archivo haya llegado completo y, si la descarga falla, la vuelve a intentar de forma automática.

La biblioteca incluye una parte de su código que el navegador debe compilar antes de ejecutarla, lo que demora algunos segundos. Durante ese tiempo, la pantalla del alumno muestra un aviso y mantiene deshabilitados los botones.

Recepción del video

Durante la calibración y el examen, el navegador del alumno se conecta directamente al módulo de captura y recibe el video. Cada fotograma JPEG que llega reemplaza al anterior, y el propio

navegador se encarga de decodificarlo sin código adicional. El video completo no se muestra al alumno. En su lugar, la pantalla presenta el recorte del ojo que se analiza. Sobre esa imagen se dibuja un recuadro que marca la región de interés (ROI), que indica toda el área en la que el detector se aplica y ayuda a centrar el ojo.

La conexión se abre al ingresar a la calibración o al examen y se cierra al salir de ellos o ante un error. Cada vez que se abre se fuerza una conexión nueva, para que el navegador no muestre una imagen antigua guardada de un acceso anterior.

Ejecución del seguimiento

Una vez compilada, la biblioteca pone sus funciones a disposición del código *JavaScript* de la aplicación, que las invoca como si fueran propias. Los cálculos pesados, como la conversión a escala de grises o la comparación de plantillas, se ejecutan en el código compilado de la biblioteca. *JavaScript*, en cambio, solo coordina el proceso y actualiza la interfaz con los resultados.

Para trabajar con las imágenes, *OpenCV.js* las representa como matrices ubicadas en una zona de memoria propia, separada de la que administra el navegador. Por este motivo, la aplicación reserva una única vez las matrices del fotograma, del recorte y de su versión en escala de grises y las reutiliza en cada ciclo. Las matrices temporales, en cambio, se liberan de forma explícita después de cada uso, ya que el navegador no las elimina por sí solo. Sin esta precaución, la memoria ocupada crecería con cada fotograma.

Las plantillas obtenidas en la calibración también se conservan como matrices de *OpenCV.js*. Para guardarlas en el navegador se convierten en imágenes y, al volver a la pantalla del alumno, se transforman nuevamente en matrices.

Pantallas

Las pantallas se agrupan en tres categorías, según quién las utiliza: las de acceso, la del alumno y las del panel.

Pantallas de acceso

Son las pantallas a las que cualquier usuario puede ingresar sin importar su rol.

- Inicio (o *Home*): es la primera pantalla que se muestra al acceder a levi.local. Presenta un mensaje de bienvenida y dos botones: uno para ingresar como alumno y otro para hacerlo como usuario externo. El primero dirige a la pantalla del alumno y el segundo, a la de inicio de sesión.
- Inicio de sesión: los profesores y los tutores ingresan su usuario y su contraseña. Desde allí también pueden registrarse, completando un formulario con sus datos, o recuperar una contraseña olvidada. En ambos casos se solicita la clave maestra del rol.
- Invitado: se accede desde la pantalla de inicio de sesión. Presenta un campo para la pregunta y entre dos y cuatro campos para las opciones, que pueden agregarse o quitarse. Al enviarla, la pregunta aparece en la pantalla del alumno.

(Imágenes de las pantallas de inicio, inicio de sesión, registro, recuperación de contraseña e invitado)

Pantalla del alumno

El alumno ingresa sin autenticarse. Su pantalla reúne cuatro vistas que se alternan según la etapa en la que se encuentra.

- Menú: presenta un indicador del estado de carga del detector y dos botones, uno para la calibración y otro para el examen.
- Calibración: muestra seis recuadros, uno por cada plantilla, y la imagen del ojo que se analiza. Durante la secuencia, un mensaje indica hacia dónde debe mirar el alumno y una cuenta regresiva anuncia cada captura. Al finalizar, las plantillas aparecen en sus recuadros y el alumno puede guardarlas o repetir el proceso. Si ya existe una calibración realizada en la misma sesión, el botón de inicio ofrece recalibrar. Esta calibración antigua, almacenada en caché, se mantiene viva mientras el dispositivo esté conectado a la red.
- Examen: muestra en una esquina la imagen del ojo, junto con un control deslizante que ajusta el umbral de similitud. Mientras no haya un cuestionario en curso ni una pregunta de invitado, la pantalla queda en espera. Cuando llega una pregunta presenta el enunciado, su número dentro del cuestionario y cuatro cuadrantes, uno por opción, con una zona neutra en el centro. Cada cuadrante posee un color distintivo. La pantalla se divide siempre en cuatro, aunque la pregunta tenga menos opciones. En ese caso, los cuadrantes sobrantes quedan vacíos y se ignoran, de modo que las posiciones coinciden con las de la calibración.
- Resultado: en el caso de una pregunta de invitado, presenta la pregunta y la opción elegida. En el caso de un examen informa el puntaje obtenido, el máximo posible, el necesario para aprobar, el tiempo empleado y si el alumno aprobó. Después de unos segundos la pantalla vuelve sola a la espera, sin necesidad de pulsar ningún botón.

(Imágenes del menú, la calibración, el examen y el resultado)

Pantallas del panel

Son las pantallas que utilizan los profesores y los tutores. En todas ellas, las acciones que no pueden deshacerse, como eliminar un registro o finalizar un cuestionario, solicitan una confirmación antes de ejecutarse.

- Principal: muestra un saludo con el nombre del usuario, un botón para cerrar la sesión y tres botones de navegación. El primero dirige a los cuestionarios y el último, al perfil. El botón central depende del rol, ya que el profesor accede a los datos de los tutores y el tutor, a los de los profesores.
- Perfil: permite modificar los datos personales del usuario y, en una sección aparte, cambiar la contraseña, para lo cual se solicitan la actual y la nueva. No se requiere clave maestra, ya que la sesión activa confirma la identidad del usuario. También permite eliminar la propia cuenta y advierte al usuario si intenta salir con cambios sin guardar.
- Contactos: muestra los datos de contacto de los usuarios del otro rol. Además, el tutor puede eliminar a un profesor.
- Cuestionarios: para el profesor, presenta el listado de sus cuestionarios con su título, su cantidad de preguntas, el puntaje obtenido, la condición de aprobación y la fecha de inicio. Cada cuestionario cuenta con botones para editarlo, eliminarlo, cambiar su estado y finalizarlo, que se habilitan según su estado actual. El botón de estado cambia su función en cada caso, de modo que inicia, pausa o reanuda el cuestionario y, una vez finalizado, permite revisarlo. Al iniciar un cuestionario, la fecha se toma del dispositivo del profesor, ya que el módulo central no dispone de un reloj de tiempo real ni de acceso a internet para conocerla. El tutor, en cambio, ve los cuestionarios de todos los profesores, con la materia de cada uno, y solo puede revisarlos una vez finalizados.
- Editor: exclusivo para el profesor. Se accede desde el listado, tanto para crear un cuestionario como para modificar uno pendiente. Cada pregunta se carga en un bloque propio. Para

agilizar la carga, un mismo esquema de puntajes puede aplicarse a todas las preguntas, y estas pueden reordenarse o eliminarse.

- Revisión: muestra el resultado de un cuestionario finalizado. Detalla para cada pregunta la opción elegida, la correcta en caso de error y los puntos que sumó o restó.

(Imágenes de las pantallas principal, perfil, contactos, cuestionarios, editor y revisión, para ambos roles cuando corresponda)

3.7. Interfaz de usuario táctil

Con el fin de facilitar el uso del dispositivo, se incorporó una interfaz gráfica en la pantalla táctil del módulo central. Desde ella se realiza la configuración inicial del equipo y se supervisa el estado del sistema. La interfaz se desarrolló con la biblioteca *LVGL*, presentada en el capítulo 2. A continuación se describen las bibliotecas y los controladores que utiliza, su funcionamiento dentro del módulo central y las vistas que la componen.

3.7.1. Bibliotecas y controladores

La interfaz se apoya en dos bibliotecas. *TFT_eSPI* provee los controladores de bajo nivel que comunican el microcontrolador con la pantalla mediante SPI. Por sí sola, sin embargo, solo permite dibujar elementos básicos como píxeles, líneas y texto. *LVGL*, en cambio, aporta los elementos que requiere la interfaz, como las vistas deslizables, los campos de texto, el teclado virtual y los botones, y gestiona los toques del usuario. Sin ella, todos estos elementos habrían tenido que programarse desde cero.

Además, *LVGL* puede dibujar la pantalla por partes. En lugar de reservar memoria para la imagen completa, que ocuparía unos 150 KB, utiliza un búfer de alrededor de 4 KB, equivalente a una franja angosta de la pantalla. La biblioteca dibuja cada franja en ese búfer y la envía a la pantalla antes de pasar a la siguiente. Así, la interfaz ocupa una fracción mínima de la RAM del módulo central.

El panel táctil, por su parte, también se comunica mediante SPI. Sin embargo, el *ESP32* dispone de dos controladores SPI de uso general, y ambos ya están ocupados por la pantalla y por la tarjeta microSD. Además, en la placa el panel táctil está conectado a otros pines. Por este motivo, su lectura se implementó por software, generando desde el código la señal de reloj y leyendo los datos bit a bit.

3.7.2. Funcionamiento

La interfaz forma parte del mismo *firmware* que el resto del módulo central y se actualiza en cada repetición de su ciclo principal. Los datos que muestra no viajan por la red, sino que se leen directamente de la memoria del microcontrolador. Así, la interfaz consulta el gestor de sesiones para conocer el usuario conectado, y el monitor de red para obtener el estado de la cámara y la cantidad de dispositivos.

La información del examen requiere un tratamiento particular. Consultarla en la base de datos en cada ciclo implicaría acceder a la tarjeta microSD de forma continua. Por este motivo, cada vez que el alumno consulta el estado del examen, el servicio de respuestas guarda en la RAM un resumen con los datos que muestra la pantalla, y la interfaz solo lee esa copia.

Para no ocupar al microcontrolador de forma innecesaria, cada dato se vuelve a dibujar solo cuando cambia. Por ejemplo, el nombre del usuario conectado o el estado de la cámara se actualizan únicamente cuando difieren del valor que ya se muestra.

Al encender el equipo, la pantalla se inicia antes que la tarjeta microSD, la base de datos, la red y el servidor web. De esta forma, el usuario observa la interfaz desde los primeros instantes, mientras el resto del sistema termina de arrancar.

Por otra parte, para reducir el consumo de la batería, la retroiluminación de la pantalla se apaga tras un minuto sin que se la toque. El primer toque posterior solo vuelve a encenderla sin activar ningún elemento de la interfaz, de modo que el usuario no ejecute una acción por accidente.

3.7.3. Vistas

La interfaz se organiza en cuatro vistas. Tres de ellas forman parte de un contenedor deslizante, en el que el usuario pasa de una a otra con un gesto horizontal. La cuarta, en cambio, solo aparece durante la configuración inicial del equipo.

Bienvenida

Es la primera vista que se muestra al encender el equipo ya configurado. Presenta un mensaje de bienvenida y un código QR que contiene la dirección levi.local. Al escanearlo con un teléfono, el navegador abre directamente la aplicación, sin necesidad de escribir la dirección. Como el contenido del código no cambia, se lo guardó como una imagen ya generada en blanco y negro y ubicada en la memoria flash. Generarlo con una biblioteca al iniciar el equipo habría ocupado RAM, tanto para el cálculo como para la imagen resultante.

(Imagen de la vista de bienvenida)

Estado del sistema

Al deslizar la pantalla se accede a un panel de supervisión. Este muestra el nombre del alumno, el del usuario conectado al panel, el estado de la cámara y la cantidad de dispositivos conectados a la red. Los dos últimos datos se obtienen según lo descrito en la sección de red y comunicación.

(Imágenes de la vista de estado del sistema, en distintas situaciones)

Examen

La tercera vista permite seguir el examen desde el propio módulo central. Mientras no haya un cuestionario en curso muestra un mensaje de espera. Durante el examen presenta el título del cuestionario y el progreso del alumno, expresado como la pregunta actual sobre el total. Al finalizar muestra el resultado, que incluye el puntaje necesario para aprobar, el obtenido sobre el máximo posible, el tiempo empleado y si el alumno aprobó. Después de unos segundos, la vista vuelve sola a la espera.

(Imágenes de la vista de examen en espera, en curso y finalizado)

Configuración inicial

Esta vista aparece solo cuando la NVS no contiene los datos de configuración, como ocurre en el primer encendido del equipo. En ella se ingresan el nombre del alumno y las claves maestras de profesores y tutores. Para escribirlos, la interfaz despliega un teclado virtual. Si el teclado tapa el campo seleccionado, el formulario se desplaza hacia arriba para mantenerlo visible. Las claves se muestran ocultas, aunque un botón permite verlas, y no es posible guardar con campos vacíos. Al guardar, los datos se almacenan en la NVS y el equipo se reinicia para cargar las vistas habituales.

(Imagen de la vista de configuración inicial)

3.8. Alimentación

En esta sección se explicará acerca del sistema de alimentación autónomo utilizado en este trabajo. Se comenzará por nombrar los requisitos, expuestos en el capítulo 1, luego se expondrán algunas soluciones propuestas. Finalmente, se explorará en detalle el sistema elegido junto a su diseño en físico.

3.8.1. Requisitos previos

Entre las premisas de este trabajo se estipuló que el dispositivo final debe ser inalámbrico y autosuficiente en términos de consumo energético al momento de utilizarlo. Siguiendo estos lineamientos, se opta por no alimentarlo utilizando cables y entradas USB-C o MicroUSB que se incluyen en los modelos comerciales de los microcontroladores utilizados, sino que se diseñará su propio sistema de alimentación utilizando fuentes de energía independientes de cualquier dispositivo externo al momento de uso. La solución general planteada para este problema recae en el uso de baterías de litio, una de las fuentes de energía inalámbrica más comunes en los dispositivos electrónicos de bajo consumo. A pesar de su precio relativamente elevado a comparación de las pilas comunes, tienen la particularidad de que son recargables, evitando así que el usuario tenga que comprar repuestos en el momento en que se agote la energía.

La alimentación a través de estas baterías permitirá un uso prolongado del dispositivo antes de necesitar una recarga y al mismo tiempo evita la necesidad de comprar pilas comunes para cada período de uso. La energía suministrada será moldeada por otros componentes electrónicos para obtener la salida continua y estable que requieren estos microcontroladores para funcionar correctamente. Cada placa (emisora de la gorra y CYD receptora) tendrá su propio circuito de alimentación independiente de la otra, para así evitar enlazar ambas partes del proyecto mediante un cable y poder mover a la CYD con la interfaz libremente. En esencia ambos circuitos serán exactamente iguales, pues se trata con dos microcontroladores que, para su uso en este contexto, requieren de la misma señal de alimentación.

Teniendo seleccionada la fuente de energía, se presentan otros dos problemas. Se requiere una forma para recargar las baterías de manera sencilla y además un método para estabilizar la señal de alimentación. Esto último se debe a que los *ESP32* requieren una alimentación de 3,3 V o, en su defecto, de 5 V lo más estables posible, ya que una mala alimentación puede provocar mal funcionamiento, reinicios inesperados o destrucción del hardware.

A continuación se analizarán cuatro alternativas para solucionar estos problemas (suponiendo que se utilizarán baterías de litio 18650 de 3000 mAh)

3.8.2. Soluciones propuestas

1- Regulador lineal de tensión + Cargador de pilas externo

Consiste en alimentar el microcontrolador directamente desde la batería de litio a través de un portapilas (15,5 USD). A esto se le suma un regulador de tensión lineal de baja caída (LDO), como el *AMS1117* (1,8 USD), para ajustar la tensión a 3,3 V estables. Para la recarga se emplearía un cargador comercial externo (5 USD), lo que implica extraer físicamente la batería del dispositivo para colocarla en la ranura y conectarla a la red.

(Imagen del LDO)

Ventajas de esta solución:

- Mantenimiento práctico: los cargadores comerciales son componentes estándar y fáciles de conseguir en el mercado.
- Optimización de espacio: al requerir únicamente un portapilas, el regulador lineal y un par de capacitores de filtrado, se reduce considerablemente el espacio ocupado en una PCB.

Desventajas de esta solución:

- Ergonomía y desgaste mecánico: resulta impráctico para el usuario el tener que retirar la batería del dispositivo cada vez que se quiera recargar, además de que esto puede provocar desgaste mecánico con el tiempo.
- Descarga de la batería: una batería de litio estándar (como la *18650*) de 3,7 V no entrega este valor continuo de tensión a lo largo del tiempo, sino que disminuye progresivamente hasta llegar a un piso mínimo de unos 3,0 V. Este valor es menor a los 3,3 V que necesita el microcontrolador, lo que significa que el dispositivo dejaría de funcionar mucho antes de que la batería se quede sin energía para dar. Se requiere hardware adicional para aumentar el valor de tensión cuando sea necesario y agregarlo invalida la ventaja de la optimización de espacio.
- Dropout: la función del regulador es estabilizar la tensión en 3,3 V. Sin embargo, estos integrados operan con transistores MOSFET en zona activa (es decir, actuando como resistencias variables), lo que implica inevitablemente que habrá una caída de voltaje entre la entrada y la salida. Esta caída se llama *voltaje de dropout*, lo que significa que para tener 3,3 V estables a la salida, necesitaremos una tensión continua mayor que ese valor en la entrada, lo que reduce aún más el tiempo en el que se puede usar el dispositivo antes de que la batería se quede sin energía.
- Picos de corriente: el consumo de corriente de un microcontrolador es dinámico. Hay ocasiones, como al encender el Wi-Fi, en las que se exigen grandes transitorios o picos de corriente de hasta 500 mA. Por ley de ohm, al exigir más corriente al regulador, la caída de tensión de dropout por la resistencia del silicio se maximizará. Esto agrava el problema anteriormente mencionado, provocando caídas bruscas en la tensión de alimentación que disparan el detector de bajo voltaje (*brownout detector*) del microcontrolador.

2- Placa de alimentación (Módulo TP4056 + Módulo MT3608)

El módulo *TP4056* (1,5 USD), es un circuito integrado utilizado para recargar baterías como las implementadas en este proyecto, mientras que el *MT3608* (1,8 USD), es un módulo elevador de voltaje (boost) que entrega a la salida una señal muy estable del valor elegido por el diseñador. Si se interconectan ambos módulos se formaría un conjunto que cumpliría los requisitos de recarga y regulación, y se podría alimentar al *ESP32* desde el pin de 5 voltios.

(Imagen de los modulos en cuestión)

Ventajas:

- Facilidad de mantenimiento: al ser dos circuitos separados, las fallas puntuales pueden repararse fácilmente utilizando un repuesto del módulo necesario sin alterar el sistema completo.
- Carga térmica distribuida: esta separación de tareas implica que la temperatura que alcance cada uno será independiente del otro y se distribuirá uniformemente a lo largo de la PCB.

- Sin dropout ni picos de corriente: el módulo *MT3608* contiene un regulador switching, que utiliza transistores MOSFET en zona de corte y saturación, por lo que no hay caídas apreciables de tensión entre la entrada y la salida. A su vez, los picos de corriente pueden ser abastecidos por el ajuste automático del ciclo de trabajo del switch.
- Doble regulación de tensión: Al alimentar el *ESP32* a través del pin de 5 V, éste utiliza el regulador integrado en su placa para reducirlos a 3,3 V aún más estables.

Desventajas:

- Mantenimiento impráctico: A pesar de que las reparaciones son sencillas y baratas, los módulos mencionados sólo pueden conseguirse en casas especializadas en electrónica.
- Menor optimización del espacio: Al tener dos módulos distintos se requiere una PCB más espaciosa para contenerlos.
- Pérdida de eficiencia: La ventaja de la doble regulación implica inevitablemente una pérdida de eficiencia. La energía sufre dos conversiones (de la batería a los 5 V y de los 5 V a los 3,3 V en el regulador interno del *ESP32*), lo que aumenta la disipación térmica.

3- Regulador Buck-Boost

Se podría pensar que para evitar esa pérdida de eficiencia con el *MT3608* y alimentar al microcontrolador con 3,3 V se pudiera utilizar un regulador Buck-Boost, como el *XL6009* (3,0 USD), en lugar del elevador Boost para alimentarlo a 5 V. Pero la ventaja de la eficiencia contrasta con el ligero aumento del costo y la complejidad del circuito. Además, revisando la hoja de datos, se observa que este regulador funciona con una tensión nominal de entrada de 5 V, mayor a la que la batería puede suministrar.

La frecuencia de llaveo es otra desventaja. A diferencia del módulo MT3608 que conmuta a 1.2 MHz, éste lo hace a 400 kHz, lo cual puede generar tensión de ripple o rizado a la salida mucho mayor. Esto es sumamente indeseable para alimentar un servidor Wi-Fi y una cámara que debe transmitir en vivo, y suprimirlo requeriría colocar capacitores de mayor valor.

(Imagen del BuckBoost)

4- Integrado Power Bank todo en uno El integrado *IP5306* (4,5 USD) cumple la misma función que la placa de alimentación utilizando un solo chip integrado que realiza internamente el proceso de carga y de regulación de tensión.

(Imagen del IP5306)

Ventajas:

- Optimización del espacio en PCB al ser un solo módulo.
- Regulación conmutada: Al igual que los elevadores y reductores, funcionan con transistores en zona de corte y saturación, evitando el problema del dropout y los picos de corriente.
- Carga y uso simultáneo.

Desventajas:

- Mantenimiento impráctico, por la dificultad agregada de conseguir los repuestos.
- Pérdida de eficiencia por usar doble regulación.
- Carga térmica: al ser un solo chip el que hace la tarea de carga de batería y regulación de voltaje, todo el calor disipado se acumulará en un único punto del dispositivo, lo que influirá negativamente en el funcionamiento del mismo.

- No es modular: si se produjera una falla en cualquier parte del sistema, inevitablemente habría que comprar el repuesto completo del chip, lo cual es más costoso y complicado que comprar los repuestos separados de los módulos anteriormente vistos.
- Ripple en la señal de salida: al estar diseñados para cargar celulares conectándose a un PMIC interno, los chips *IP5306* conmutan a mucha menor frecuencia (entre 300 y 500 kHz) que el módulo *MT3608*, por ejemplo. Al igual que con el XL6009, se requeriría mucha capacitancia para suprimirlo lo suficiente.

Para solucionar el problema del ripple, se podría utilizar directamente un PMIC industrial, pero su elevado costo y la complejidad de soldadura de sus encapsulados lo vuelven inviable para este prototipo.

Análisis de mercado

A continuación se muestra un resumen comparativo de los costos de cada alternativa, suponiendo que las baterías utilizadas son baterías de litio de 3000 mAh y sin tener en cuenta costos menores (como borneras, botones, LEDs, etc.).

Tabla 3.3: Análisis de costos de todos los métodos de alimentación propuestos (octubre 2026)

Método propuesto	**Costo**
Batería de Litio + LDO + Cargador externo	22,3 USD
Regulador switching + Elevador	18,8 USD
Regulador Buck-Boost	20 USD
Power Bank	20 USD

Como podemos observar, todas las alterativas evaluadas rondan los 20 USD, siendo la batería de litio el componente más costoso (un 75 % del precio final). Es por esto que el criterio económico no fue considerado como prioritario a la hora de elegir una solución definitiva.

Veredicto final

La primera alternativa se descartó por la baja eficiencia de los reguladores lineales, que podrían sencillamente ser reemplazados por reguladores switching sin mucha pérdida económica ni dificultad agregada.

Tanto la tercera como la cuarta alternativa se descartaron debido a la tensión de rizado que generan a la salida, la cual es mucho mayor que el resto y requiere mas costos adicionales para filtrarla.

Por lo tanto, en una búsqueda del equilibro entre eficiencia, sencillez y estabilidad de tensión de salida es que se opta por la segunda alternativa: el regulador TP4056 y el elevador MT3608

3.8.3. Descripción detallada del sistema elegido

El primer circuito integrado que utilizaremos es el chip *TP4056*, cuyo datasheet lo describe como un cargador lineal de corriente constante/tensión constante completo para baterías Li-ion de una sola celda. Este chip toma energía de una fuente externa y la adapta para recargar la batería. La energía que proveen estas baterías de larga duración suele ser de 3,7 voltios. Por lo tanto, para adecuar y estabilizar este nivel de tensión se utiliza otro circuito integrado fundamental: el *MT3608*; el cual, en términos sencillos, se trata de un circuito elevador boost

que en la salida entrega una señal continua y estable según el ajuste que realice el diseñador. Esa salida irá directamente conectada a la entrada de 5 voltios de la placa *ESP32*.

El funcionamiento en detalle de estos integrados y sus módulos comerciales serán descritos en las siguientes secciones.

Recarga y protección

Como se expuso con anterioridad, el chip *TP4056* sirve para cargar una única celda de baterías de litio y lo hace en un ciclo de dos fases.

En primera instancia carga la batería a corriente constante, es decir, le inyecta un valor fijo de corriente elegida por el diseñador para que la carga comience siendo rápida. Al mismo tiempo, el chip mide constantemente el voltaje de la batería, y cuando llega a los 4,2 V comienza la fase de carga con voltaje constante, y permite a la propia batería que absorba la corriente necesaria para no destruirse. Si toda la carga fuera a corriente constante, se le inyectaría más energía de la que puede soportar y acabaría destruyéndose. Por el contrario, si toda la carga fuese a voltaje constante, el proceso llevaría demasiado tiempo, lo cual es poco práctico para la experiencia de usuario. Es por eso que este integrado oscila automáticamente entre esos dos modos.

Por otra parte, el *TP4056* está diseñado para detener la carga en la fase de voltaje constante, cuando la corriente absorbida por la batería sea inferior a una décima del valor programado por el usuario, lo cual evita sobrecargarla.

Para resumir, sus características más relevantes son:

- Corriente de carga programable hasta 1000 mA.
- Operación dinámica entre corriente constante y voltaje constante.
- Carga hasta un valor fijo de 4,2 V.
- Fin de carga automático.
- UVLO.
- Umbral de carga lenta.
- Arranque suave de corriente.

UVLO (*Under Voltage Lockout* / Bloqueo por bajo voltaje): Es una funcionalidad del chip que permite que cuando se introduzca una alimentación mala o inestable, la carga se bloquee para evitar que el proceso dañe la pila o los circuitos.

Umbral de carga lenta: Para evitar sobrecargar una batería en mal estado (con voltaje inferior a 3,0), el *TP4056* entra en modo *carga lenta*, enviando solo el 10 % de la corriente programada hasta que supere el umbral. Luego de eso la carga procede con normalidad.

Arranque suave de corriente: Es una medida de protección a la fuente externa. Garantiza que al enchufar el circuito a la misma, la absorción de corriente no sea tan brusca, sino que aumente gradualmente con el tiempo para evitar daños.

Su diagrama de pines es el siguiente:

1. TEMP: Entrada de sensor de temperatura. Sirve para monitorear la temperatura de la pila al momento de la carga mediante un termistor externo. Debido al uso que se le va a dar en este proyecto y a la presencia de otros circuitos de protección, se opta por prescindir de su uso.



Figura 3.16: Diagrama de pines del módulo *TP4056*

2. PROG: Entrada de programación de corriente de carga. A este pin se le debe conectar una resistencia denotada como *RPROG* para establecer el valor de la corriente de carga. El valor esta se calcula con la siguiente fórmula:

$$Rprog = 1200 * Ibat \tag{3.1}$$

   Donde *Ibat* es la corriente en miliamperios elegida por el diseñador, con un valor máximo de 1000 mA.

3. GND: Tensión de referencia.
4. VCC: Entrada de alimentación externa.
5. BAT: Salida que se conecta al terminal positivo de la pila. Le brinda la corriente necesaria y regula el voltaje a 4,2 V.
6. STDBY: Salida en alta impedancia que se vuelve nula cuando la carga está completa.
7. CHRG: Salida en alta impedancia que se vuelve nula mientras la pila se está cargando.
8. CE: Habilitación del chip. Conectándolo a un 0 lógico se bloquean todas las funcionalidades del chip (Modo *shutdown*).

En el siguiente gráfico se ilustran las fases de corriente constante y voltaje constante descritas anteriormente:

Con un voltaje menor a 4,2 V, se programó en ese caso una corriente de 800 mA. El ciclo comienza con un aumento gradual de la corriente hasta llegar a ese valor programado. El voltaje mientras tanto va aumentando, y al llegar a los 4,2 V la corriente disminuye hasta finalizar la carga.

Se explica a continuación un circuito de uso típico con este chip, correspondiente al módulo que se encuentra comercialmente en las casas de electrónica y se utiliza en este proyecto:

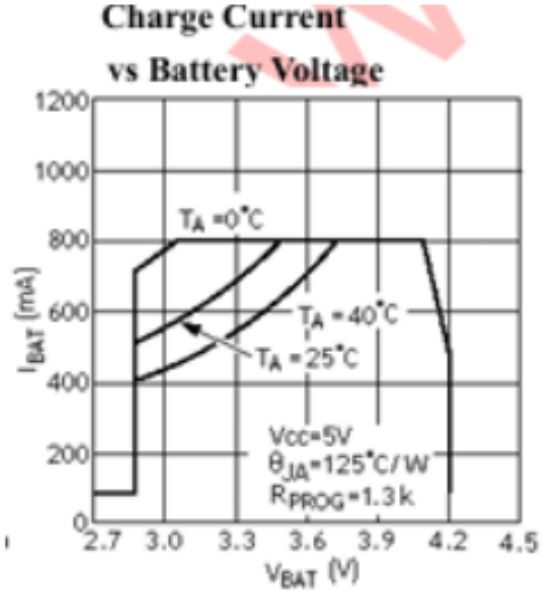


Figura 3.17: Corriente vs voltaje de carga

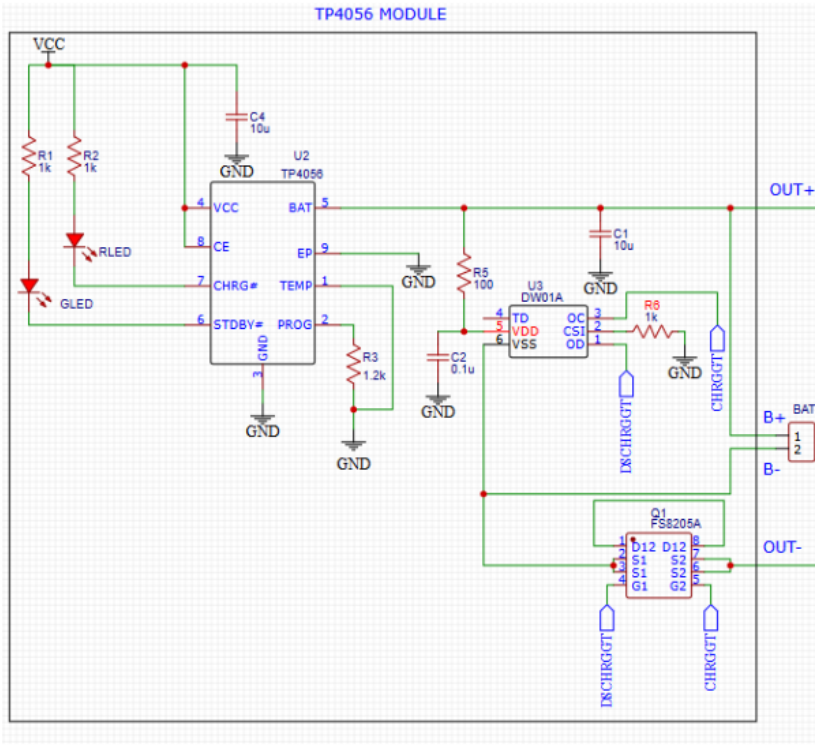


Figura 3.18: Circuito típico de aplicación del *TP4056*

Lo primero que se puede notar es un puente entre la entrada CE y VCC, esto es debido a que para este uso práctico no se requiere deshabilitar el chip manualmente. Se observa también un capacitor C4 utilizado para drenar el ruido de la entrada

En las salidas CHRG y STDBY están conectados los cátodos de dos diodos LED. Uno verde para STDBY y uno rojo para el CHRG. Estos son LEDs indicadores que informan al usuario en

qué estado se encuentra la carga.

En el pin PROG se conecta una resistencia RPROG (o R3 en el diagrama) de 1.2 kΩ, que es el valor por defecto comercialmente, lo que implica una corriente de carga programada de 1 A, y una corriente de corte de 100 mA.

El pin de sensor de temperatura se conectará a GND ya que no se utiliza, al igual que el pin 9 (EP), que sirve para disipación.

El pin 5 (BAT), no va conectado únicamente a la pila, sino que además pasa por un capacitor de desacople (C1) y por otros dos integrados que también se incluyen en el módulo: El *DW01A* y el *FS8205A*.

El *DW01A* es un monitor del voltaje y corriente de la batería. Si nota que su tensión es inferior o superior a lo esperado, o detecta algún cortocircuito, le informa al *FS8205A* mediante los pines CHRGGT y DSCHRGGT, para que este último actúe desconectando físicamente la pila del circuito mediante el pin B-.

En condiciones normales, la batería se conecta, mediante los terminales B+ y B-, al pin BAT y a GND respectivamente. Si el integrado *FS8205A* actúa, se abre un MOSFET aislando el terminal B- de GND y por lo tanto abriendo el circuito de carga.

Estabilización de voltaje

El chip *MT3608* es un circuito *Convertidor Step-up de alta eficiencia*. O más coloquialmente conocido como *Elevador de tensión*. Su función es complementarse con un circuito externo típicamente conocido como *Elevador boost* para tomar un determinado valor de voltaje a la entrada y obtener a su salida un valor más grande y más estable, acondicionado para alimentar el microcontrolador.

Para explicar sus características, se comienza por exponer la funcionalidad de un circuito boost mostrado a continuación:

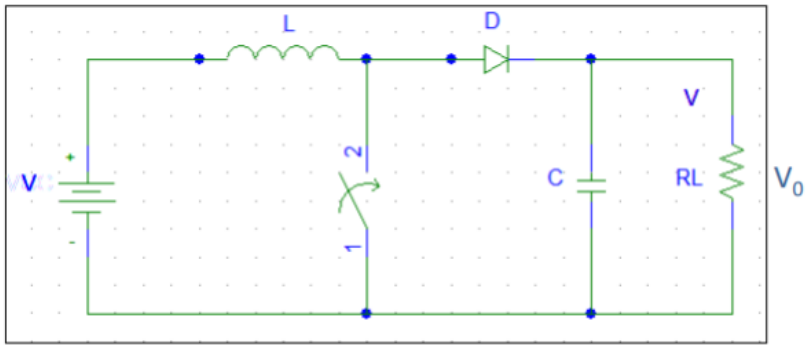


Figura 3.19: Circuito elevador Boost

Este circuito es el típico que se utiliza para las fuentes de tensión tipo *switching*. En este caso particular, la tensión continua de la entrada es la salida de la batería de litio, que oscila entre 3,0 y 3,7 V. Esta señal pasa por un conjunto de una bobina, un diodo, un capacitor y un switch que se abre y cierra intermitentemente para brindar a la salida el valor deseado. Para ello se aprovechan los transitorios de la bobina en los primeros instantes en que esta llave se abre y cierra.

Cuando la llave se cierra, la bobina comienza a cargarse con la energía de la pila, y la tensión a la salida es igual a la tensión previamente almacenada en el capacitor, el cual no puede descargarse

en el cortocircuito debido a la presencia del diodo D, que se polariza en inversa y causa que toda la corriente vaya desde el capacitor hacia la carga.

Cuando la llave se abre, la bobina se descarga y cambia su polaridad, enviando corriente a través del diodo, cargando el capacitor y otorgando un valor de tensión a la salida igual a la suma de las tensiones de entrada y de la bobina.

La tensión de la bobina cuando la llave se abre, y por consecuencia la tensión del capacitor cuando la llave se cierra, dependen de los tiempos del llaveo, más específicamente del ciclo de trabajo o *duty cycle*:

$$DutyCycle = ton/(ton + toff) \tag{3.2}$$

Este ciclo de trabajo es regulado por el propio *MT3608*, el cual es la llave física de este circuito (un MOSFET), sumado a un circuito interno que monitorea la tensión de salida, la compara con una tensión de setpoint elegida por el diseñador y controla los tiempos de llaveo según requiera el caso. Si la tensión de salida cae, el ciclo de trabajo será mayor, mientras que si tenemos un exceso de tensión, este ciclo de trabajo será menor.

Las características más importantes de este integrado son:

- Switch: MOSFET de potencia de 80 mΩ
- Tensión de entrada permitida: Entre 2 y 24 V
- Tensión de salida ajustable de hasta 28 V
- Frecuencia de llaveo: 1.2 MHz
- Límite de corriente a través de la llave de 4 A
- Funcionamiento con modulación de ancho de pulso (PWM)

Su circuito interno es el siguiente:

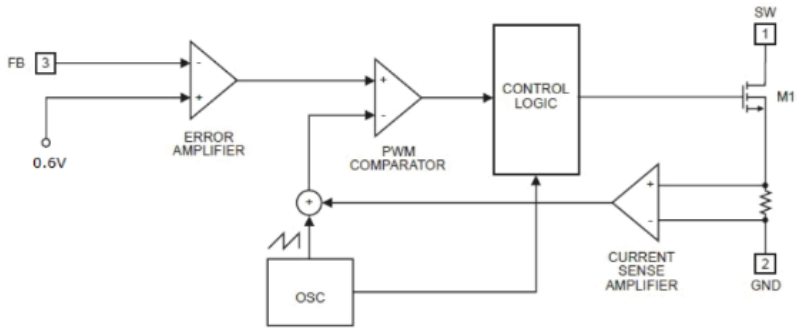


Figura 3.20: Circuito interno del chip *MT3608*

El transistor M1 es el switch que se observa en el diagrama del circuito boost, que se manipula mediante un bloque lógico de control conectado a su entrada de compuerta.

El control comienza con el amplificador de error, que compara la entrada de feedback del chip (una muestra de la tensión de salida final), con una señal de 0,6 voltios muy estable. A la salida de este amplificador tenemos una señal proporcional al error de la tensión en la carga. Se debe procurar que cuando la tensión en la carga se encuentre en su valor óptimo, la entrada

de feedback sea igual a esos 0,6 voltios, para que en esa situación la salida sea constante y no afecte a la modulación.

Cuando la salida es mayor a su valor esperado ($V_{FB} > V_{REF}$), el comparador de error reacciona disminuyendo la señal de error, mientras que cuando la salida es menor ($V_{FB} < V_{REF}$), la señal de error aumenta. Finalmente, si ambas entradas son iguales ($V_{FB} = V_{REF}$), el error será una señal constante, como se mencionó anteriormente. Todo lo anterior se refleja en la siguiente imagen:

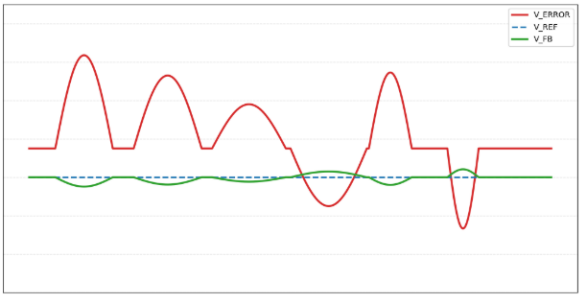


Figura 3.21: Gráfico explicativo del comportamiento del amplificador de error

Este valor de error ingresa en el amplificador comparador PWM junto a una señal diente de sierra generada por un oscilador, para que a su salida se obtenga la señal de control que finalmente manipule los estados del transistor M1. El mismo oscilador diente de sierra es quien activa el pulso de la compuerta, cerrando así la llave y permitiendo que la bobina se cargue. Mientras la tensión de error supere a la señal del oscilador, el pulso se mantiene activo. Cuando la señal diente de sierra supera a la señal de error, el pulso se desactiva abriendo la llave y dejando que la bobina se descargue hasta el próximo ciclo.

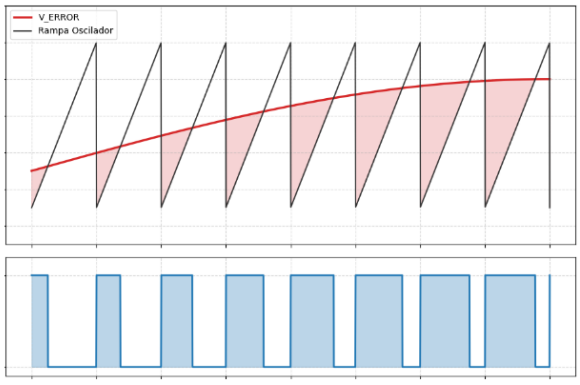


Figura 3.22: Modulación por ancho de pulso (PWM)

Si la bobina pasa más tiempo cargándose (ciclo de trabajo grande), la tensión que genera al momento de descargarse aumentará junto con la tensión de salida, y si pasa más tiempo descargándose (ciclo de trabajo pequeño), la tensión de descarga será menor disminuyendo la salida final.

De todo este análisis se puede inferir la siguiente conclusión: si la tensión de salida decae, el ancho del pulso, y por consiguiente la tensión de salida, aumentan. Por el contrario, si ocurre un exceso de voltaje en la carga, tanto el ancho de pulso como la tensión de salida disminuyen.

Por último, en el diagrama de bloques se observa un último amplificador (el sensor de corriente), que cumple dos funciones importantes: por un lado actúa como sistema de protección, detectando situaciones en las que la corriente a través del switch aumenta a valores peligrosos. Para

ello, lee la tensión sobre una resistencia del orden de los miliohmios (para interferir lo menos posible con el circuito original) y, ante un pico indeseado, envía una señal a la lógica de control que abre la llave para evitar la destrucción del circuito. Por otro lado, durante el funcionamiento normal, esta misma lectura de corriente se suma a la señal del oscilador para alimentar al comparador PWM, esto permite que el chip ajuste el ancho de pulso instantáneamente en cada ciclo, garantizando estabilidad ante variaciones rápidas de carga.

Con todo el funcionamiento explicado, podemos pasar a mostrar el diagrama de pines:

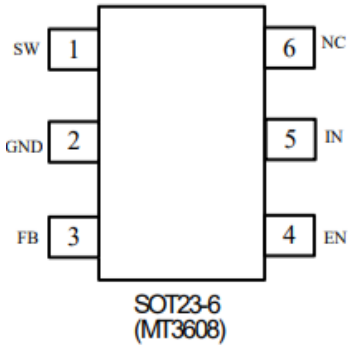


Figura 3.23: Diagrama de pines del módulo *MT3608*

- SW: Es el pin que conecta al MOSFET M1 con el circuito boost externo (El drenador del transistor).
- GND: Tensión de referencia.
- FB: Tensión de feedback. Es una muestra de la tensión de salida que luego se compara con los 0,6 V internos.
- EN: Habilitación del chip.
- IN: Tensión de entrada.
- NC: No se utiliza.

A continuación se muestran algunos gráficos de funcionamiento típico.

Regulación de carga: Indica la corriente máxima que puede entregar a una carga de determinado valor de tensión (en este caso, con 500 mA es más que suficiente para suplir las exigencias del *ESP32*).

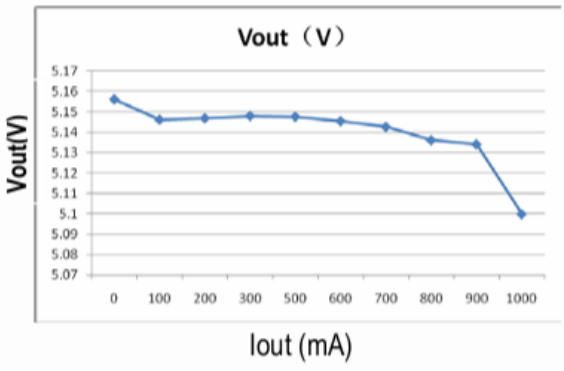


Figura 3.24: Regulación de carga del *MT3608*

Frecuencia de conmutación según tensión de entrada: Vemos que en nuestro caso llegamos a valores de entre 0,9 y 1 Mhz. No es el ideal de 1,2 que se describió anteriormente, pero a fines prácticos sirve para mitigar lo suficiente el ripple de salida.

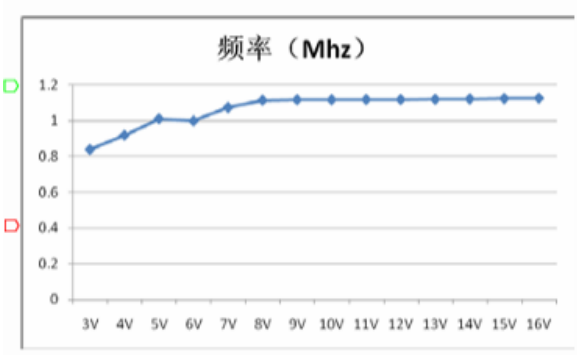


Figura 3.25: Frecuencia vs Tensión de entrada en el chip *MT3608*

Eficiencia vs tensión de entrada: Mientras mayor la tensión de entrada, más eficiente el circuito. En este caso se a roza el 90 %

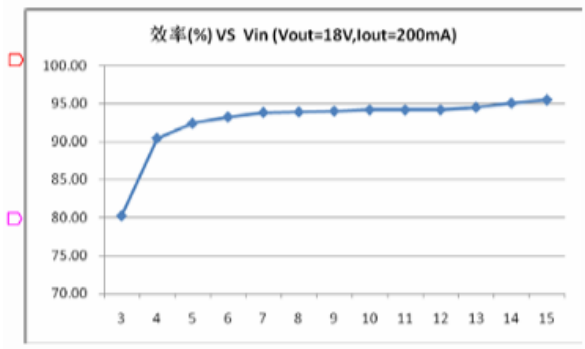


Figura 3.26: Eficiencia vs Tensión de entrada en el chip *MT3608*

A continuación se muestra el circuito que viene en los módulos *MT3608* comerciales, que incluyen el circuito boost externo:

El circuito comienza con los pines de entrada y habilitación cortocircuitados y conectados a la tensión de 3,7 voltios provenientes de la pila, ya que al igual que con el *TP4056*, no se requiere deshabilitar manualmente el integrado. A su vez, el pin de entrada se conecta a uno de los terminales de la bobina de 22 uH, para cargarla en los momentos en que la llave se encuentra cerrada.

El pin de switch (el drenador del MOSFET), se conecta al otro terminal de la bobina y al ánodo de un diodo de alta velocidad, como indica el modelo del circuito boost expuesto anteriormente.

Finalmente, el pin de feedback se conecta a un potenciómetro de 100 kΩ en serie con una resistencia de 2,2 kΩ. La resistencia de este potenciómetro se ajusta con un tornillo externo para seleccionar el valor de tensión a la salida (dado por la señal obtenida en el amplificador de error). De este modo, la salida puede variar entre un mínimo de 0,6 V y un máximo absoluto de 28 V.

Se observa también el capacitor C6 del circuito boost, un capacitor C5 de desacople, y el pin de GND conectado a la masa del circuito.

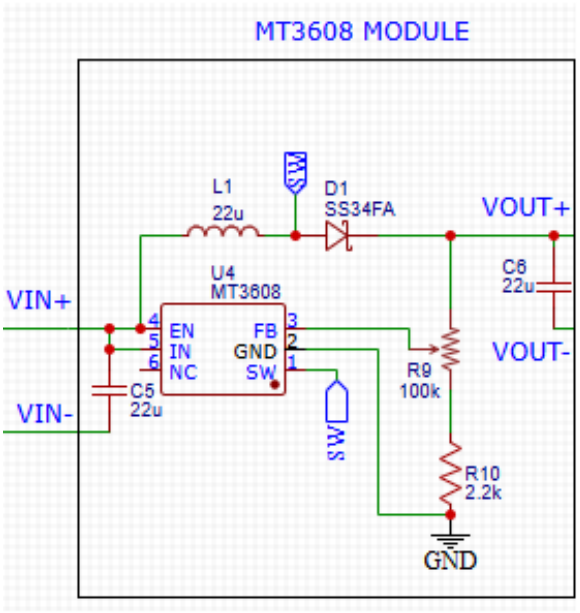


Figura 3.27: Circuito comercial de los módulos *MT3608*

Componentes a utilizar en el dispositivo final

Con todo el sistema explicado, se listan a continuación todos los materiales necesarios para construir los circuitos de alimentación:

- 4 Borneras de 2 terminales
- 4 Capacitores cerámicos de 100 nF (para reducir el ruido).
- 2 Baterías de Litio de 3,7 voltios con 3000 mAh de energía.
- 2 Capacitores electrolíticos de 10 uF (para reducir picos de corriente).
- 2 Módulos de carga *TP4056* comerciales (con chips *DW01A* y *FS8205A* incluidos).
- 2 Módulos elevadores *MT3608* comerciales.
- 2 PCBs de las dimensiones adecuadas.
- 2 Resistencias de 1/4 de vatio de 160 ohmios para los diodos LED.
- 2 Switches para encendido del dispositivo.
- 2 Zócalos o portapilas individuales.
- 1 Bornera de 3 terminales.
- 1 Diodo LED azul.
- 1 Diodo LED infrarrojo emisor.
- Zócalos hembra-macho para portar la *ESP32-CAM* en la PCB

3.9. Diseño físico

3.9.1. Montaje

En esta sección se describe y expone el diseño físico final del dispositivo, comenzando exclusivamente por los circuitos de alimentación y posteriormente con el armado final.

Conexiones y PCB

El esquema de conexiones final para el circuito de alimentación del capturador de imágenes es el siguiente:

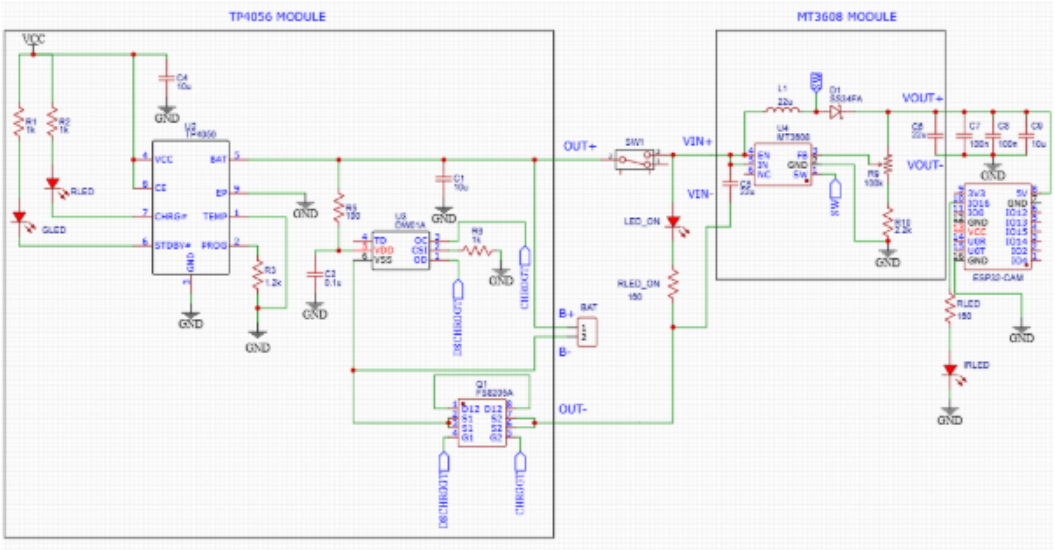


Figura 3.28: Esquema de conexiones para la PCB del circuito capturador

Incluye los módulos *TP4056*, *MT3608*, un switch intermedio entre ellos para desconectar manualmente el dispositivo y finalmente el zócalo donde se inserta la *ESP32*, al cual se le conecta el LED IR para normalizar la iluminación en el ojo.

Al tener los módulos comerciales, el esquemático final de la PCB se reduce a este simple esquema:

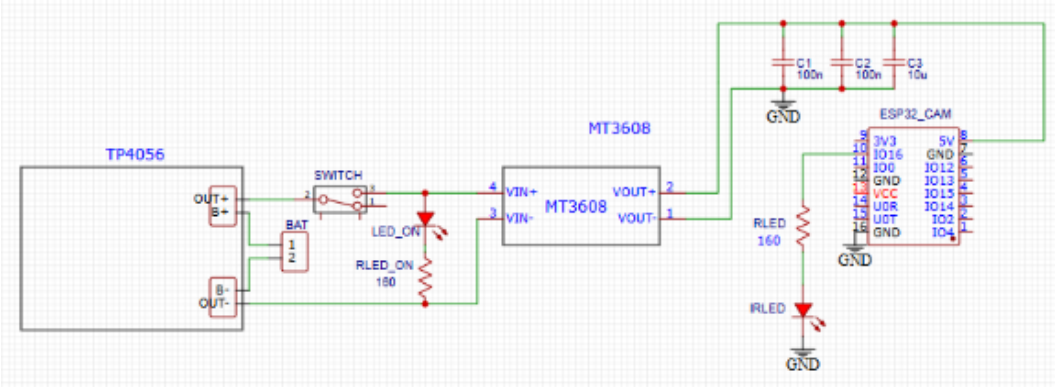


Figura 3.29: Esquema simplificado de la PCB del circuito capturador

La entrada del *TP4056* se conecta mediante USB-C, por lo que no se incluye en el esquemático de la PCB.

A su vez, el esquema de conexiones de la placa que actuará como servidor web es el siguiente:

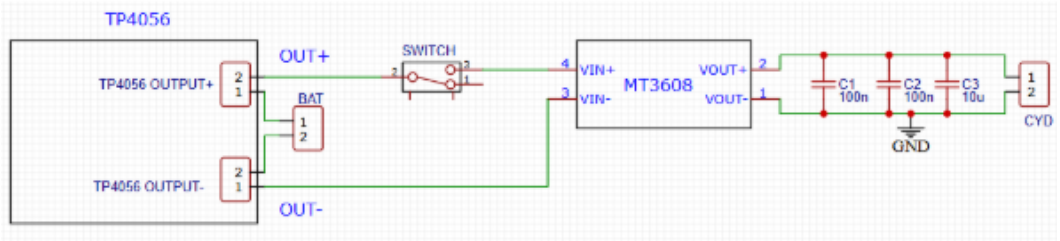


Figura 3.30: Esquema simplificado de la PCB del circuito servidor

Prácticamente igual al anterior, sin los diodos LED ni la *ESP32-CAM*. En su lugar se coloca una bornera doble de la que saldrán cables para alimentar a la CYD.

Diseño de PCB

A continuación se muestran imágenes del diseño de las PCBs para ambos dispositivos, diseñados en el software web *EasyEDA*.

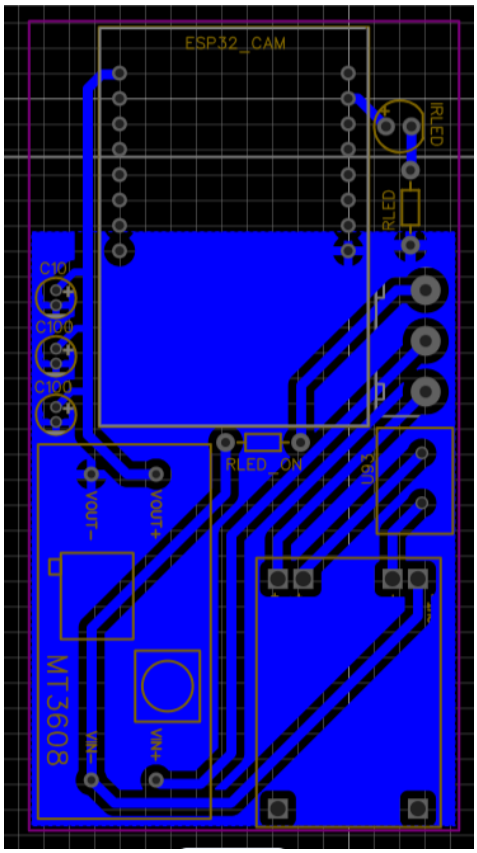


Figura 3.31: Layout de la PCB del circuito capturador

En lugar de colocar directamente el LED de encendido en la PCB, se opta por utilizar una bornera de 3 terminales para conectarlo externamente. Esto para poder decidir con más libertad su posición exacta en el dispositivo final.

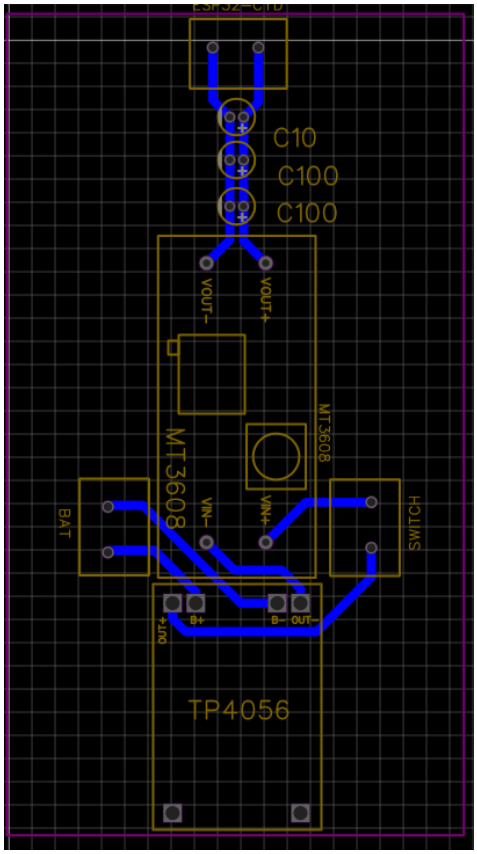


Figura 3.32: Layout de la PCB del circuito servidor

3.10. Costo total de desarrollo

(Tabla hecha por natty)

Capítulo 4

Conclusiones

4.1. Resultados obtenidos

Como resultado de este trabajo se obtuvo un dispositivo de sólido funcionamiento, cómodo, práctico y económico. Si bien no se cumplieron las expectativas iniciales de implementar el seguimiento ocular mediante redes neuronales, se encontró una alternativa matemática que resultó ser mucho más efectiva y eficiente. Este cambio de rumbo fue fundamental para lograr que un hardware tan limitado pudiera cumplir a la perfección su función de actuar como un sistema de evaluación de opción múltiple.

A lo largo del desarrollo, muchas de las ideas y procedimientos propuestos al principio tuvieron que ser descartados debido a problemas inevitables de precisión, latencia y disponibilidad de recursos. Es importante destacar la cantidad y variedad de áreas en las que se tuvo que profundizar para llegar a la solución definitiva: programación de sistemas embebidos, desarrollo de aplicaciones web, protocolos de comunicación (como SPI y Wi-Fi), diseño de redes y electrónica básica, entre otros conceptos que ya fueron expuestos a lo largo de este informe.

Fue justamente esta mezcla de conceptos y ámbitos tan distintos lo que permitió construir el sistema actual. Se logró crear una solución que gestiona usuarios y sesiones de forma segura, administra cuidadosamente la escasa memoria y potencia del procesador, aprovecha al máximo la comunicación inalámbrica aislada, ofrece interfaces de usuario interactivas y, lo más destacable: todo esto se consiguió utilizando componentes fácilmente accesibles, económicos y con gran disponibilidad en el mercado.

Por todo lo expuesto se puede concluir que, pese a que el dispositivo final acabó teniendo una funcionalidad técnica distinta a la propuesta originalmente y presenta mejoras a tratar en futuras versiones, el desarrollo fué un éxito. Se puede considerar un desarrollo satisfactorio especialmente si se tiene en cuenta la premisa social impuesta desde el primer día: se logró fabricar un artefacto de asistencia sumamente versátil, portable e intuitivo para cualquier tipo de usuario.

4.2. Comparación con productos similares

Como bien se puede deducir por lo expuesto en la sección anterior, este dispositivo no busca competir en potencia y velocidad de hardware con los equipos comerciales de detección ocular de gama alta. Al depender de los recursos limitados de un microprocesador sencillo, es lógico que sus especificaciones técnicas sean inferiores. Lo mismo ocurre con el software: al ser un programa embebido en un microcontrolador, no cuenta con un sistema operativo complejo ni con la infraestructura de ciberseguridad necesaria para una comercialización masiva a gran escala.

Sin embargo, como contrapartida a estas limitaciones técnicas, el proyecto presenta un punto fuerte fundamental frente a todos sus competidores: su precio y nivel de accesibilidad. Mientras que dispositivos como los *Tobii Glasses* tienen precios que rondan los miles de dólares, el costo total del hardware utilizado para construir este dispositivo fue de apenas unas decenas de dólares. Además, aquellos productos rara vez se encuentran disponibles en los mercados locales de la región desde donde se desarrolló este trabajo, lo que suma costos de envío e importación que dificultan aún más su adquisición.

Otra gran ventaja, si lo comparamos con alternativas de software puro como las de *EyeComTec*, es que este proyecto cuenta con una interfaz y funcionalidades diseñadas de manera exclusiva para el entorno académico y familiar del estudiante. El sistema fue pensado desde cero para crear y responder preguntas altamente personalizables, brindando comodidad a docentes y usuarios en general que no están familiarizados con tecnologías complejas, y evitando el tener que realizar configuraciones previas que suelen ralentizar el proceso.

En resumen, si bien no es un dispositivo pulido para competir en el mercado del hardware de alto rendimiento, cumple con creces su objetivo principal: ofrecer una experiencia de comunicación sencilla, intuitiva y a un precio realmente accesible para cada usuario.

4.3. Trabajo futuro